REMOTE & CLOUD

远程之门

SSH、堡垒机与对象存储的技术细节

THE REMOTE GATEWAY

Dr. Dezeming

2026

从一次 ssh 登录，到云端的数据存取

远程之门

SSH、堡垒机与对象存储的技术细节

Dr. Dezeming

2026

远程之门

版权

本书作者为 Dr. Dezeming。

书中出现的所有主机名、IP 地址、用户名、密钥文件名、桶名都是示例，用来演示配置的写法，不指向任何真实的服务器、账号或存储资源。请勿把示例直接用于生产环境。

书中涉及的命令与配置以 OpenSSH 9.x、Visual Studio Code 1.9x、JumpServer 3.x、腾讯云 COS 与 AWS S3 的公开接口为准。软件的版本更新可能导致界面与参数变化，遇到差异请以你所使用版本的官方文档为准。

本书的文字、插图与排版由作者完成。插图由配套代码生成，代码随书发布。未经许可，不得用于商业用途的再分发。

第一版 2026 年

前 言

十九岁那年我第一次登进机房里的一台服务器。屏幕上的提示符和我笔记本上的一模一样，可它背后那台机器在两百公里之外。那种错位感很久没有散去。我敲下的字符去了哪里，它凭什么把结果送回来，中间经过的那些设备看到了什么。

这件事在今天已经平常到不值得追问。写代码的人把程序放到云上跑，训练模型的人把几万张显卡当成一台机器用，做数据的人把几百 TB 的文件交给一个网址去管。计算离开桌面，是过去五十年里最安静也最彻底的一次变化，而支撑它的那些约定，几乎没有人从头学过。

计算是怎么走到别处的

最早的时候，计算机只有一台，放在机房里，谁要用就走到它面前。后来出现了分时系统，一台主机接上许多终端，每个人守着自己的键盘和纸带，看起来像是在独占机器。那时的终端没有算力，只负责显示和输入，真正干活的是远处那台大机器。远程访问的第一层意义在这里就定了型，你看到的东西和真正做事的东西，可以不在同一个地方。

再往后，终端从串口线换成了网络。主机可以躲在防火墙后面，用户可以在另一个城市。这时候出现了新的麻烦。数据要穿过许多不属于你的设备，密码和命令都是明文，任何一台中间机器都能抄一份。SSH 就是为这个麻烦发明的，它把通道加密，把身份验证做在加密之后，还顺手解决了文件传输和端口转发。

第三步是责任的外移。机器不再属于你所在的团队，它属于云厂商；数据不再放在自己的磁盘上，它存在对象存储里；运维不再由每个人各自进行，而是被收拢到一台专门的设备上，留下记录。技术问题于是变成了管理问题，谁能进、能做什么、做完留下什么证据，这些都需要机制来回答。

这三步留下的东西叠在一起，就是今天你面前的工具箱。终端里的 `ssh`，编辑器里的远程开发，公司要求必须经过的堡垒机，浏览器里那个像网盘一样的存储控制台。它们看起来属于四个不同的产品，其实是同一组问题在不同年代、不同层面的答案。

这套东西为什么难懂

因为这些机制被刻意藏在了里面。好的工具会把复杂度吃掉，只把最常用的那一条路铺平。你敲一条命令就能进服务器，点一个按钮就能连上远端项目，拖一个文件就能上传。操作越简单，藏在下面的东西越多。

麻烦在于，一旦你偏离了那条铺好的路，就完全没有线索。配置里多出一行看不懂的参数，报错里出现一个没见过的单词，连接在某个阶段停住不动，这些时刻没有任何提示告诉你该往哪里想。于是只能试，试对了算运气，换个环境又不灵。我自己的例子是端口。早年在机房给一台机器装驱动，网线插好，地址配好，`ping` 得通，`ssh` 就是连不上。折腾了半个多小时才发现对

端 sshd 监听在 2222，而我一直往 22 敲。那时候我并不知道端口是什么，只知道要加一个 `-p`。这种经验很脆，风一吹就散。

这类困惑有个共同点，它们都是机制在表层留下的痕迹。用户名里出现两个 at 符号，是堡垒机把三层身份压进了一个字段。`.pem` 和 `id_rsa` 名字不同，是两种编码格式和两种算法被混着叫了很多年。上传文件在浏览器里顺利、在命令行报 403，是签名算法对请求的每一个字节都较真。VS Code 连不上时要看哪段日志，取决于它把哪一部分搬到了远端。这些痕迹分布在不同的层，看不出联系，是因为中间那套结构没有露出来。

这本书要做的就是把那套结构露出来。不按产品讲，也不按操作清单讲，而是按问题讲。每个概念先说它要解决什么，再说它用什么办法解决，最后说它在你的终端、配置文件、报错信息里长什么样。这样安排的好处是，学到的不是几条命令，而是一张能自己往里填细节的地图。

读完之后，你应该能做到三件事。看懂别人给的配置，知道哪一行是关键、哪一行可以删。读懂报错，判断问题出在网络、认证、授权还是签名。判断一份方案是否合理，比如为什么某个端口不该暴露在公网，为什么某个桶不该开公共读。这三件事都不依赖记住具体参数，依赖的是理解机制。

这本书假设你会什么

假设你用过 Windows 或 macOS 的电脑，会在终端里敲几条命令，知道文件和目录的区别，写过程序，哪怕只是 Python 的入门水平。不假设你学过计算机网络，不假设你懂密码学，也不假设你管过 Linux 服务器。凡是需要的背景知识，书里都会从头讲起。

书里出现的每一条命令都可以直接运行，除了少数需要真实服务器的地方会用占位符代替。占位符会写成这样 〈你的用户名〉，需要你替换成自己的信息。

全书的组织方式

全书按四层推进，一层比一层远离键盘。

第一篇讲链路。一条命令从你的手指出发，经过网卡、路由器、运营商的骨干网、堡垒机，最后落到远端的一个进程上。这一篇把这条路上每一段的责任摊开，也把“连不上”这件事拆成可以逐段验证的判断题。

第二篇讲信任。链路通了只说明字节能过去，不说明对面是谁，也不说明中间有没有人偷看。密码学在这里不是装饰，而是 SSH 每一个设计决定的来源。这一篇会把对称加密、非对称加密、密钥交换、哈希、证书按需引入，然后完整走一遍 SSH 的握手、认证与隧道。

第三篇讲控制。一台机器、一个人的时候，信任可以靠熟人关系维持；几百台机器、几十个人的时候，就必须靠制度和技术。堡垒机把分散的凭据收拢，把每次操作录下来。VS Code 的远程开发则把开发环境整个搬到远端，只把界面留在手边。这两件事看起来无关，底下的机制是同一套。

第四篇讲数据。代码跑起来之后会产生数据，数据要存在别人的磁盘上，还要能被下一台机

器读到。对象存储的接口简单到只有几个 HTTP 方法，可用起来处处是坑，签名错一个字节就是 403。这一篇把数据模型、分片上传、签名算法与权限体系讲透，最后把四层串成一条完整的链路。

关于版本与命令

书里的命令以 OpenSSH 9.x、VS Code 1.9x、JumpServer 3.x、腾讯云 COS 与 AWS S3 的公开接口为准。这些软件都在持续演进，算法默认值、界面文字、命令行参数都可能变化。遇到不一致时，先看 `--help` 和你所用版本的官方文档，再用书里的原理去判断差异。原理的寿命比参数长得多。

书中的服务器地址、用户名、密钥文件名都是示例，不指向任何真实资产。你自己环境里的私钥、密码、访问密钥，任何时候都不要写进文档、截图或者聊天窗口。

Dr. Dezeming

2026 年于实验室

目 录

前 言 5

计算是怎么走到别处的 5

这套东西为什么难懂 5

这本书假设你会什么 6

全书的组织方式 6

关于版本与命令 7

第 1 章 你按下回车之后 15

1.1 一份照着填的配置 15

1.2 那条命令交给了谁 16

1.3 四段路与三个身份 16

1.4 把复杂拆成几层 17

1.5 一次连接的完整时间线 18

1.6 怎么看见这些步骤 19

1.7 后面要解决的问题 19

第 2 章 地址、端口与那条看不见的路 21

2.1 一台机器在网上叫什么 21

2.2 端口是同一台机器上的不同门 21

2.3 NAT 与地址改写 22

2.4 域名只是给人看的 23

2.5 TCP 保证了什么 23

2.6 数据是怎么被装进包里的 24

2.7 自己动手看连接 25

2.8 常见端口一览 26

第 3 章 服务器上发生了什么 27

3.1 登录成功那一刻 27

3.2 用户、组与权限 28

3.3 该记住的几个路径 29

3.4 伪终端是什么 29

3.5 环境变量与 PATH 30

3.6 常驻的服务 30

3.7 日志里能看到什么 31

第 4 章 把话藏起来 33

4.1 三种敌人 33

4.2 一把钥匙的对称加密 33

4.3 哈希与消息认证码 34

4.4 非对称加密的两个用途 34

4.5 密钥交换 35

4.6 为什么需要证书 37

4.7 前向保密 37

4.8 这些工具在 SSH 里怎么分工 37

第 5 章 SSH 协议的三层结构 39

5.1 一行明文的版本号 39

5.2 三个子协议 39

5.3 一个数据包长什么样 40

5.4 算法协商 41

5.5 密钥交换与密钥派生 42

5.6 重密钥 44

5.7 认证层 44

5.8 连接层 45

5.9 看看自己的机器支持什么 45

5.10 抓包能看到什么 45

第 6 章 密钥、指纹与信任 47

6.1 生成一对钥匙 47

6.2 私钥文件里装的是什么 47

6.3 公钥认证到底怎么验证 48

6.4 authorized_keys 的写法 49

6.5 主机密钥与首次信任 50

6.6 ssh-agent 与密钥转发 51

6.7 证书 51

6.8 一份能用的客户端配置 52

第 7 章 端口转发，把远方的服务搬到本地 54

7.1 只开一个端口的世界 54

7.2 本地转发 54

7.3 远程转发 55

7.4 动态转发 56

7.5 跳板与 ProxyJump 57

7.6 转发与堡垒机的关系 58

7.7 让隧道稳定下来 58

7.8 一条通道能装多少东西 59

第 8 章 为什么需要堡垒机 61

8.1 一台没人记得动过的机器 61
8.2 网状连接的代价 61
8.3 4A 62
8.4 流量怎么走 63
8.5 合规要求 64
8.6 堡垒机保护不了什么 64

第 9 章 JumpServer 的内部构造 65

9.1 组件地图 65
9.2 资产模型的四层 66
9.3 三合一用户名 67
9.4 一次 SSH 网关会话 68
9.5 命令过滤与会话录像 68
9.6 凭据是怎么存的 69
9.7 部署时的几个关键项 69

第 10 章 VS Code 是怎么连过去的 70

10.1 一个编辑器，四个进程 70
10.2 连接时发生了什么 71
10.3 那条隧道的两端 72
10.4 ssh_config 的解析规则 72
10.5 逐行拆解那份配置 73
10.6 出问题时看哪里 74

第 11 章 对象存储的模型 77

11.1 为什么不用文件夹 77
11.2 三种存储形态 78

11.3 接口就是 HTTP 79
11.4 大文件怎么传 79
11.5 元数据与存储类型 80
11.6 一致性与持久性 81
11.7 工具与挂载 81
11.8 和堡垒机的关系 81

第 12 章 签名、密钥与 403 83

12.1 每个请求都要证明自己 83
12.2 访问密钥 83
12.3 签名的计算过程 83
12.4 预签名 URL 86
12.5 临时密钥 86
12.6 权限是怎么判的 87
12.7 403 家族 88
12.8 几条实践建议 88

第 13 章 把四件事串成一条链路 90

13.1 一个真实的工作流 90
13.2 第一步，把登录打通 91
13.3 第二步，让编辑器连上 92
13.4 第三步，让训练机读写桶 92
13.5 第四步，把结果取回来 93
13.6 排错的时候按顺序来 94
13.7 一份可以照着做的检查清单 94
13.8 最后 95

附录 A 命令速查 96

A.1 客户端连接 96

A.2 密钥管理 96

A.3 agent 97

A.4 诊断 97

A.5 服务端 97

A.6 对象存储 98

A.7 常用配置项 98

附录 B 术语表 100

后 记 103

第 一 篇

把远方接到手边

一台服务器放在机房里，或者放在云上某个你从未去过的地方。它有自己的处理器、内存、磁盘和操作系统，但没有屏幕，没有键盘。你想让它做事，就得把指令送过去，把结果取回来。

这件事在今天看起来理所当然。可在计算机刚出现的年代，人们要带着一叠打孔卡片走到机房，交给操作员，等上几个小时才能拿到打印结果。后来有了分时系统，多个终端通过串口连到同一台主机，每个人守着自己的电传打字机。再后来有了网络，终端可以是几百公里外的一台电脑。今天你笔记本上那个黑色的窗口，本质上和五十年前的电传打字机在做同一件事，只是线路从一根铜缆变成了跨越若干运营商的光纤。

这一篇要把这条线路讲清楚。不是抽象地讲，而是跟着你敲下的一条命令，一段一段走过去。走到最后你会发现，所谓远程登录并没有魔法，只有几层约定俗成的协议，每一层负责一件事，出了问题时也各自留下自己的痕迹。知道这些痕迹在哪里，排错就不再是碰运气。

第 1 章

你按下回车之后

今天写代码的人，越来越少在自己面前的机器上干活。编译在远端的服务器上跑，训练在机房的集群里排队，数据放在一个只有网址的存储里，连编辑器都只负责画界面，真正读写文件的是几百公里外的另一个进程。计算被拆成了两半，一半在你眼前，一半在别处。

一旦拆开，就必须有一条通路把它们连起来。这条通路要回答三个问题。指令怎么送过去，对面凭什么相信发指令的人是你，出了事又怎么知道当时发生了什么。这三个问题听起来朴素，却是几十年里无数设计取舍的来源。SSH 回答第一个和第二个，堡垒机回答第二个和第三个，对象存储则用另一套办法回答“谁在动我的数据”。

这一章先不做解答，只做一件事，把一条命令的完整旅程走一遍。看清楚这条路上有哪些角色、分几段、每段负责什么。后面的章节会把每一段放大，直到你能自己判断问题出在哪里。

1.1 一份照着填的配置

先说一段我自己的经历，它适合用来说明背景知识缺位时会发生什么。几年前，一位做算法的朋友发给我一份配置指南，说照着填就能用 VS Code 连上实验室的机器。指南不长，核心就是配置文件里的两段：

```
Host l40-server
    HostName 124.221.133.165
    User alice@ubuntu@119.45.34.99
    Port 58222
    IdentityFile C:/Users/alice/.ssh/ssh.jumpserver.pem
```

我照抄进去，把 `alice` 换成自己的账号，居然连上了。但随后冒出来一串问题。既然要连的是 `119.45.34.99`，为什么 `HostName` 写的是另一个地址。用户名里为什么有两个 at 符号，Linux 用户名不是不允许这种字符吗。端口 58222 是谁的端口，目标机上明明开着 22。`IdentityFile` 指向的 `.pem` 文件，和常见教程里的 `id_rsa` 又是什么关系。

这些问题在当时都没有答案。我照着做，能用，但一旦连不上，就完全不知道该动哪一行。后来花了不少时间把这些机制补上，才发现那份指南里的每一行都有明确的来历，而它们之所以看起来奇怪，是因为中间多了一台你平时看不见的机器。这类“照着填能用、离开指南就废”的处境，几乎都来自同一个原因，缺的不是操作步骤，是操作背后的结构。

1.2 那条命令交给了谁

在终端里敲 `ssh 140-server` 然后回车，最先动起来的是一个叫 `ssh` 的程序。它属于 OpenSSH 客户端，Windows 10 以后的系统自带，Linux 和 macOS 也自带。这个程序做的事情按顺序是下面几步。

它先读命令行参数，再去读配置文件 `~/.ssh/config`，把 `140-server` 这个名字展开成一堆具体参数，包括真实地址、端口、用户名和私钥路径。然后它向那个地址的端口发起一条 TCP 连接。连接建立之后，双方开始协商加密方式、交换密钥、验证身份。等这一切都完成了，它才把你的键盘输入打包发出去，把对方的输出解包显示在屏幕上。

这条链路上，你自己的电脑里其实有三个角色在配合。最外面是你看到的终端窗口，负责显示字符、接收按键。中间是 `ssh` 客户端进程，负责协议。最底下是操作系统的网络栈，负责把字节变成电信号或者光信号送上网线。三者分工明确，所以出问题时可以分别检查。窗口没反应，可能是客户端卡住了；客户端报连接超时，那就是网络栈之下的事情。

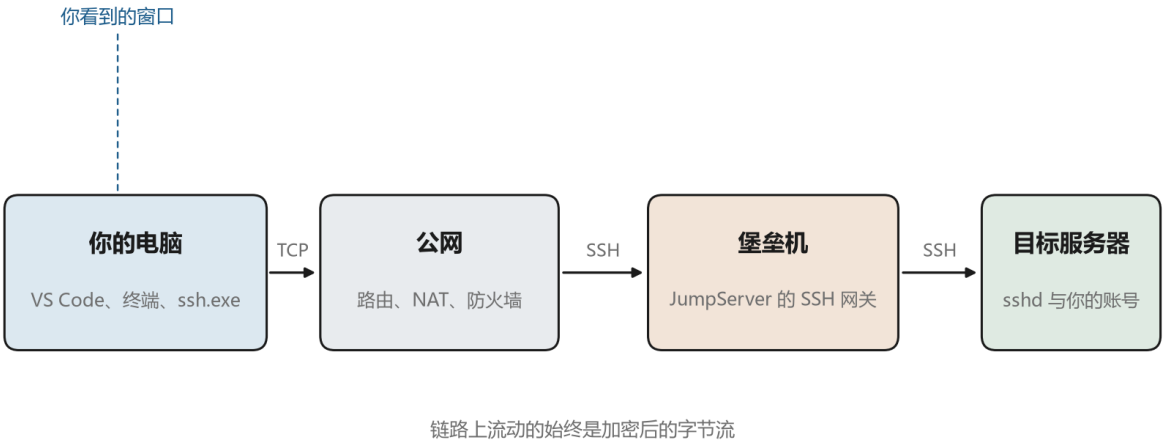


图 1-1 一次远程登录经过的四段路径，以及每段路上跑的东西

1.3 四段路与三个身份

从你的电脑到目标服务器，中间通常隔着三段路，如图 1-1。第一段从你的电脑到公网，经过家里的路由器或者公司的出口网关。第二段在公网里穿行，经过若干台你完全不知道存在与否的路由器。第三段落在堡垒机上，它接受你的连接，然后再替你向目标机发起一条新的连接。

这段结构解释了指南里那个奇怪的配置。`HostName` 填的是堡垒机的地址，因为你的电脑能够直接访问的只有它。目标机的地址 `119.45.34.99` 被写进了 `User` 字段，因为堡垒机需要用这个信息找到终点。端口 58222 也是堡垒机的端口，目标机上那个 22 端口由堡垒机去连，你在本机根本碰不到。

同样地，这条链路上有三个不同的身份在起作用。本机上的 Windows 账户决定了你的文件放在哪里、`~/.ssh` 目录属于谁。堡垒机上有一个你自己的账户，用来登录堡垒机、接受审计。目标机上还有一个 Linux 账户，那是真正执行命令的身份。指南里的

`alice@ubuntu@119.45.34.99` 把后两个身份和终点地址拼成了一串，交给堡垒机去解析。第三章和第九章会把这套约定拆开细讲，现在只需要记住，一串字符里塞进了三层信息，看起来别扭是正常的。

这里还有一个容易忽略的事实。从你的电脑到堡垒机的那一段是加密的，从堡垒机到目标机的那一段也是加密的，但这是两条独立的加密连接。堡垒机在两段之间解密再加密，所以它在理论上能看到你输入的每一个字符。这正是它在企业里存在的原因，也是它的风险所在。

1.4 把复杂拆成几层

网络协议几乎都采用分层设计。分层的好处在于，每一层只需要关心自己那点事，上层不必知道下层用什么介质传输，下层也不必知道上层在传什么内容。图 1-2 是这条链路上涉及的层次。

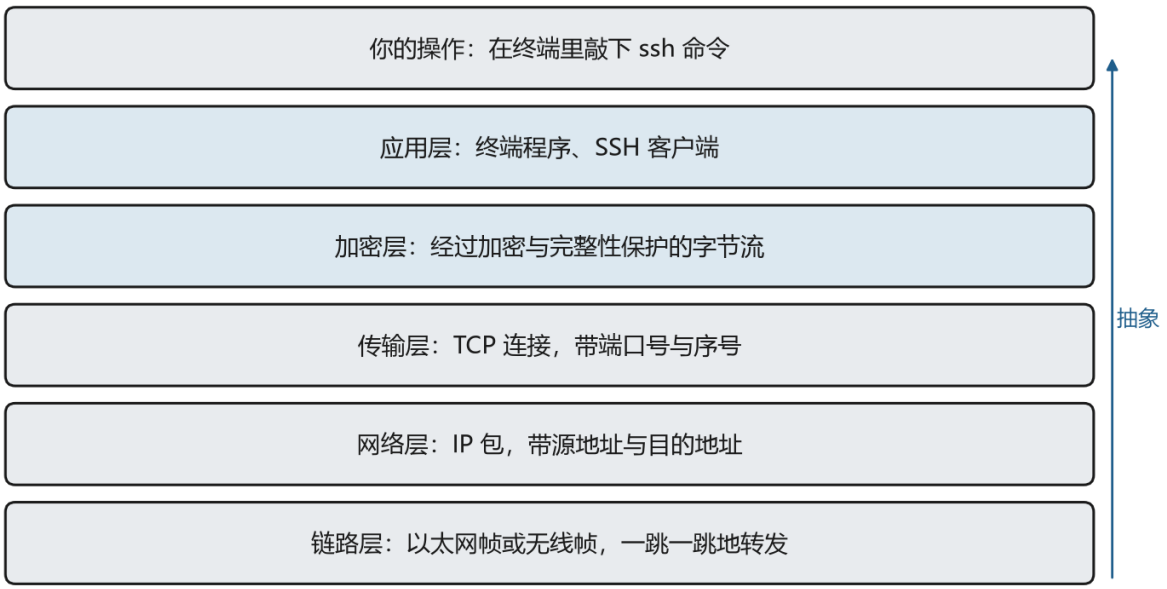


图 1-2 从人的操作到物理线路之间的层次

最上面是你的意图，你想在远端执行一条命令。往下一层是应用程序，终端和 SSH 客户端都属于这一层。再往下是加密层，SSH 在这一层把明文变成密文，并加上完整性校验。然后是传输层，TCP 负责把字节流可靠地送到对端，处理丢包、重传、顺序。再往下是网络层，IP 负责寻址和路由，让数据包能跨越多个网络。最底下是链路层，负责在一段物理介质上把帧送出去。

分层不是为了好看，而是为了定位问题。当 `ssh` 报“连接超时”，问题多半在网络层或链路层，因为 TCP 连握手包都没送到。当它报“Permission denied”，说明网络一路通畅，问题出在认证层。当它连上了却卡在“Entering interactive session”，问题可能出在连接层的通道建立或者服务端的 shell 启动。知道每一层负责什么，报错信息就从天书变成了路标。

1.5 一次连接的完整时间线

把上面这些串起来，一次成功的登录大致经过图 1-3 里的几步。



图 1-3 从 TCP 握手到拿到 shell 的时间线

第一步是 TCP 三次握手，这一步和 SSH 没有关系，任何基于 TCP 的服务都要走。握手成功后，双方各自发送一行版本字符串，形如 `SSH-2.0-OpenSSH_9.6`。这一行是明文，也是整条连接里唯一一定为明文的部分。接着双方各自列出自己支持的算法清单，从中挑出一套双方都支持的组合。再往后是密钥交换，双方通过数学运算协商出一个只有彼此知道的会话密钥。密钥算好之后，各自发送一个 NEWKEYS 消息，从这一刻起，后面的所有内容都加密。

加密通道建立之后才开始认证。这个顺序常被误解，很多人以为先验证身份再加密，实际上 SSH 是先建立加密通道，再在通道里验证身份。这样做的好处是，密码和公钥签名都不会以明文出现在网络上。认证成功后，客户端请求打开一个会话通道，服务端分配一个伪终端并启动 shell，你才看到那个熟悉的提示符。

表 1-1 把各阶段和常见的失败表现对应起来。这张表在第十三章的排错部分还会再用一次。

表 1-1 连接阶段与典型故障对照

阶段	你看到的现象	问题通常在哪里
TCP 握手	`Connection timed out`	地址错误、端口未开放、防火墙丢包

阶段	你看到的现象	问题通常在哪里
TCP 握手	`Connection refused`	对端没有程序监听这个端口
版本交换	`Connection closed by remote host`	服务端限制、封禁、协议不匹配
算法协商	`no matching key exchange method found`	客户端与服务端没有共同算法
主机密钥	`Host key verification failed`	指纹与 known_hosts 记录不一致
身份认证	`Permission denied (publickey)`	私钥不对、公钥未授权、权限过宽
通道建立	卡住不动或者登录后立刻断开	shell 启动失败、磁盘满、配额限制

1.6 怎么看见这些步骤

SSH 客户端自带调试输出，这是学习协议最方便的工具。在命令后面加 `-v` 会打印主要步骤，加 `-vv` 和 `-vvv` 会越来越详细。

```
ssh -vv l40-server 2>&1 | head -60
```

输出里每一行都以 `debug1` 或 `debug2` 开头，后面跟着这条信息属于哪个模块。你会依次看到连接建立、本地版本号、远端版本号、双方算法列表、密钥交换结果、可用的认证方式、认证尝试、通道打开。把这几十行读上三五遍，图 1-3 里的每个阶段就都能对上号了。

同样的信息在服务端也有一份。Linux 上 sshd 的日志通常在 `/var/log/auth.log`（Debian 系）或 `/var/log/secure`（Red Hat 系），用 systemd 的系统还可以用 `journalctl -u ssh` 查看。排查连接问题时，客户端和服务端的日志要对照着看，客户端说“我发了”，服务端说“我没收到”，问题就在中间那一段。

还有一个细节值得早点知道。调试输出里会打印主机密钥的指纹，形如 `ED25519 key fingerprint is SHA256:...`。这个字符串是你判断“对面是不是真机器”的唯一依据，第六章会专门讲它。现在只要记住，第一次连接时出现的那个 yes/no 提问不是走过场。

1.7 后面要解决的问题

这一章只给了一张地图，还没有进入任何一层的细节。第二章从最底下的地址与端口讲起，把“为什么连不上”变成可以逐段验证的判断题。第三章走到服务器内部，看看登录成功之后究竟是什么在等你。第四章和第五章进入密码学与 SSH 协议本身，解释那些算法名字背后的取舍。第六章到第七章处理密钥、指纹与隧道，把配置文件里剩下的几行讲完。

第三篇会换一个角度。前面讲的都是“一个人连一台机器”，而现实里往往是一群人连几百台机器，中间还要留下记录。堡垒机就是为这个场景发明的。第四篇则离开终端，去看数据存放的那一端，对象存储的接口看起来简单，签名与权限却能让人反复碰壁。

读完之后再回头看那份配置指南，你应该能一眼看出哪一行是关键，哪一行可以删掉，以及连不上时该先试什么。

第 2 章

地址、端口与那条看不见的路

2.1 一台机器在网上叫什么

要在网络上找一台机器，需要一个地址。今天最常见的地址形如 `124.221.133.165`，四组数字，每组 0 到 255，这叫 IPv4 地址。它长 32 位，理论上可以提供约 43 亿个地址。上世纪八十年代设计这套体系的时候，没人认为 43 亿会不够用，可后来的事情大家都知道了，手机、电脑、摄像头、汽车都要联网，地址很快就不够分了。

解决办法有两个。一个是新版本的地址格式 IPv6，长 128 位，写成带冒号的一串十六进制，多到几乎用不完，但你日常接触的服务器大多仍然用 IPv4。另一个是复用，也就是让很多设备共用一个公网地址，私底下各自用一套只在内部有效的地址。这套内部地址就是私有地址，`192.168.x.x`、`10.x.x.x` 和 `172.16.x.x` 到 `172.31.x.x` 三段属于此列。

私有地址有一个重要性质，它在公网上不可路由。也就是说，你家里那台 `192.168.1.23` 的电脑，从外面的服务器是没法直接连过去的，因为全世界有几千万台机器都在用这个地址，路由器不知道该送到哪一家。这个性质看起来是个限制，实际上是一层保护。多数人的电脑不需要被外部主动连接。

而你租用的云服务器不一样。它有一个公网地址，任何人都能向它发起连接。这也是为什么云主机一开通就要立刻配置安全组和防火墙，把不需要的端口关掉。

2.2 端口是同一台机器上的不同门

一台机器上可能同时跑着网页服务、数据库、SSH 服务。数据包到了这台机器，操作系统要知道该交给哪个程序。这个区分工作由端口号完成。端口是一个 16 位整数，范围 0 到 65535。IP 地址定位到机器，端口定位到机器上的程序，两者合起来叫套接字地址，写成 `124.221.133.165:22` 这种形式。

端口号被分成几段使用。0 到 1023 是知名端口，只有特权程序才能监听，比如 SSH 的 22、HTTP 的 80、HTTPS 的 443。1024 到 49151 是注册端口，各种服务按惯例占用，比如 MySQL 的 3306、Redis 的 6379。再往上是临时端口，操作系统在发起连接时随机分配，用完之后回收。你家里的电脑连出去时用的就是临时端口，所以同一个网站会看到你每次连接的源端口都不一样。

这里有一个必须分清的概念，监听和连接是两件事。服务器上的 sshd 在 22 端口监听，等待别人来连。客户端连出去的时候，本地会随机选一个临时端口作为源端口，目标端口才是 22。一条 TCP 连接由四个要素唯一确定，源地址、源端口、目的地址、目的端口。四要素中任何一项不同，就是另一条连接。

2.3 NAT 与地址改写

私有地址的设备要访问公网，需要中间有一台设备帮它做地址转换，这就是 NAT，网络地址转换。图 2-1 画的是这个过程的简化版本。

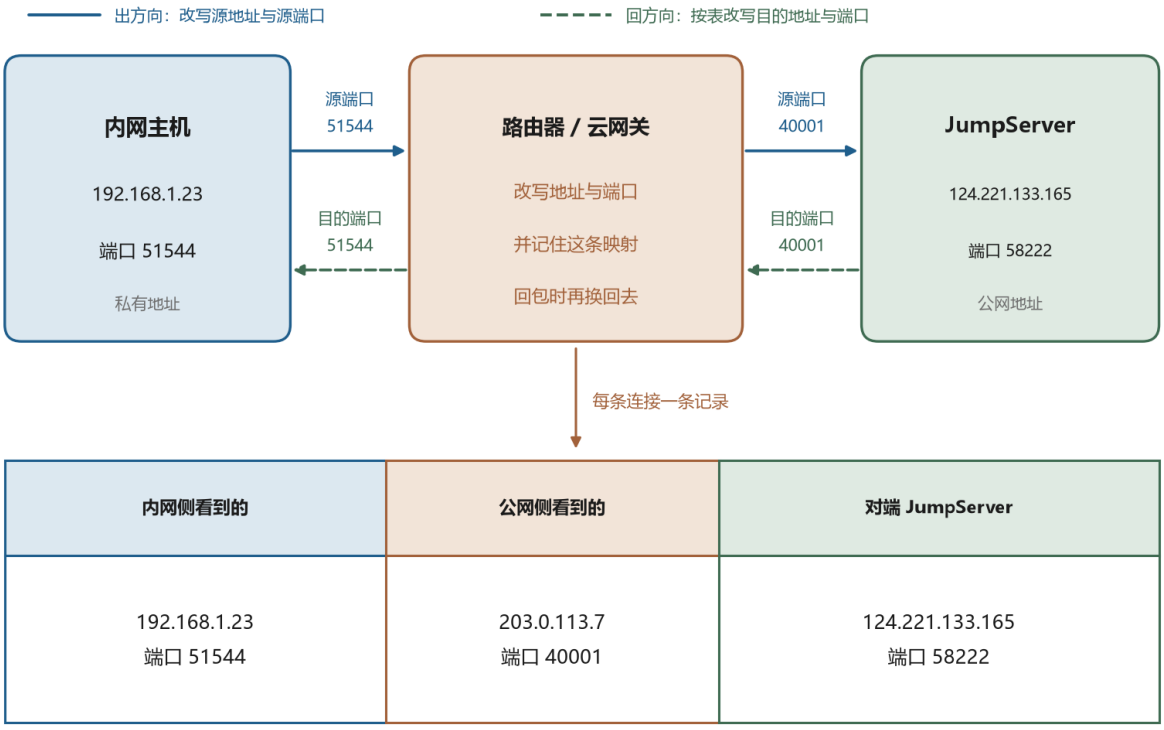


图 2-1 NAT 把内网地址改写成公网地址并记住对应关系

内网主机 `192.168.1.23` 用源端口 51544 访问 `124.221.133.165` 的 58222 端口。数据包经过网关时，网关把源地址换成自己的公网地址 `203.0.113.7`，同时把源端口换成一个它自己管理的端口，比如 40001，并在内存里记下一条映射。回包到达网关时，目的地址是 `203.0.113.7:40001`，网关查表把它改回 `192.168.1.23:51544`，再送进内网。

这个机制解释了两件常见的事。第一，从服务器端看到的客户端地址是网关的地址，不是你的真实地址，所以基于 IP 的访问控制经常失效，一家公司几百人可能共用一个出口地址。第二，外部无法主动向内网发起连接，因为网关的表里没有对应的条目。要让外部连进来，必须在网关上手工配置端口映射，或者用第七章讲的隧道技术，从内部主动把通道建立出去。

2.4 域名只是给人看的

IP 地址难记，于是有了域名。`jumpserver.example.com` 这样的名字由 DNS 系统翻译成地址。翻译过程大致是，你的系统先查本地缓存，再问配置的 DNS 服务器，DNS 服务器如果不知道就层层向上问，最后把结果返回并缓存一段时间。缓存时间由记录的 TTL 决定，这也是为什么改了域名解析之后，有些地方要等一会儿才生效。

对远程登录来说，域名带来一个额外的好处。你可以把同一个域名指向不同的机器，机器换了地址，配置文件不用改。云厂商的负载均衡和故障转移也依赖这一点。缺点是引入了新的失败点，DNS 解析不出来的时候，看起来就像网络不通。第一章提到的 `ssh -vv` 输出里会有一行 `debug1: Connecting to ...`，那里显示的是解析之后的地址，如果域名有问题，这一步之前就会报 `Could not resolve hostname`。

2.5 TCP 保证了什么

IP 层只负责把包送到，不保证顺序，不保证不丢，也不保证不重复。SSH 需要的是一个可靠的字节流，也就是“我发出去的字节，按顺序、不重不漏地到达对端”。这个保证由 TCP 提供。

TCP 的做法是给每个字节编号。发送方发出数据时带上序号，接收方收到后回一个确认，告诉对方“我已经收到第 N 个字节之前的全部内容”。发送方如果在一段时间内没收到确认，就重发。接收方如果收到乱序的包，先缓存起来，等前面的缺口补上再交给应用。这套机制里还有流量控制和拥塞控制，前者防止发送太快把接收方撑爆，后者防止把网络压垮。

建立连接要三次握手，断开要四次挥手，如图 2-2。

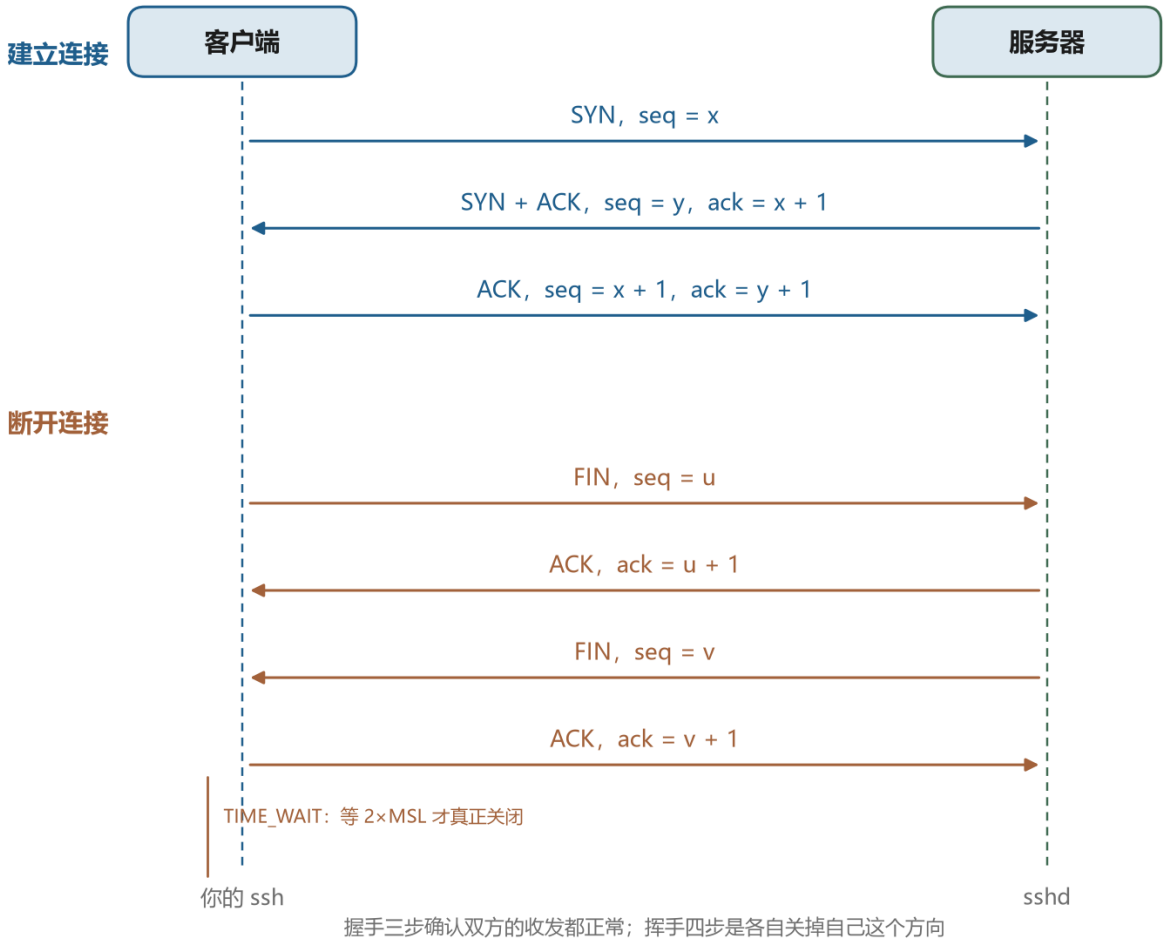


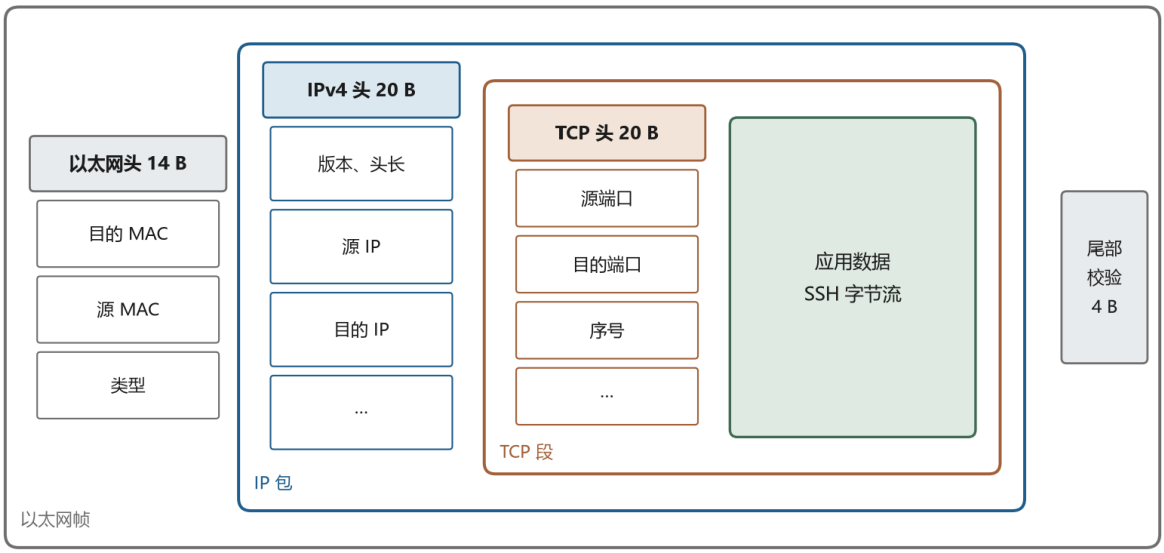
图 2-2 TCP 的连接建立与断开

三次握手的实质是双方各自确认两件事，我能发出去，你能收到。第一次客户端发 SYN，第二次服务端回 SYN 加 ACK，第三次客户端再回 ACK。为什么不是两次，因为两次只能让客户端确认自己的发送能力，服务端无法确认客户端是否收到了自己的包。断开时的四次挥手是因为 TCP 连接是全双工的，两个方向要各自关闭。主动关闭的一方最后会进入 TIME_WAIT 状态，等一段时间确保最后的确认不会丢失。

这些细节和日常使用有什么关系。当你看到 `Connection timed out`，说明 SYN 发出去没有回应，可能是地址不可达或者被防火墙静默丢弃。看到 `Connection refused`，说明 SYN 到了，但对方回了一个拒绝包，通常是那个端口上没有程序在监听。两种错误的含义完全不同，处理方式也不同，前者要去查网络路径和防火墙规则，后者要去查服务是否启动。

2.6 数据是怎么被装进包里的

应用层的数据要一层层加上头部才能上路。图 2-3 画的是这个过程。



从外到内依次套住：以太网帧、IP 包、TCP 段、应用数据；每层只在前方加一个头部，帧尾再加 4 字节校验

图 2-3 从应用数据到以太网帧的四层封装

SSH 客户端产生的字节先交给 TCP，TCP 加上源端口、目的端口、序号、校验和等等，形成 TCP 段。TCP 段交给 IP，IP 加上源地址、目的地址、生存时间等，形成 IP 包。IP 包交给链路层，加上源 MAC、目的 MAC 和帧校验，形成以太网帧，最后变成线路上的电信号。

每一层加的头部都有固定长度。TCP 头部一般是 20 字节，选项会让它变长。IPv4 头部也是 20 字节。以太网帧的头部 14 字节，尾部校验 4 字节。这些数字在排查 MTU 问题时会用到。链路层能承载的单个帧有大小上限，通常是 1500 字节，减去各层头部，留给应用的有效载荷大约 1460 字节。如果某一层的数据超过了链路的上限，就要分片，而分片会让丢包的概率上升，也会让某些防火墙处理起来更费劲。

SSH 在建立连接时会自己控制包的大小，通常不会主动制造分片。但如果经过隧道或者 VPN，外层又套了一层头部，可用空间就变小了，可能出现“能连上但传大文件卡住”的怪现象。这种情况在第七章讲隧道时会再提一次。

2.7 自己动手看连接

理论讲完，需要一些工具把状态看清楚。Linux 上最常用的是 `ss`，老一些的系统用 `netstat`。

```
ss -lntp            # 列出所有 TCP 监听端口和对应进程
ss -tnp             # 列出当前已建立的 TCP 连接
ss -tnp state established '( dport = :22 )'   # 只看连到 22 端口的连接
```

`-l` 表示只看监听，`-n` 表示不解析服务名，`-t` 表示 TCP，`-p` 显示进程。第一行输出能告诉你这台机器上有哪些服务在等别人连接，第二行输出能告诉你此刻谁连着谁。排障时这两个视角缺一不可。

Windows 上的对应命令是 `netstat -ano`，加上 `| findstr :22` 过滤。想知道某个端口通不通，可以用 `Test-NetConnection`，Linux 上可以用 `nc -vz host port` 或者 `curl -v telnet://host:port`。

```
Test-NetConnection 124.221.133.165 -Port 58222
```

还有一个常被忽略的工具是 `traceroute`（Windows 上叫 `tracert`）。它逐跳显示数据包经过的路由器，能看出是在哪一段断掉的。如果是公司网络内部就不通，那和服务器没关系；如果出了公网才断，就要找运营商或者云厂商的安全组。

2.8 常见端口一览

表 2-1 列出这本书里会反复出现的端口。记住其中几个，读配置文件时会轻松很多。

表 2-1 书中涉及的常见端口

端口	服务	说明
22	SSH	OpenSSH 默认端口，公网上被扫描得最频繁的端口之一
2222	SSH 备用端口	常用来避开针对 22 的暴力破解
58222	JumpServer 网关	部署时自定义的端口，本书示例中使用
80	HTTP	明文网页，多数站点会重定向到 443
443	HTTPS	TLS 加密的网页，对象存储的 API 也走这里
3306	MySQL	数据库，绝不应该直接暴露在公网
6379	Redis	缓存，默认无认证，暴露在公网等于把数据送人
1080	SOCKS	第七章动态转发时本地监听的端口

这张表里第 22 和第 58222 两行值得多看两眼。它们说明了一件事，端口号本身没有任何魔力，它只是双方约定好的一个数字。把 SSH 挪到别的端口，安全性并不会因此提高多少，因为端口扫描很容易发现它，但能挡掉大量无差别攻击的噪音，让日志干净一些。真正决定安全的是密钥长度、认证方式和访问控制，这些是第二篇的主题。

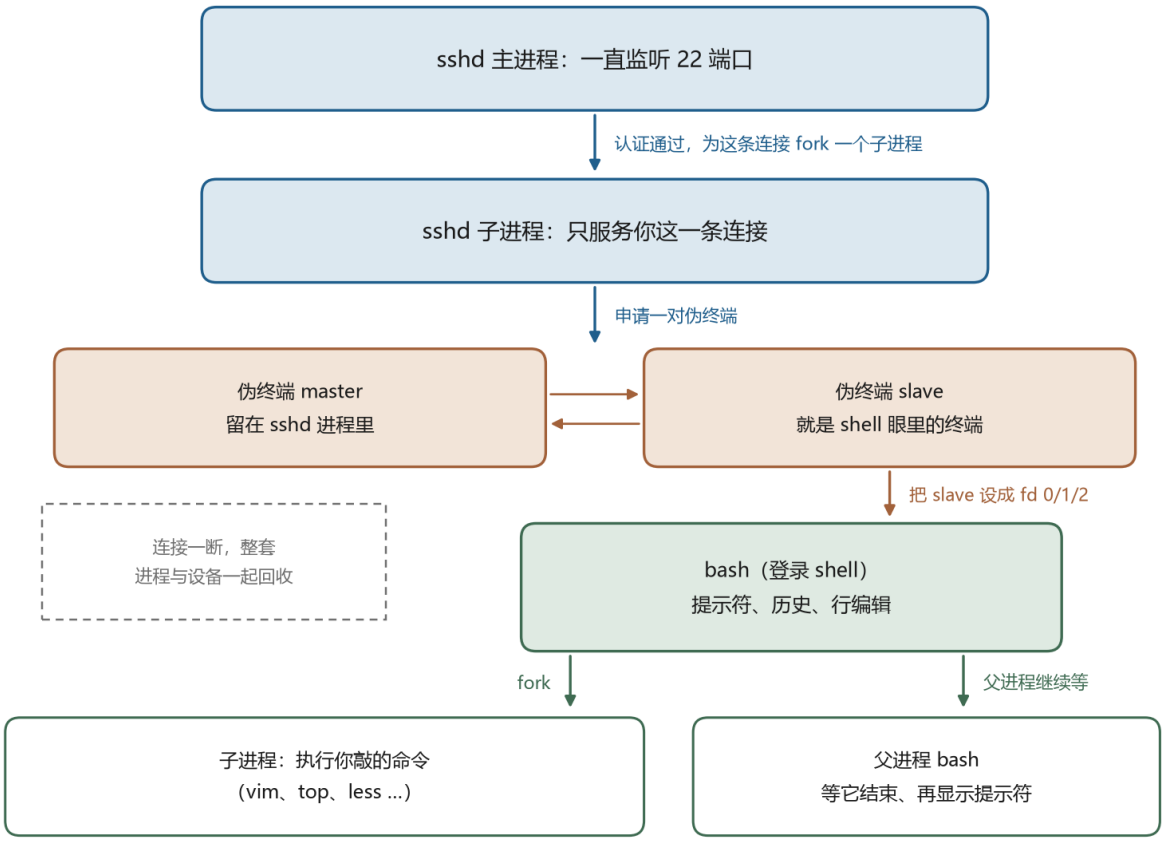
第 3 章

服务器上发生了什么

3.1 登录成功那一刻

认证通过之后，sshd 并不会把当前的进程直接交给你，而是做三件事。它先创建一个伪终端设备，把你的输入输出接到这个设备上。然后 fork 出一个子进程，在子进程里切换成你的用户身份，加载你的登录环境。最后在子进程里执行你的登录 shell，通常写在 `/etc/passwd` 的最后一段，比如 `/bin/bash`。

所以你在远端看到的那个提示符，背后是一条这样的链，sshd 在 22 端口监听，收到连接后 fork 出一个子进程处理这次会话，子进程认证通过后又 fork 出 shell，shell 再 fork 出你敲的每一条命令。这条链上的每个进程都有自己的进程号、自己的环境变量、自己的文件描述符。理解这一点，很多现象就顺了。比如你关掉终端窗口，shell 收到挂断信号退出，但它启动的后台任务可能继续跑，也可能跟着退出，取决于它们有没有脱离终端。



你看到的“终端”其实是内核里的一对设备：写 master，slave 那头就能读到

标准输入、标准输出、标准错误（fd 0/1/2）全部指向 slave

图 3-1 一次登录在服务器上生成的进程与伪终端

3.2 用户、组与权限

Linux 用数字标识用户，这个数字叫 UID。用户名只是给人看的别名，系统内部认的是数字。每个用户至少属于一个组，组也有数字标识，叫 GID。`/etc/passwd` 记录用户名、UID、GID、家目录和登录 shell，`/etc/group` 记录组的成员。

```
id                    # 我是谁，属于哪些组
whoami                # 只输出用户名
getent passwd alice   # 查某个用户的记录
```

文件权限用九个二进制位表示，分成三组，分别是属主、属组和其他人的读、写、执行权限。`ls -l` 输出的第一列就是这九个位，前面还有一个字符表示文件类型，`-` 是普通文件，`d` 是目录，`l` 是符号链接。

```
-rw-r--r--  1 alice dev  1204 Nov  4 10:12 notes.txt
drwxr-x---  2 alice dev  4096 Nov  4 10:12 project
```

第一行表示属主可读写，属组可读，其他人可读。换成八进制是 644。第二行表示属主可读写执行，属组可读执行，其他人什么都没有，八进制是 750。目录的执行权限含义特殊，它表示“能否进入这个目录”，没有它就无法访问目录里的任何文件，哪怕文件本身是 755。

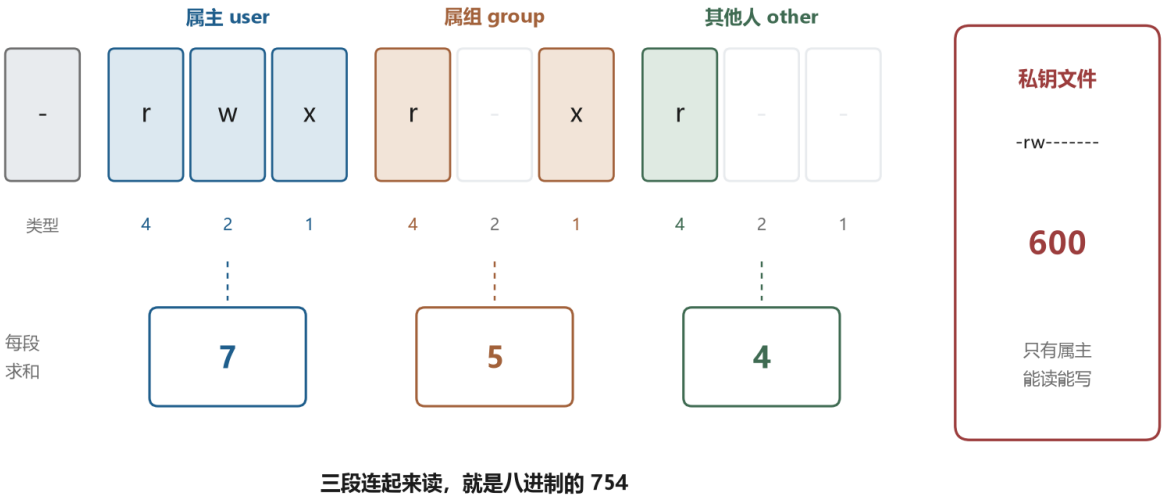


图 3-2 九个权限位与八进制的对应关系

这套权限体系和 SSH 有一个硬性交集，私钥文件的权限必须是 600。第六章会讲原因，这里先说结论，SSH 客户端在读取私钥之前会检查文件权限，如果发现同组或其他用户也能读，它会直接拒绝使用这把钥匙并给出警告。这条检查看起来吹毛求疵，实际很有必要，因为私钥一旦泄露，等于把身份送人。

```
chmod 600 ~/.ssh/id_ed25519
chmod 700 ~/.ssh
```

目录 `.ssh` 用 700，只有自己能进。公钥文件 `id_ed25519.pub` 可以 644，因为公钥本来就是给别人看的。`authorized_keys` 建议 600，虽然 sshd 对这个文件的要求因配置而异，但收紧一点没有坏处。

3.3 该记住的几个路径

服务器上的文件散落在各处，但和 SSH 相关的只有几个固定位置。表 3-1 是这本书里会反复用到的。

表 3-1 与 SSH 相关的常用路径

路径	所在机器	作用
`~/.ssh/config`	客户端	主机别名与连接参数
`~/.ssh/id_ed25519`	客户端	私钥，权限 600
`~/.ssh/id_ed25519.pub`	客户端	公钥，可以公开
`~/.ssh/known_hosts`	客户端	记下见过的服务器主机密钥
`~/.ssh/authorized_keys`	服务端	允许登录的公钥清单
`/etc/ssh/sshd_config`	服务端	sshd 的全局配置
`/etc/ssh/ssh_config`	客户端	全系统的客户端默认配置
`/var/log/auth.log`	服务端	Debian 系的认证日志
`/var/log/secure`	服务端	Red Hat 系的认证日志

注意 `ssh_config` 和 `sshd_config` 只差一个字母，含义完全不同。前者是客户端读的，后者是服务端读的。搜索配置项时一定要看清楚，把服务端的选项写进客户端的文件里，通常不会报错，只是静默失效，这种问题最难查。

3.4 伪终端是什么

早期的计算机，终端是一台真实的设备，通过串口线连到主机。操作系统里有一套终端驱动程序负责和它打交道。后来终端变成了软件，为了让那些只会和终端打交道的程序继续工作，系统提供了伪终端，用一对虚拟设备模拟串口的两端。一端叫 master，由 sshd 持有，另一端叫 slave，交给 shell。

这样做的结果是，shell 和它启动的程序都以为自己连着一台真正的终端。它们能收到窗口大小变化的信号，能使用行编辑、能显示颜色、能处理 Ctrl+C。如果 sshd 不分配伪终端，比如你执行 `ssh host "ls -l"` 这种一次性命令，那么 `ls` 会认为输出被重定向到了文件，于是不显示颜色，也不做列对齐。

伪终端的分配是一个可以控制的选项。客户端用 `-t` 强制分配，`-T` 禁止分配，配置文件里用 `RequestTTY`。什么时候需要关心它，通常是脚本里跑交互式程序的时候。比如用 `ssh` 执行 `sudo`，如果不分配伪终端，`sudo` 会因为拿不到终端而拒绝读取密码。

3.5 环境变量与 PATH

登录之后，shell 会读几个文件来设置环境变量。`/etc/profile` 对所有用户生效，`~/.bash_profile` 或 `~/.profile` 对当前用户生效。如果你用的是 zsh，对应的文件是 `~/.zshrc`。环境变量里最重要的一个是 `PATH`，它是一串用冒号分隔的目录，shell 找命令时就按这个顺序逐个目录搜索。

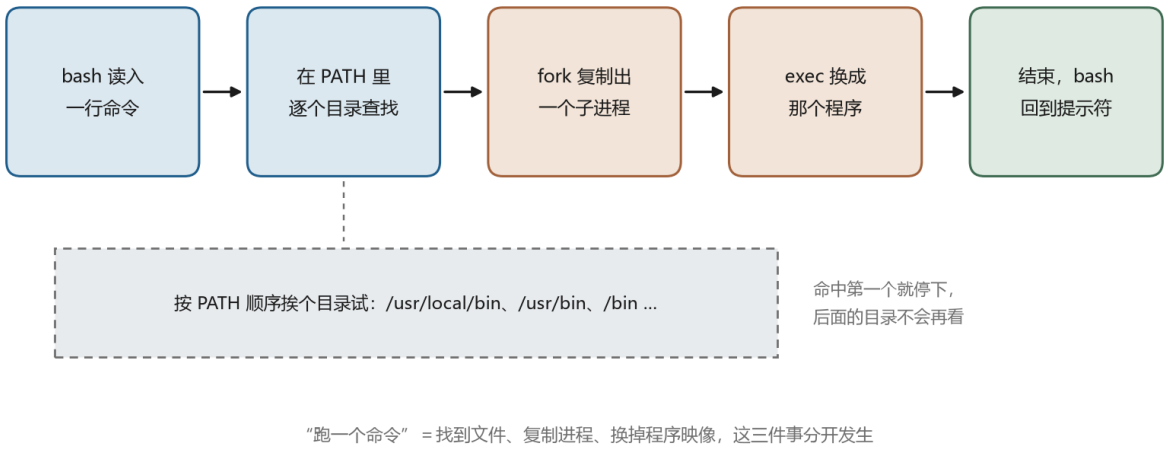


图 3-3 shell 查找命令、创建进程、回收子进程的过程

```
echo $PATH
which python3
export PATH="$HOME/.local/bin:$PATH"
```

这里有一个新手常踩的坑。你在 `~/.bashrc` 里加了 `export PATH=...`，用 `ssh host` 登录后发现没生效，原因是交互式登录 shell 读的是 `~/.bash_profile` 而不是 `~/.bashrc`，而 `~/.bash_profile` 里往往又只有一句 `source ~/.bashrc`，具体读哪个取决于发行版的默认配置。遇到“命令找不到”，先确认自己在哪种 shell 里，再确认那个 shell 读了哪些文件。

另一个坑和 sudo 有关。`sudo` 默认会重置环境变量，所以你在自己账户下能用的 `PATH`，切到 root 之后可能就变了。要用 `sudo -E` 保留环境，或者在 `sudoers` 里配置 `secure_path`。

3.6 常驻的服务

sshd 是一个常驻后台的进程，由 systemd 管理。开机时 systemd 读 `/etc/systemd/system/` 和 `/lib/systemd/system/` 下的单元文件，按依赖关系启动服务。sshd 的单元文件叫 `ssh.service` 或者 `sshd.service`，取决于发行版。

```
systemctl status ssh
systemctl restart ssh
journalctl -u ssh -n 50 --no-pager
```

这三条命令分别用于查看状态、重启服务和看最近的日志。改完 `sshd_config` 之后一定要重启或者 `systemctl reload ssh`，否则新配置不生效。重启前建议先保留一个已经登录的会话，因为配置写错可能导致再也连不上，那时候只能通过云厂商的救援控制台进去改。

systemd 还有一个能力值得知道，叫 socket 激活。有些发行版把 sshd 配置成由 systemd 监听 22 端口，收到连接才启动 sshd 进程。这种配置下，用 `ss -lntp` 看到的监听进程是 `systemd` 而不是 `sshd`，第一次见会以为装错了。判断方法很简单，看 `systemctl status ssh.socket` 是否存在。

3.7 日志里能看到什么

认证相关的日志包含的信息比多数人以为的多。一次成功的公钥登录会留下这样几行。

```
sshd[2417]: Accepted publickey for alice from 203.0.113.7 port 51544 ssh2: ED25519 SHA25\
    6:xxxx
sshd[2417]: pam_unix(sshd:session): session opened for user alice by (uid=0)
sshd[2417]: Received disconnect from 203.0.113.7 port 51544:11: disconnected by user
```

第一行告诉你用的哪种认证方式、从哪个地址来、用的哪把钥匙的指纹。第二行说明会话已经建立。第三行是断开。失败时格式类似，只是把 `Accepted` 换成 `Failed`，后面会跟上原因，比如 `invalid user`、`authentication failure`、`Connection closed by authenticating user`。

如果日志里出现大量 `Failed password for invalid user admin`，说明有人在扫描。这类噪音在公网上是常态，处理方法在后面的章节会讲。真正需要警惕的是针对你自己账户名的连续失败，那说明有人在有针对性地尝试。

到这里，服务器这一侧的基础就差不多了。你现在应该能回答几个问题。登录之后是谁在跟你说话，你的命令是怎么变成一个进程的，为什么私钥的权限必须是 600，日志里的那些字段各是什么意思。下一篇要进入这本书最硬的部分，也就是把这些连接加密起来的那套数学。

第 二 篇

信任是怎么建立的

连接建起来之后，问题才真正开始。

你连过去的那台机器，凭什么相信你是你。你凭什么相信它不是别人伪装的。中间经过的那些设备，凭什么看不到你在传什么。这三个问题不解决，远程登录只是把风险从线下搬到了线上。

这一篇讲的是答案。答案由数学给出，具体说是密码学。但这一篇不打算从公式开始，而是从需求开始。先说清楚要防什么，再说用什么工具防，最后看 SSH 怎么把这些工具组装成一套协议。中间会用到一些数学，都控制在理解原理所必需的范围，不需要你会推导。

读完之后，你应该能看懂 `ssh -Q kex` 列出的那些名字，知道每个名字代表什么取舍，也能判断一份配置里的算法选择是否合理。

第 4 章

把话藏起来

4.1 三种敌人

远程登录要防的不是抽象的“黑客”，而是三种具体的攻击。

第一种是窃听。数据包经过的每一台路由器、每一个交换机、每一个运营商的接入设备，理论上都能把经过的字节抄一份。早年间 telnet 和 FTP 的用户名密码就是这样泄露的，因为它们是明文。第二种是篡改。攻击者不一定需要读懂内容，只要能在传输过程中改动几个字节，就可能让远端执行另一条命令。第三种是冒充。你连过去的地址可能是攻击者伪造的，或者攻击者在中间同时骗过两端，这叫中间人攻击。

三种攻击对应三种技术。加密解决窃听，完整性校验解决篡改，身份认证解决冒充。SSH 把这三件事都做了，而且做得很彻底。理解它的设计，需要先理解它用的三样工具。

4.2 一把钥匙的对称加密

对称加密是最古老也最快的加密方式。发送方和接收方用同一把密钥，用同一个算法把明文变成密文，再用同一个算法反过来算回去。图 4-1 是这个过程的示意。

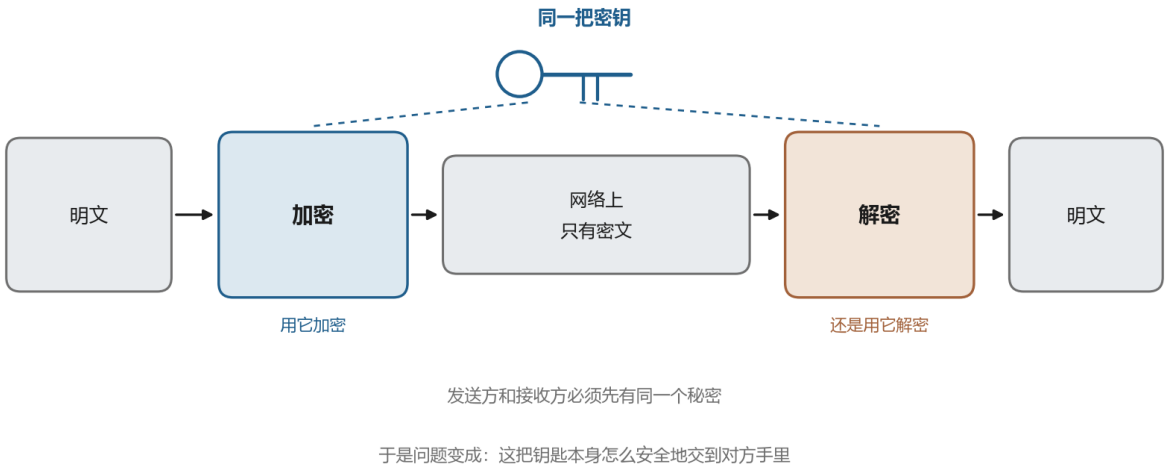


图 4-1 对称加密，加密与解密用同一把钥匙

今天的对称加密算法主要有两类。一类是分组密码，把数据切成固定大小的块逐块处理，AES 属于这一类，块长 128 位，密钥长度可以是 128、192 或 256 位。另一类是流密码，逐字节生成密钥流再和明文异或，ChaCha20 属于这一类。AES 在有大面积硬件加速的服务器上速度极快，ChaCha20 在没有 AES 硬件指令的手机和嵌入式设备上表现更好。SSH 两者都支持，连接时会协商使用哪个。

分组密码有个问题，如果对每个块都独立加密，相同的明文块会产生相同的密文块，攻击者

能从重复模式里推断内容。所以实际使用时要配合工作模式。CTR 模式把一个计数器加密后作为密钥流，等价于把分组密码变成流密码。GCM 模式在 CTR 的基础上加了认证，一次运算同时完成加密和完整性校验。

对称加密的优点是快，缺点是密钥分发。双方必须事先共享同一把钥匙，而在一个几万台机器的机房里，让每两台机器都共享一把不同的钥匙，管理成本是平方级的。这个问题由非对称加密解决。

4.3 哈希与消息认证码

哈希函数把任意长度的输入压成固定长度的输出，而且满足几个苛刻的性质。给定输入算输出很容易，给定输出反推输入几乎不可能。输入改一个比特，输出会面目全非，这叫雪崩效应。找到两个不同输入产生同一输出，在计算上也做不到。

SHA-256 是最常用的哈希函数，输出 256 位，写成 64 个十六进制字符。你在下载软件时看到的校验值，在证书里看到的指纹，都是这种形式。哈希不是加密，因为它不可逆，没有“解密”这一步。它的用途是比对，只要两段数据的哈希相同，就可以认为数据相同。

单独用哈希还不够。如果攻击者能同时改数据和哈希值，比对就失去意义。所以需要一把密钥参与，这就是消息认证码。HMAC 是最常见的构造方式，把密钥和数据一起送进哈希函数，只有知道密钥的人才能算出正确的值。SSH 用它来保证每个数据包没有被改动。

```
HMAC-SHA256(key, message) = SHA256((key ^ opad) || SHA256((key ^ ipad) || message))
```

这行公式不必记住，但要理解它的效果。没有密钥的人算不出正确的 HMAC，也就无法伪造一个能通过校验的包。SSH 的每个数据包后面都跟着一段 MAC，接收方先验 MAC 再解密，验不过就直接断开。这种“先认证再解密”的顺序可以避免把恶意构造的密文送进解密器。

4.4 非对称加密的两个用途

非对称加密用一对密钥，公钥和私钥。公钥可以公开，私钥必须自己保管。这一对密钥在数学上相关，但从公钥算不出私钥。它的用途有两个，方向刚好相反。

第一个用途是加密。用对方的公钥加密，只有对方的私钥能解开。这适合传递秘密，因为不需要事先共享密钥。第二个用途是签名。用自己的私钥对数据算一个签名，任何人用你的公钥都能验证这个签名确实出自你手，而且数据没被改过。这适合证明身份。

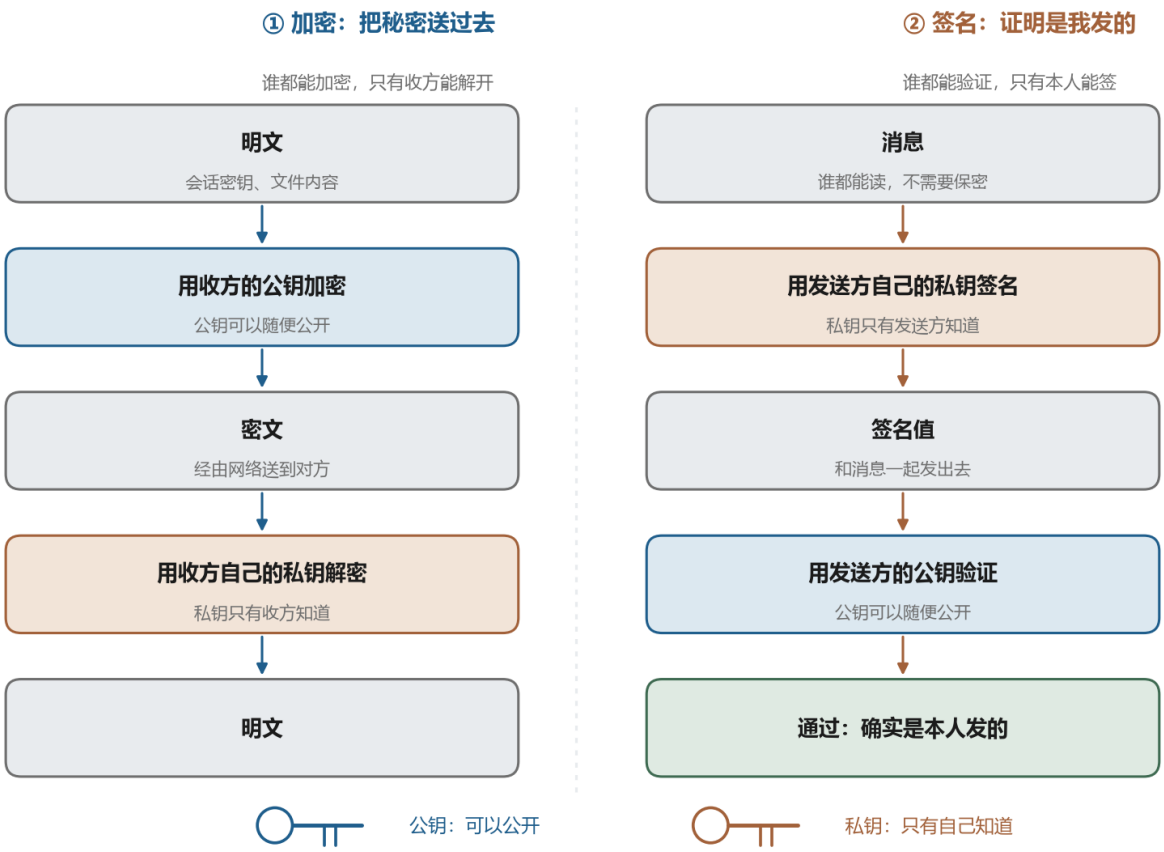


图 4-2 非对称密钥的两种用法，左边加密，右边签名

两种用途容易混淆，记住一句话就够了，加密是“给别人看”，签名是“证明是我”。SSH 的公钥认证用的是签名，不是加密。很多人以为登录时服务器用公钥加密一段数据发给客户端，客户端用私钥解开，这个理解是错的，SSH 的流程在第六章会详细展开。

常见算法有两族。RSA 依靠大整数分解的困难性，密钥长度通常 2048 位以上，签名和验证都比较慢。椭圆曲线算法 ECC 依靠离散对数的困难性，密钥短得多，256 位的 ECC 提供的安全强度大致相当于 3072 位的 RSA。Ed25519 是基于椭圆曲线的签名算法，密钥只有 32 字节，签名 64 字节，速度快，实现也简单，是今天 SSH 的推荐选择。

4.5 密钥交换

非对称加密虽然解决了密钥分发，但它的运算速度比对称加密慢几个数量级，用它加密大量数据不划算。所以实际做法是，用非对称的方法协商出一把临时的对称密钥，然后全部数据都用这把密钥加密。协商的过程叫密钥交换。

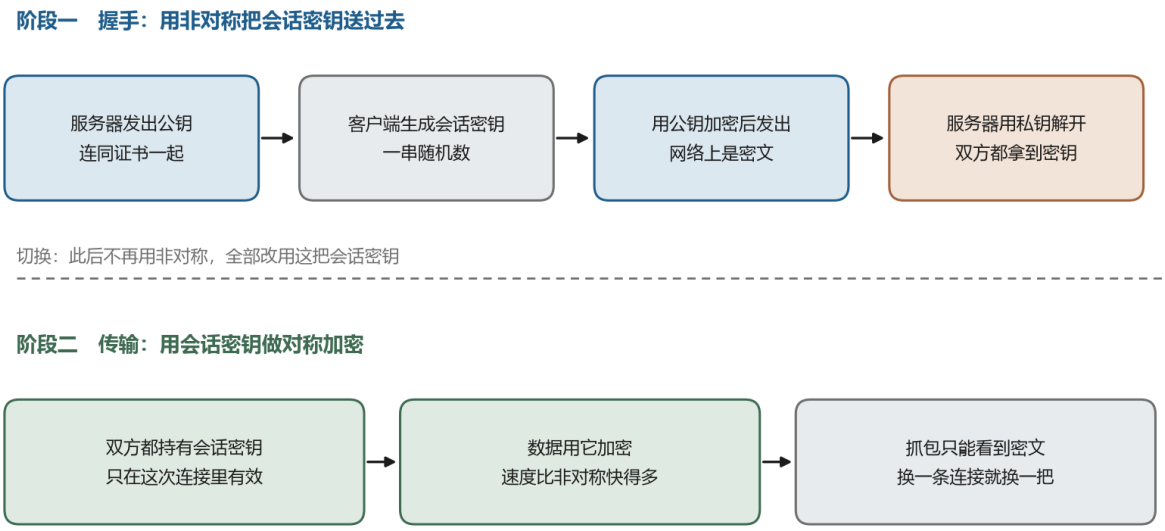


图 4-3 先协商会话密钥，再切换到对称加密

最经典的密钥交换协议是 Diffie 和 Hellman 在 1976 年提出的。双方各自选一个私密数字，用它算出对应的公开值，把公开值发给对方，再用对方的公开值和自己的私密数字算出同一个结果。图 4-4 是这个过程的示意。



图 4-4 双方各自算出相同的共享密钥，窃听者只能看到公开值

这个协议的巧妙之处在于，窃听者能看到两个公开值，但要从公开值反推出私密数字，需要解离散对数问题，在计算上不可行。于是双方得到了一个只有他们知道的共享秘密，而这个秘密从未在网络上传输过。

SSH 里最常用的密钥交换是 `curve25519-sha256`，用 Curve25519 这条椭圆曲线做 Diffie-Hellman，再用 SHA-256 做哈希。它的优点是速度快、密钥短、实现不容易出错。此外还有基于 NIST 曲线的 `ecdh-sha2-nistp256` 等，以及传统的 `diffie-hellman-group14-sha256`。

后者的参数是固定的，安全性依赖那个固定的素数，前向保密性略差一些。

4.6 为什么需要证书

非对称加密留下最后一个问题。你怎么知道手里这把公钥真的属于对方。如果攻击者在中间，把“服务器的公钥”换成自己的，你照样能建立加密通道，只是通道的另一端是攻击者。

HTTPS 用证书解决这个问题。服务器把公钥和身份信息交给一个受信任的第三方，也就是证书颁发机构，由它用自己的私钥签名，形成证书。浏览器内置了一批根证书，只要能沿着签名链一路验到某个根证书，就认为这把公钥是可信的。这套体系叫 PKI，公钥基础设施。

SSH 走了另一条路。它默认不使用证书，而是采用一种叫“首次使用即信任”的策略。第一次连接某台服务器时，客户端把服务器的主机公钥记到 `known_hosts` 里，以后每次连接都核对。如果哪天指纹变了，客户端会大声警告。这套办法的成本低，不需要第三方，缺点是第一次连接时你无法判断对面是不是真的。第六章会详细讨论这个取舍。

值得说明的是，SSH 也支持证书。企业里可以自建一个 CA，给几百台机器签发主机证书，客户端只需要信任这个 CA 的公钥，就能验证所有机器，省掉了每次手工确认指纹的麻烦。JumpServer 这类系统在托管资产时用的就是类似思路。

4.7 前向保密

假设某天你的服务器私钥泄露了。攻击者拿到私钥，能不能解开他过去录下来的加密流量。

如果密钥交换用的是 RSA 那种“用服务器公钥加密会话密钥”的方式，答案是能。因为过去每一段流量的会话密钥，都是用服务器公钥加密后传输的，攻击者只要保存了这些密文，拿到私钥就能一一解开。这种设计没有前向保密性。

如果密钥交换用的是 Diffie-Hellman 一族，答案是能也不能。攻击者拿到的是身份私钥，可以用来冒充服务器骗未来的连接，但过去的会话密钥是双方临时算出来的，从来没有用身份私钥加密过，也没有存储在磁盘上，所以解不开。这就是前向保密，也叫完美前向保密。

这也是为什么今天的 SSH 默认使用临时密钥交换算法，而早期的 `diffie-hellman-group1-sha1` 之类的算法被逐步淘汰。判断一个算法有没有前向保密，看它的名字里有没有 `dhe` 或者 `ecdh`。`ssh-rsa` 是签名算法，不涉及密钥交换，但用 RSA 做密钥交换的老方案确实没有前向保密。

4.8 这些工具在 SSH 里怎么分工

把上面几样工具放到一起，SSH 的一次连接是这样用它们的。

密钥交换阶段用 Diffie-Hellman 变体，双方算出共享秘密，由此派生出会话密钥。这一步保证了前向保密。服务器在密钥交换过程中用自己的主机私钥对交换内容签名，客户端用对应的公钥验证，这一步解决了冒充问题，前提是客户端知道正确的公钥。之后的所有数据用对称加密

算法处理，每个包附带 MAC，这一步解决了窃听和篡改。

三者缺一不可。没有签名，中间人可以替换公钥。没有对称加密，速度慢到不可用。没有 MAC，攻击者可以改包。理解了这套分工，第五章的协议细节就不难读了。

顺带说一句密钥长度的现实。表 4-1 是几种常见算法组合的密钥与签名长度，这些数字来自算法规范，不涉及任何性能测量。

表 4-1 常见签名算法的密钥与签名长度

算法	公钥长度	私钥长度	签名长度
Ed25519	32 字节	32 字节	64 字节
ECDSA P-256	64 字节（未压缩点）	32 字节	约 70 到 72 字节
RSA 2048	约 270 字节（编码后）	约 1190 字节（PEM）	256 字节
RSA 4096	约 530 字节（编码后）	约 3270 字节（PEM）	512 字节

这组数字解释了一个现象，用 RSA 密钥登录会比 Ed25519 慢一点，因为每次认证都要传更长的签名，做更慢的运算。差异在日常使用中不明显，但在自动化脚本里批量连接几千台机器时，能感觉出来。更重要的是，Ed25519 的密钥短，抄写、备份、贴进配置文件都不容易出错。

第 5 章

SSH 协议的三层结构

5.1 一行明文的版本号

连接建立之后，双方各自先发一行文本，形如 `SSH-2.0-OpenSSH_9.6`。这一行是明文，以回车换行结束，最长 255 字节。它包含协议版本、软件名和可选的注释。之所以允许明文，是因为此时还没有协商出任何密钥，双方需要先说清自己讲的是哪种语言。

服务器通常先发，客户端后发，但协议允许任意顺序，双方都要能容忍对方先开口。这一行的第二个字段是协议版本，`2.0` 表示 SSH-2。历史上存在过 SSH-1，它的设计有严重缺陷，早被废弃，OpenSSH 从 7.6 版起默认不再支持。

这里有个有意思的细节。有些网络设备会往连接里注入自己的横幅，干扰版本交换。OpenSSH 的做法是，在收到以 `SSH-` 开头的行之前，把收到的其他行都当作噪音丢掉，最多容忍 1024 行。如果你在调试输出里看到 `banner exchange` 之后跟着一堆奇怪的内容，多半是中间设备的锅。

5.2 三个子协议

SSH 协议被拆成三个子协议，一层叠一层，各自负责一件事。图 5-1 是这个结构。

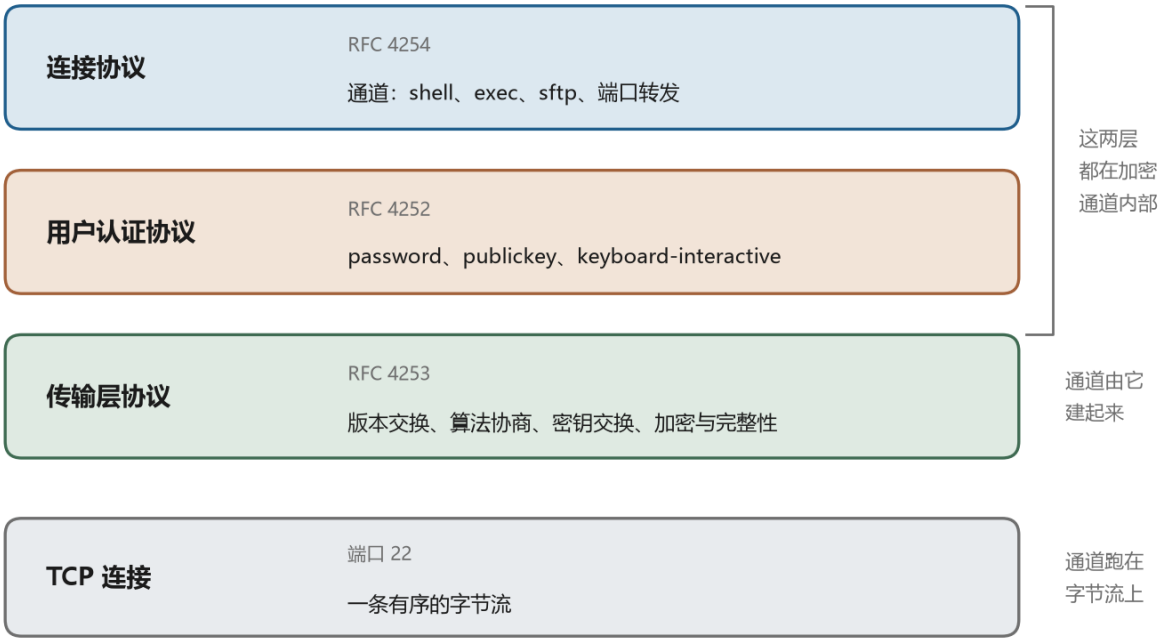


图 5-1 SSH 的三个子协议与它们承载的东西

最底下是传输层协议，也就是 RFC 4253 描述的部分。它负责在 TCP 之上建立一个加密的、

经过完整性保护的通道，完成版本交换、算法协商、密钥交换、密钥派生、重密钥和断开。它不关心你是谁，也不关心你要干什么。

中间是用户认证协议，RFC 4252。它跑在加密通道里面，负责让服务器确认“对面的这个人确实是某个账户的主人”。它支持多种认证方法，客户端可以按顺序尝试，服务器决定接受哪一种。认证成功之后，这个连接就绑定了某个用户身份。

最上面是连接协议，RFC 4254。它也跑在加密通道里，负责把一条连接切成多个逻辑通道，每个通道可以承载一种用途，交互式 shell、执行单条命令、文件传输、端口转发。一条 SSH 连接里可以同时开着很多个通道，它们互不干扰。

这个分层解释了为什么一条 SSH 连接能干那么多事。你开一个终端窗口跑命令，同时用 `scp` 传文件，又用 VS Code 在远端开发，如果都走同一条连接，只需要一次认证、一次密钥交换，后面全是通道复用。第七章讲端口转发时会用到这个性质。

5.3 一个数据包长什么样

加密通道建立之后，SSH 传输的是二进制包。每个包的格式固定，如图 5-2。



图 5-2 SSH 二进制包的字段结构

开头 4 个字节是包长度，表示后面还有多少字节。接下来 1 个字节是填充长度，然后是载荷，最后是随机填充。填充的作用是让包的长度是分组大小的整数倍，同时增加一点随机性。协议要求填充至少 4 字节，整个包（不含长度字段和 MAC）至少 16 字节。载荷最长 32768 字节，整包不超过 35000 字节，这是 RFC 4253 给出的下限要求，所有实现都必须能处理。

包尾是 MAC，长度由协商的算法决定，HMAC-SHA256 是 32 字节。接收方收到包之后，先算 MAC 比对，通过之后再解密载荷。这种顺序叫“先认证后解密”，可以防止攻击者把构造的密文喂给解密器做旁路分析。

后来又出现了一种改进叫 ETM，全称是 encrypt-then-MAC。传统的顺序是先加密整个包（含长度字段），再对密文算 MAC。ETM 的做法是不加密长度字段，但把长度字段也纳入 MAC 的计算范围。这样做的好处是接收方可以先看长度、先验 MAC，再决定要不要解密，既省了算力，

也避免了一些边界攻击。OpenSSH 的默认 MAC 算法都带 `-etm` 后缀。

载荷的第一个字节是消息类型编号，后面跟着这个消息自己的字段。表 5-1 是书里会用到的几个编号。

表 5-1 SSH 常用消息编号

编号	消息	出现在哪个阶段
1	SSH_MSG_DISCONNECT	任意时刻，表示断开及原因
20	SSH_MSG_KEXINIT	算法协商，双方各发一次
21	SSH_MSG_NEWKEYS	密钥交换结束，通知切换密钥
30	SSH_MSG_KEX_ECDH_INIT	客户端发送临时公钥
31	SSH_MSG_KEX_ECDH_REPLY	服务端回复临时公钥、主机公钥与签名
50	SSH_MSG_USERAUTH_REQUEST	认证请求
51	SSH_MSG_USERAUTH_FAILURE	认证失败，并列出可用的方法
52	SSH_MSG_USERAUTH_SUCCESS	认证成功
60	SSH_MSG_USERAUTH_PK_OK	公钥被接受，可以开始签名
80	SSH_MSG_GLOBAL_REQUEST	全局请求，比如请求转发端口
90	SSH_MSG_CHANNEL_OPEN	打开一个通道
98	SSH_MSG_CHANNEL_REQUEST	通道内的请求，比如分配伪终端

这些编号不需要背，但看到抓包或者日志里出现 `SSH_MSG_KEXINIT` 这样的字符串时，能立刻知道现在处于哪个阶段，排查会快很多。

5.4 算法协商

版本交换完成后，双方各发一个 `SSH_MSG_KEXINIT`。这个消息里有一个 16 字节的随机 cookie，然后是一串名字列表，分别列出自己支持的密钥交换算法、主机密钥算法、加密算法、MAC 算法、压缩算法、语言，每个方向各一份。

```
kex_algorithms          curve25519-sha256,ecdh-sha2-nistp256,diffie-hellman-group14-sha2\
    56
server_host_key_algs    ssh-ed25519,rsa-sha2-512,rsa-sha2-256
encryption_c2s          chacha20-poly1305@openssh.com,aes256-gcm@openssh.com,aes256-ctr
mac_c2s                 hmac-sha2-256-etm@openssh.com,hmac-sha2-512-etm@openssh.com
compression_c2s         none,zlib@openssh.com
```

协商规则很朴素，从客户端的列表里按顺序取，取第一个服务端也支持的。这个规则叫“客

户端优先”，意味着最终选择主要由客户端决定。图 5-3 是这个比较过程的示意。

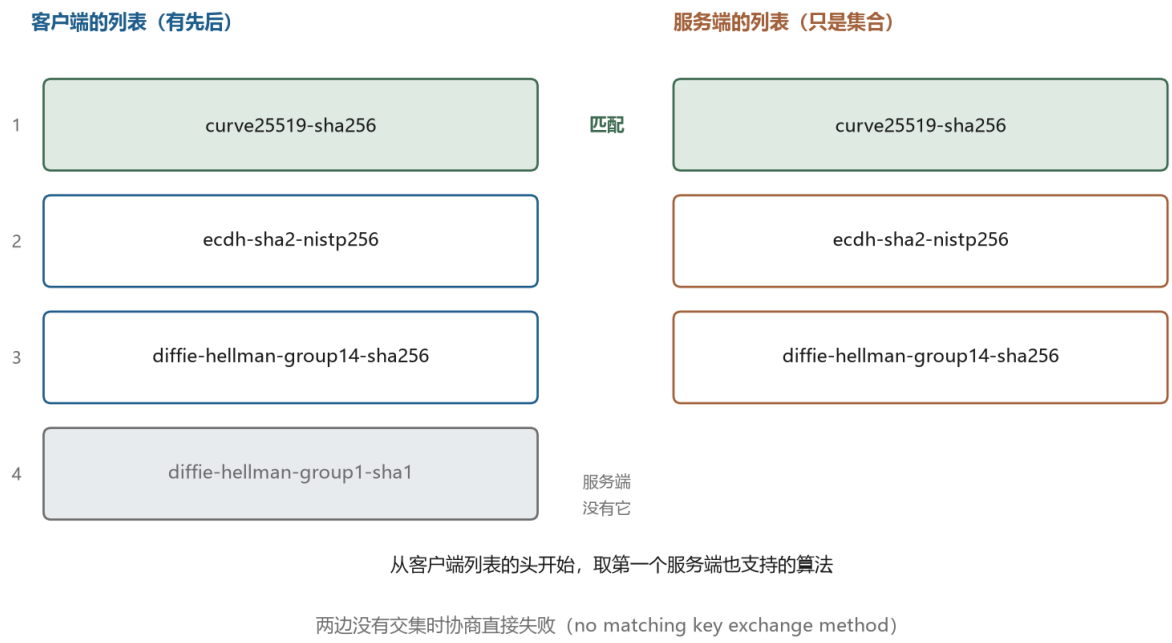


图 5-3 算法协商，取第一个双方都支持的项

客户端优先的设计有一个副作用。如果客户端被换成了老版本，只能提供弱算法，服务端又兼容它，那这条连接就会用弱算法。这让老客户端的兼容性变好了，代价是安全性由最弱的一方决定。企业里如果统一了客户端版本，可以在服务端配置里写死允许的算法，把弱的全部禁掉。

协商失败时会看到这样的报错。

```
Unable to negotiate with 10.0.0.5 port 22: no matching key exchange method found.
Their offer: diffie-hellman-group1-sha1
```

这类报错说明双方没有交集，处理方法有两个，要么升级一端，要么在客户端用 `-oKexAlgorithms=+diffie-hellman-group1-sha1` 显式允许那个老算法。后者是临时手段，不该长期使用。

5.5 密钥交换与密钥派生

协商完成后进入密钥交换。以 `curve25519-sha256` 为例，过程如图 5-4。



图 5-4 一次 ECDH 密钥交换的完整过程

客户端生成一对临时密钥，把临时公钥 `Q_C` 放进 `SSH_MSG_KEX_ECDH_INIT` 发出去。服务端也生成一对临时密钥，用自己的主机私钥对一段哈希值签名，然后把主机公钥、临时公钥 `Q_S` 和签名一起放进 `SSH_MSG_KEX_ECDH_REPLY` 发回。双方各自用自己的临时私钥和对方的临时公钥算出共享秘密 `K`。

那段被签名的哈希值叫交换哈希，记作 `H`。它的计算方式是把双方的版本字符串、双方的 KEXINIT 原文、主机公钥、双方的临时公钥、共享秘密按固定顺序拼接后做哈希。

```
H = SHA256(V_C || V_S || I_C || I_S || K_S || Q_C || Q_S || K)
```

这个 `H` 有两个作用。一是作为签名对象，让客户端确认服务端确实持有主机私钥。二是作为派生密钥的材料，`K` 只用来算 `H`，后面的会话密钥都从 `H` 派生。这样设计的好处是，即使 `K` 的某些比特有偏差，也不会直接影响密钥质量。

派生过程用同一个哈希函数反复计算，每次在末尾拼一个不同的字母。

```
IV_c2s   = HASH(K || H || "A" || session_id)
IV_s2c   = HASH(K || H || "B" || session_id)
Key_c2s  = HASH(K || H || "C" || session_id)
Key_s2c  = HASH(K || H || "D" || session_id)
MAC_c2s  = HASH(K || H || "E" || session_id)
MAC_s2c  = HASH(K || H || "F" || session_id)
```

六个字母对应六个方向的密钥材料，客户端到服务端和服务端到客户端各有一套，互不共用。

这叫双向独立密钥，好处是即使某一方向的密钥被攻破，另一方向仍然安全。`session_id` 是第一次密钥交换产生的 `H`，它在整条连接的生命周期里不变，用于把后续的重密钥绑定到最初的这次 交换上。

密钥长度不够时，把前面的结果再哈希一次并拼接，直到够长。这套“扩展”规则在 RFC 4253 的第 7.2 节，实现起来只有几行代码。

5.6 重密钥

会话密钥不能一直用。用得越久，泄露的风险越大，被分析的材料也越多。协议建议在传输了 1GB 数据或者用满 1 小时之后重新做一次密钥交换，这两个阈值在 OpenSSH 里可以用 `RekeyLimit` 调整。

```
ssh -oRekeyLimit=500M l40-server
```

重密钥的过程和第一次密钥交换一样，只是不再重新协商算法，直接复用之前的结果。新算出的密钥替换旧的，`session_id` 保持不变。这个过程对上层完全透明，你的 shell 不会有任何感觉。

顺带解释一个现象。有时候 `ssh -vv` 的输出里会突然出现一段新的 `SSH2_MSG_KEXINIT`，看起来像是重新连接了，其实只是触发了重密钥。如果连接质量不好、丢包严重，重密钥可能失败并导致断开，这时把 `RekeyLimit` 调大一点能缓解。

5.7 认证层

加密通道建好之后，客户端发送 `SSH_MSG_USERAUTH_REQUEST`，里面包含用户名、要使用的服务名（固定是 `ssh-connection`）和认证方法。服务器可以接受，可以拒绝并列出它还支持哪些方法，也可以要求继续。

SSH 定义了三种基本方法。`password` 最简单，明文密码放在加密通道里传过去。`publickey` 用签名证明持有私钥，是这本书的重点，细节放在第六章。`keyboard-interactive` 是一问一答的形式，服务器可以问任意多个问题，适合一次性口令、短信验证码、多因素认证。此外还有 `hostbased` 和 `gssapi-with-mic`，前者用主机密钥认证，后者走 Kerberos，日常较少见。

认证可以多轮。服务器返回失败时会给一个列表，客户端按自己的策略依次尝试。OpenSSH 客户端默认最多尝试 6 次，可以用 `NumberOfPasswordPrompts` 限制。堡垒机场景下经常配置成只允许 `publickey`，这时密码登录会被直接拒绝，报错里会写 `Permission denied (publickey)`。

有一个细节容易让人困惑。公钥认证的第一步，客户端会先发一个“试探”请求，只带公钥不带签名，问服务器“这把钥匙你能接受吗”。服务器如果接受，回一个 `SSH_MSG_USERAUTH_PK_OK`，客户端才开始签名。这个设计的目的是避免在服务器不认可这把钥匙时白做一次签名运算，同时也让客户端知道该用哪把钥匙。有些服务器为了不泄露“某个用户名是否存在”，会跳过这一步直接要求签名，两种行为都合法。

5.8 连接层

认证成功之后，客户端可以打开通道。每个通道有本地编号和远端编号，双方各自维护一套，互不干扰。通道类型有几种，`session` 用于 shell 和命令执行，`direct-tcpip` 用于本地转发，`forwarded-tcpip` 用于远程转发。

打开通道之后，客户端在通道里发请求。最常见的几个是 `pty-req` 请求分配伪终端并告知窗口大小，`shell` 请求启动登录 shell，`exec` 请求执行一条命令，`subsystem` 请求启动一个子系统（`sftp` 就是通过它启动的），`window-change` 在窗口大小变化时通知对方。

连接层还有流量控制。每个通道有一对窗口大小，表示对方还能接收多少字节的数据。发送方发完数据要扣减窗口，接收方处理完数据后回一个 `CHANNEL_WINDOW_ADJUST` 把窗口补回去。如果窗口耗尽，发送方必须停下来等。这套机制防止了快速的生产者把慢速的消费者撑爆。传大文件时如果速度上不去，有时候和窗口大小有关。

全局请求独立于通道，比如 `tcpip-forward` 用于请求服务端监听某个端口，这是第七章远程转发的基础。还有一类请求叫 `keepalive@openssh.com`，客户端定期发一个，相当于心跳，防止中间的 NAT 设备因为长时间没有流量而把连接表项清掉。这就是 `ServerAliveInterval` 选项背后做的事。

5.9 看看自己的机器支持什么

OpenSSH 提供了查询算法清单的选项，比翻文档快。

```
ssh -Q kex          # 支持的密钥交换算法
ssh -Q key          # 支持的主机密钥与用户密钥类型
ssh -Q cipher       # 支持的对称加密算法
ssh -Q mac          # 支持的 MAC 算法
ssh -Q key-sig      # 支持的签名算法
```

把这些输出和 `sshd -T` 的结果对照，就能知道协商会落在哪个算法上。

```
sudo sshd -T | grep -E 'kexalgorithms|ciphers|macs|hostkeyalgorithms'
```

`sshd -T` 会打印服务端生效的完整配置，包括默认值和配置文件里的覆盖值。这是排查“为什么用了这个算法”的最快方式，比读配置文件可靠，因为配置文件可能有 `Include`、有 `Match` 块，肉眼追不全。

5.10 抓包能看到什么

最后说一个常见疑问。既然 SSH 加密了，抓包还有用吗。

有用，但只能看到结构。版本交换那一行是明文，能直接读出来。之后的包，载荷是不可读的随机字节；长度字段是否可见取决于双方是否启用 ETM，启用了就能看到明文长度，没启用则

连长度也一起加密。无论哪种情况，包的大小和出现的时间都瞒不住。比如你敲一个字符，客户端可能立刻发一个很小的包，交互式命令的输出会产生一串中等大小的包，传文件时包长接近上限。通过分析包长和时序，可以推测出你正在做什么类型的事，这类研究叫流量分析。

防御流量分析需要固定长度的填充，代价是带宽浪费，SSH 没有这么做。企业里如果确实担心这个，通常是在外层再套一层隧道，或者在网络层面做流量整形。对绝大多数场景，这个风险可以忽略。

抓包在排错时仍然很有价值。`tcpdump -n -i any port 22` 能看到连接是否建立、有没有重传、RST 是谁发的。这些信息足以区分“网络不通”和“服务拒绝”。真要看协议内容，用 `ssh -vv` 比抓包方便得多，因为客户端本来就会把每个阶段的解析结果打印出来。

第 6 章

密钥、指纹与信任

6.1 生成一对钥匙

登录一台服务器，最稳妥的方式是公钥认证。它需要在客户端准备一对密钥，把公钥交到服务器上，私钥留在自己手里。生成密钥的命令是 `ssh-keygen`。

```
ssh-keygen -t ed25519 -C "alice@laptop" -f ~/.ssh/id_ed25519_lab
```

`-t` 指定算法类型，`-C` 是注释，会写进公钥末尾，方便你在服务器上认出这是哪台机器的钥匙。`-f` 指定文件名，不指定的话默认是 `~/.ssh/id_ed25519`。命令执行过程中会问你要不要设置口令，这个口令用来加密私钥文件本身，不是服务器的密码。

设不设口令是一个真实的取舍。设了口令，私钥文件被偷走也打不开，但每次用钥匙都要输一遍，除非配合 ssh-agent。不设口令，用起来方便，但私钥文件一旦泄露就等于身份泄露。自动化脚本里的密钥通常不设口令，靠文件权限和磁盘加密来保护。人用的密钥建议设口令。

生成之后会得到两个文件。`id_ed25519_lab` 是私钥，权限自动设为 600。`id_ed25519_lab.pub` 是公钥，内容是一行文本。

```
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIExampleKeyMaterialHere alice@laptop
```

把这行文本追加到目标机上的 `~/.ssh/authorized_keys` 里，就能用对应的私钥登录了。这件事手工做很容易出错，`ssh-copy-id` 可以代劳。

```
ssh-copy-id -i ~/.ssh/id_ed25519_lab.pub alice@10.0.0.5
```

它会连过去，把你的公钥追加进去，还会顺手修好目录和文件的权限。Windows 上没有这个命令，可以手动把公钥内容粘贴进去。

6.2 私钥文件里装的是什么

打开一个 OpenSSH 格式的私钥文件，看到的是这样的文本。

```
-----BEGIN OPENSSH PRIVATE KEY-----
b3BlbnNzaC1rZXktdjEAAAAABG5vbmUAAAAEbm9uZQAAAAAAAAABAAAAMwAAAAtzc2gtZW
QyNTUxOQAAACDxampleBase64MaterialGoesHereAndContinuesForSeveral
LinesUntilTheKeyBlobIsCompleteAAAAJgExampleCommentFieldHere
-----END OPENSSH PRIVATE KEY-----
```

开头和结尾的两行是标记，中间是一段 base64 编码的二进制数据。解码之后是一个按固定

格式排列的结构，包含算法名、公钥、私钥、注释，可能还有用于校验的字段。如果私钥设了口令，私钥那部分会用口令派生的密钥加密，派生函数是 bcrypt，迭代次数很高，暴力破解的代价很大。

这里要澄清一个常见的混乱。`.pem`、`id_rsa`、`id_ed25519` 这些名字看起来像是三种不同的东西，其实只是两种格式和两种算法的组合。`id_rsa` 里的 `rsa` 指算法，文件格式可能是老的 PEM 也可能是新的 OpenSSH 格式。`.pem` 是文件扩展名，指 PEM 编码，它本身不说明里面装的是 RSA 还是 ECC。从 JumpServer 下载的那种 `ssh.jumpserver.pem`，打开看内容，多数是标准的 OpenSSH 或 PEM 私钥，只是名字沿用了云厂商的习惯。

格式可以用 `ssh-keygen` 互相转换。老的 PEM 格式兼容性更好，某些第三方工具只认它。

```
ssh-keygen -p -m PEM -f ./ssh.jumpserver.pem     # 转成 PEM 格式
ssh-keygen -p -m RFC4716 -f ./key                # 转成 RFC 4716 格式
ssh-keygen -y -f ./ssh.jumpserver.pem            # 从私钥导出公钥
```

最后那条命令很实用。当你只有私钥，想知道对应的公钥是什么样，用它就能打印出来，不用去翻服务器上的 `authorized_keys`。

6.3 公钥认证到底怎么验证

很多人以为公钥登录的过程是“服务器用公钥加密一段数据，客户端用私钥解开”，这个理解是错的。真正的过程是签名，如图 6-1。



私钥始终留在客户端：发出的只有签名，私钥本身从不传输

图 6-1 公钥认证的四步，私钥始终不出本机

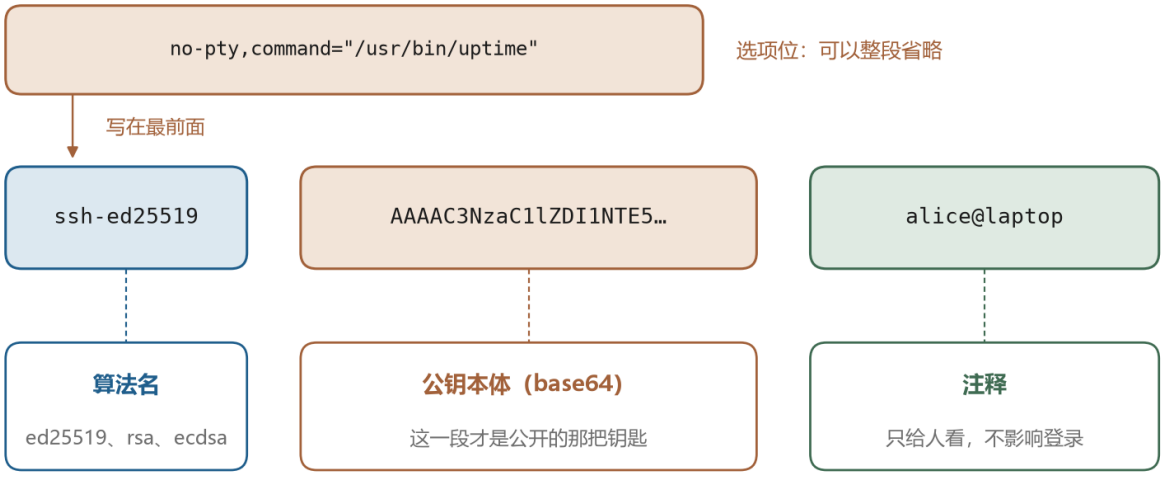
客户端先发一个试探请求，告诉服务器“我想用这把公钥登录”，里面只有公钥本身。服务器去 `authorized_keys` 里查，找到了就回一个确认。然后客户端构造一段数据，包含会话标识和这次的认证请求原文，用自己的私钥对它签名，把签名发过去。服务器用客户端提供的公钥验证签名，验过了就认为客户端确实持有对应的私钥。

签名的对象里有会话标识，这一点很关键。会话标识是密钥交换时算出的那个哈希 `H`，每条连接都不一样。把会话标识放进签名，意味着这个签名只对当前这条连接有效，攻击者录下来也没法重放到另一条连接上。这是第五章提到的交换哈希的又一个用途。

整个过程中私钥从未离开客户端，连加密后的形式都没有传输过。这比“私钥加密、公钥解密”的方案安全得多，因为后者的私钥要参与运算并可能暴露时序信息。也正因为如此，服务器上只需要存公钥，即使服务器被攻破，攻击者拿到的也只是公钥，没法冒充你登录别的机器。

6.4 authorized_keys 的写法

这个文件的每一行是一把被授权的公钥，格式如图 6-2 所示，三个字段用空格分开。



三段只用空格隔开，一行就是一条公钥；匹配到哪一行，那一行的选项跟着生效。

图 6-2 authorized_keys 一行内容的字段构成

第一个字段是算法名，第二个是公钥本体的 base64，第三个是注释。注释不参与验证，可以随便写，很多人用它记录这把钥匙属于谁、在哪台机器上。用 `ssh-keygen -l -f` 可以查看文件里每把钥匙的指纹和注释。

```
ssh-keygen -l -f ~/.ssh/authorized_keys
```

这一行前面还可以加选项，用来限制这把钥匙能做什么。这是堡垒机和大规模部署里非常有用的功能。

```
from="203.0.113.0/24",no-pty,no-port-forwarding ssh-ed25519 AAAAC3... alice@laptop
command="/usr/local/bin/backup.sh",restrict ssh-ed25519 AAAAC3... backup@nas
```

第一行限制来源网段，禁止分配伪终端，禁止端口转发。第二行强制这条钥匙只能执行指定命令，其他一切都禁用。`restrict` 是一个总开关，等价于关掉所有可选功能，再按需打开。这类选项在需要给第三方临时权限时特别有用，你给的是一把只能做一件事的钥匙，而不是一个完整的 shell。

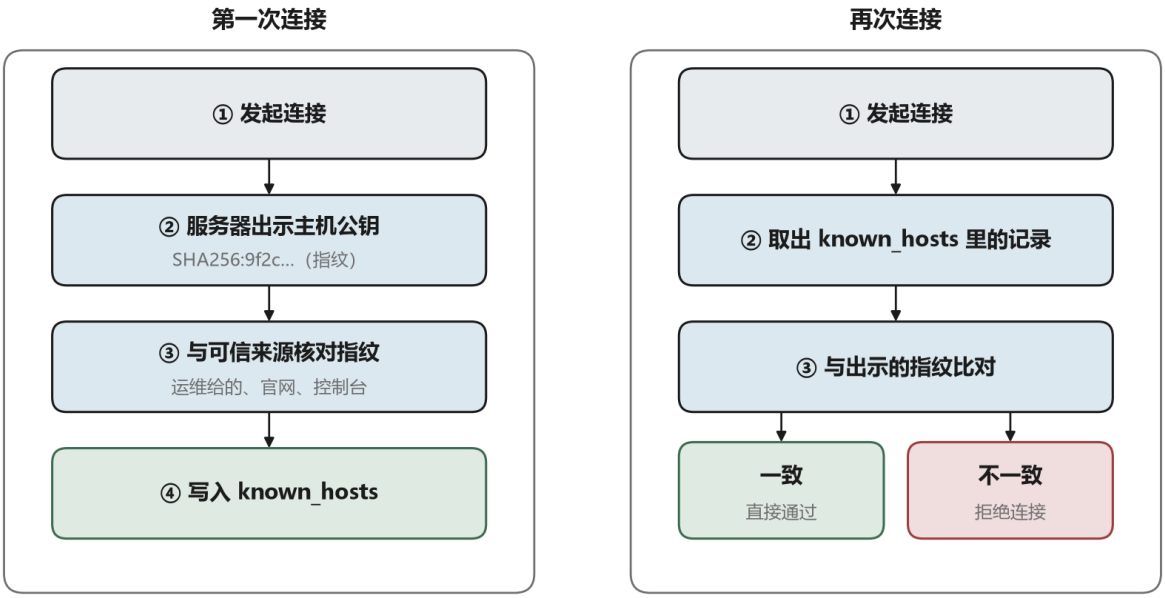
6.5 主机密钥与首次信任

前面讲的是服务器验证你，反过来你也需要验证服务器。如果攻击者在中间，他可以假装成服务器，骗你输入密码或者骗取你的公钥签名。SSH 用主机密钥解决这个问题。

每台运行 sshd 的机器在安装时会生成一对主机密钥，公钥放在 `/etc/ssh/ssh_host_ed25519_key.pub` 之类的文件里。客户端第一次连接时，服务器把主机公钥发过来，客户端算一个指纹显示给你。

```
The authenticity of host '10.0.0.5 (10.0.0.5)' can't be established.
ED25519 key fingerprint is SHA256:ExampleFingerprintStringHere.
Are you sure you want to continue connecting (yes/no/[fingerprint])?
```

你回答 yes，这把公钥就被写进 `~/.ssh/known_hosts`。以后每次连接，客户端都拿服务器发来的公钥和记录比对，一致就通过，不一致就报警并拒绝连接。这套策略叫首次使用即信任，简称 TOFU，如图 6-3。



指纹对不上：可能是中间人，也可能是服务器重装了系统。先核实，再决定是否删掉旧记录。

图 6-3 首次连接记录指纹，之后每次比对

指纹变了意味着什么。有两种可能，一是服务器重装了系统或者换了主机密钥，二是真的有人在中间。客户端无法区分这两种情况，所以它只能停下来问你。

```
WARNING: REMOTE HOST IDENTIFICATION HAS CHANGED!
IT IS POSSIBLE THAT SOMEONE IS DOING SOMETHING NASTY!
```

看到这段警告不要条件反射地删掉 `known_hosts` 里那一行。先确认服务器是不是最近重装过，或者找管理员核对指纹。确认没问题之后，用下面的命令删掉旧记录。

```
ssh-keygen -F 10.0.0.5              # 查这条记录在文件里的位置
ssh-keygen -R 10.0.0.5              # 删除这条记录
ssh-keyscan -t ed25519 10.0.0.5     # 重新抓取主机公钥
```

顺带说一句 `known_hosts` 的一个坑。如果你用 IP 连过一次，又用域名连过一次，文件里会有两条记录，因为它们被当作两台不同的主机。云端机器换 IP 之后重新分配，也容易出现指纹不匹配的告警。企业里机器多的时候，这套机制会变得难以维护，所以有了下面的证书方案。

6.6 ssh-agent 与密钥转发

设了口令的私钥每次用都要输一遍，很烦。`ssh-agent` 是一个后台进程，你把私钥解密后交给它保管，之后所有 SSH 连接都向它请求签名，不用再输口令。

```
eval "$(ssh-agent -s)"       # start the agent
ssh-add ~/.ssh/id_ed25519    # add the key, passphrase asked once
ssh-add -l                   # list loaded keys
```

Windows 上 OpenSSH 客户端自带的 agent 是一个系统服务，在“服务”里叫 `OpenSSH Authentication Agent`，启动之后用 `ssh-add` 添加即可，需要管理员权限才能设为自动启动。

密钥转发是另一个功能，叫 Agent Forwarding。开启之后，你在跳板机上执行 `ssh` 连第三台机器时，跳板机会把认证请求转发回你的本机，由本机的 agent 完成签名。这样私钥始终留在自己电脑上，跳板机上不需要放任何密钥。

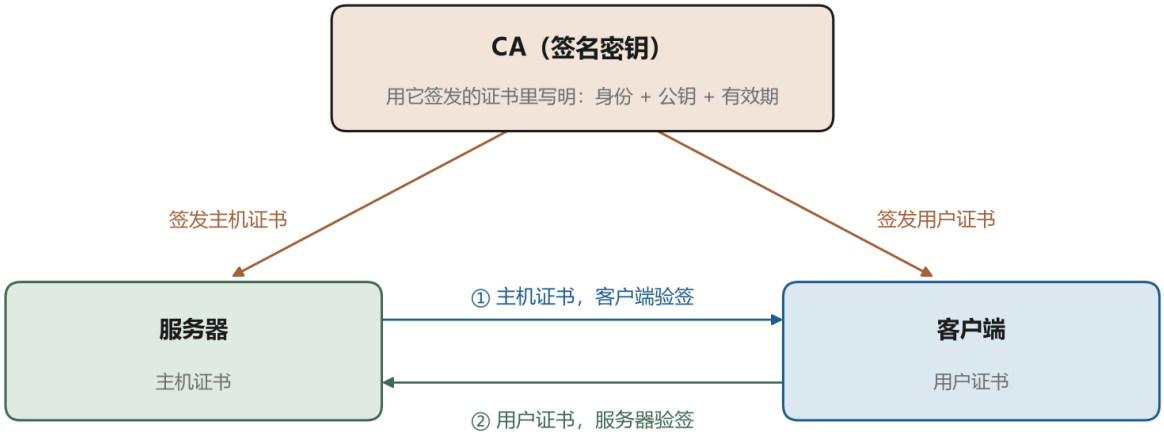
```
ssh -A jump-host
```

这个功能很方便，风险也不小。跳板机上的 root 用户可以通过 agent 的 socket 让你的 agent 签名任意数据，相当于在你不注意的时候用你的身份登录其他机器。Agent Forwarding 的原则是只在你信任的跳板机上开启，用完立刻退出。更好的做法是用第七章的 `ProxyJump`，它不需要转发 agent，认证在本机完成。

6.7 证书

机器一多，`known_hosts` 和 `authorized_keys` 就变成噩梦。每加一台机器都要分发公钥，每换一次密钥都要通知所有人。SSH 证书是为了解决这个问题而设计的。

思路和 HTTPS 一样，建立一个 CA，用它的私钥签发证书。主机证书里写明“这台机器的主机公钥属于这个主机名”，用户证书里写明“这把公钥属于这个用户，有效期到某天”。客户端只需要信任 CA 的公钥，就能验证所有由它签发的证书，如图 6-4。



双方只在本地预置同一个 CA 公钥：不再逐个核对主机指纹，也不再逐台分发 authorized_keys。

证书里还写着有效期与用途，过期即失效，可以按时换发。

图 6-4 由 CA 签发主机证书与用户证书

```
ssh-keygen -t ed25519 -f ca_key -C "ssh-ca"          # CA key pair
ssh-keygen -s ca_key -I alice -n alice -V +52w alice.pub   # user cert
ssh-keygen -s ca_key -h -I web01 -n web01.example.com -V +52w host.pub  # host cert
```

`-I` 是证书标识，`-n` 是允许的主体名，`-V` 是有效期，`-h` 表示这是主机证书。签发出来的文件以 `-cert.pub` 结尾，客户端拿着它就能登录，服务器端只要在 `sshd_config` 里配置 `TrustedUserCAKeys` 指向 CA 公钥即可。

JumpServer 这类系统在托管资产时，本质上做的也是这件事，只不过它把签发和管理都封装进了界面。第九章会讲它的具体做法。

6.8 一份能用的客户端配置

把这一章的内容落到配置上，`~/.ssh/config` 大致长这样。

```
Host *
    ServerAliveInterval 30
    ServerAliveCountMax 3
    AddKeysToAgent yes

Host lab
    HostName 10.0.0.5
    User alice
    Port 22
    IdentityFile ~/.ssh/id_ed25519_lab
    IdentitiesOnly yes
```

`ServerAliveInterval` 每 30 秒发一个心跳，防止 NAT 设备把闲置连接清掉。`AddKeysToAgent` 让第一次输入的口令被记住。`IdentitiesOnly yes` 只使用配置里指定的私钥，避免客户端把 agent 里的每把钥匙都试一遍，服务器限制尝试次数时会因此被拒绝。

这个文件从上往下读，第一个匹配的 `Host` 块生效，后面的同名选项覆盖前面的。`Host *` 放在最前面作为默认值，后面的具体主机块覆盖它，这是社区里通行的写法。第十章会详细讲匹配规则，包括通配符和 `Include`。

第 7 章

端口转发，把远方的服务搬到本地

7.1 只开一个端口的世界

很多内网只允许外部访问一个端口，通常就是 SSH 的 22。数据库、内部网页、监控面板全在里面，从外面碰不到。这种设计出于安全考虑，但也带来不便。你需要临时看一眼内网的某个网页，或者用本地的图形化工具连一下内网的数据库，怎么办。

办法是把那些服务“搬”到你本地来。SSH 的连接层本来就支持多路复用，一条加密连接里可以开很多个通道，每个通道承载一路数据。端口转发就是利用这个能力，把某个 TCP 端口上的流量塞进 SSH 通道，从另一端送出去。对使用者来说，本地凭空多了一个端口，连它就等于连到了远端的某个服务。

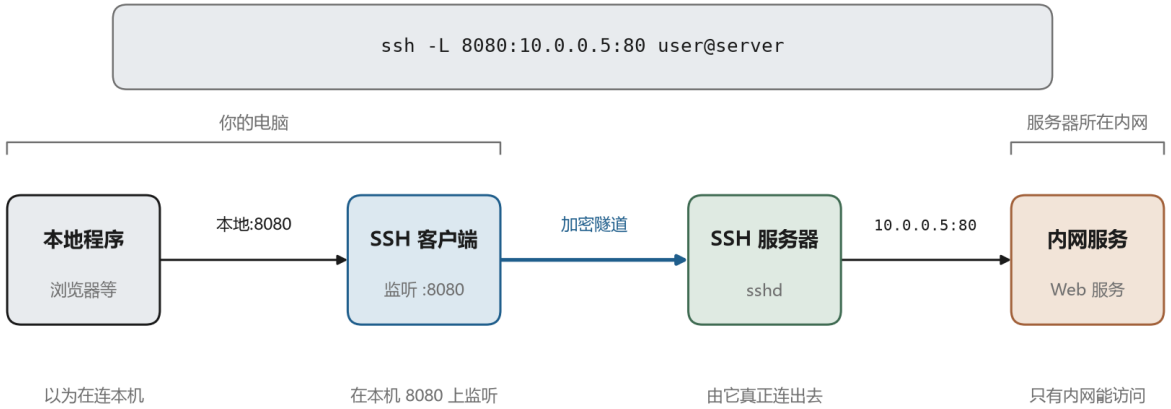
SSH 定义了三种转发方式，本地转发、远程转发、动态转发。它们的区别只在于谁监听、谁发起连接。

7.2 本地转发

本地转发的语法是 `-L`，格式为 `本地地址:本地端口:目标地址:目标端口`。

```
ssh -L 127.0.0.1:8080:10.0.0.9:80 alice@jump.example.com
```

这条命令的意思是，在本机监听 8080 端口，任何连到这个端口的流量，都通过 SSH 连接送到 `jump.example.com`，再由那台机器向 `10.0.0.9` 的 80 端口发起连接。图 7-1 是这个结构。



监听的那一端在本地，真正去连服务的那一端在远端。

图 7-1 本地转发，监听在本地，连接由远端发起

这里有一个特别容易搞错的地方。目标地址 是由 SSH 服务器去解析和连接的，不是由你的本机。所以 `10.0.0.9` 这个地址只需要在服务器那一侧可达，你的本机根本不需要认识它。很多人以为要先把 `10.0.0.9` 写进本机的 hosts，其实不必。

另一个容易搞错的地方是监听地址。写 `127.0.0.1:8080` 表示只监听本机回环，其他机器连不进来。写 `0.0.0.0:8080` 表示监听所有网卡，同一个局域网里的人也能用你的机器做跳板。默认建议用前者，除非你明确知道自己在做什么。

```
ssh -L 8080:10.0.0.9:80 alice@jump.example.com    # bind 127.0.0.1 by default
ssh -L 0.0.0.0:8080:10.0.0.9:80 alice@jump.example.com
```

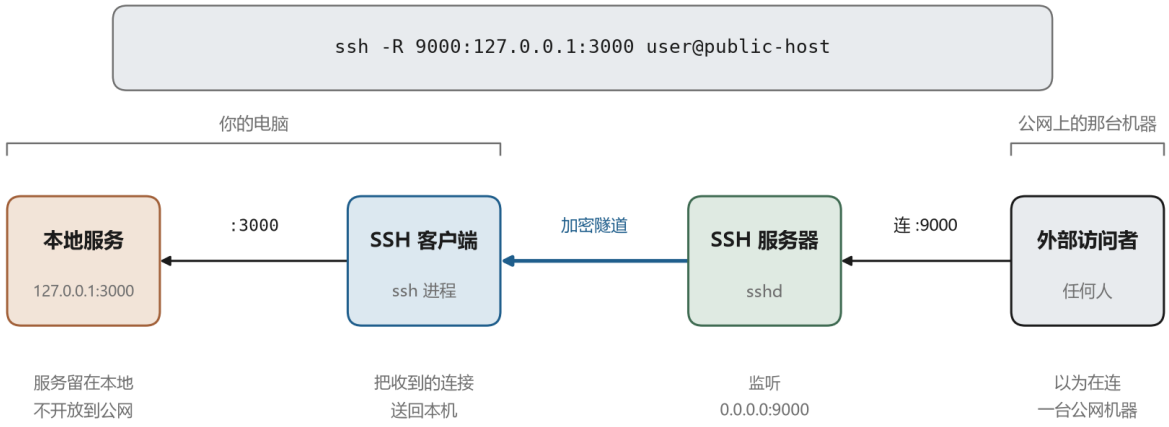
本地转发最典型的用法有三个。看内网的网页面板，把远端的 80 或 443 映射到本地。连内网的数据库，把 3306 映射出来，用本地的客户端连 `127.0.0.1:3306`。访问只允许从特定机器访问的服务，比如某个只对跳板机开放的管理接口。

7.3 远程转发

远程转发把方向倒过来，语法是 `-R`，格式为 远程地址:远程端口:目标地址:目标端口。

```
ssh -R 0.0.0.0:9000:127.0.0.1:3000 alice@public.example.com
```

这条命令的意思是，在 `public.example.com` 上监听 9000 端口，任何连到它的流量，通过 SSH 连接送回你的本机，再由你的本机连到 `127.0.0.1:3000`。图 7-2 是这个结构。



同样的隧道，反了个方向：监听放在远端，服务留在本地。

图 7-2 远程转发，监听在远端，连接回到本地

这个方向适合的场景是，你的开发机在内网，没有公网地址，但你想让外面的人访问你本机跑的服务。或者反过来，你在家里，想把公司内网某台机器的服务暴露到一台有公网地址的跳板机上，临时给同事看。

远程转发有一个常见的坑，就是监听地址。默认情况下，即使你写了 `0.0.0.0:9000`，服务端

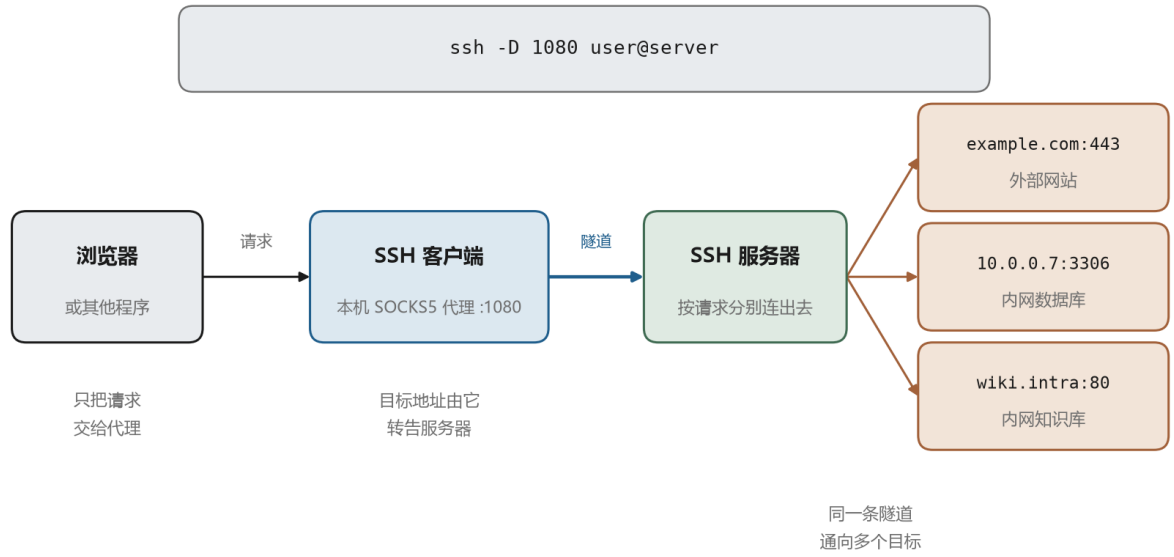
也可能只在回环地址上监听，外面仍然连不进来。原因是 sshd 的配置项 `GatewayPorts` 默认是 `no`，它禁止把转发的端口绑定到非回环地址。要让外部能连，服务端必须设置 `GatewayPorts yes` 或者 `GatewayPorts clientspecified`，然后重启 sshd。这件事你自己改不了，需要服务器管理员配合。

7.4 动态转发

前面两种转发都要事先指定目标地址和端口。如果你想让浏览器把所有网站都走 SSH 出去，总不能每个网站配一条 `-L`。这种需求用动态转发，语法是 `-D`。

```
ssh -D 127.0.0.1:1080 alice@jump.example.com
```

这条命令在本机 1080 端口起了一个 SOCKS5 代理。浏览器配置成使用这个代理之后，每次访问网站，浏览器先把“我要访问 example.com 的 443 端口”告诉代理，代理通过 SSH 通道让服务器去连，再把数据转发回来。目标地址是运行时由客户端指定的，所以一条命令能覆盖所有目标。图 7-3 是这个结构。



连哪台机器、哪个端口，都写在运行时的请求里：隧道本身没有固定的目标。

图 7-3 动态转发，目标地址在运行时决定

动态转发的用途主要是两类。一类是临时改变出口位置，让流量从服务器那一侧出去。另一类是访问内网资源，浏览器通过 SOCKS 代理访问内网的多个站点，不用一个个配 `-L`。

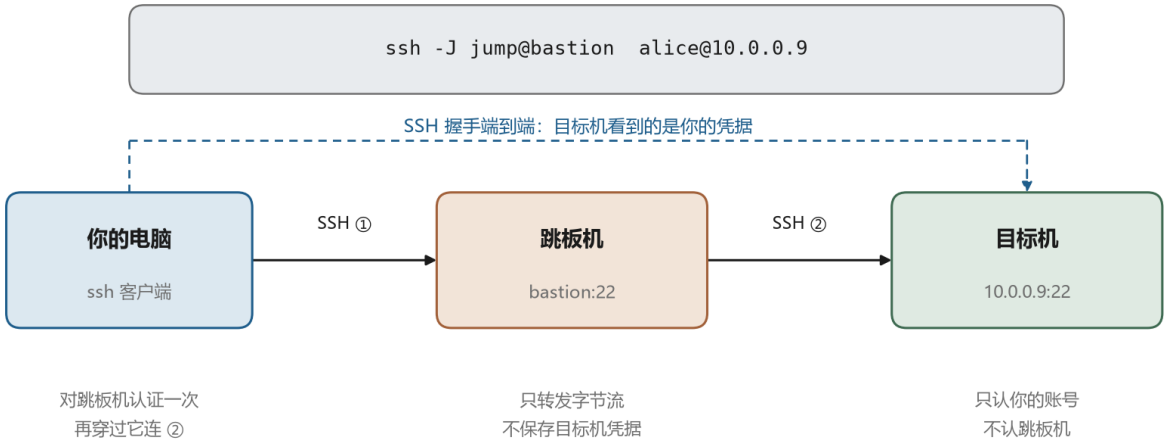
需要提醒的是，`-D` 只处理 TCP。有些应用用 UDP，比如某些 DNS 查询和视频通话，这些不会走 SOCKS 代理。另外，浏览器自己会做 DNS 解析，如果配置不当，目标域名可能在本地就被解析了，泄露你在访问什么。Firefox 可以在 `about:config` 里把 `network.proxy.socks_remote_dns` 设为 true，让 DNS 也走代理。

7.5 跳板与 ProxyJump

前面讲的转发是在一条已经建立的连接上做文章。还有一类需求是在建立连接的过程中绕路，也就是先连 A，再从 A 连 B。这件事由 `ProxyJump` 完成。

```
ssh -J alice@jump.example.com alice@10.0.0.5
```

这条命令先连跳板机，认证通过之后，在跳板机上再发起一条到 `10.0.0.5` 的连接，然后把两条连接拼接起来。你的客户端看到的是一条到 `10.0.0.5` 的 SSH 连接，实际上中间经过了跳板机。图 7-4 是这个结构。



第二段 TCP 由跳板机代开，两段都是 22 端口；对目标机来说，来源是跳板机的地址。

图 7-4 两跳连接，第二条连接由本机驱动，经跳板机转发

这里有个关键细节，第二条连接的认证是由你的本机完成的，只是数据包经过跳板机转发。跳板机上不需要放任何私钥，也不需要开启 Agent Forwarding。这是它比 `ssh -A jump` 再手动 `ssh` 更安全的地方。

`ProxyJump` 可以串联多级，用逗号分隔。

```
ssh -J alice@jump1,alice@jump2 alice@target
```

配置文件里的写法更清爽。

```
Host jump
    HostName jump.example.com
    User alice

Host lab
    HostName 10.0.0.5
    User alice
    ProxyJump jump
```

老一些的环境可能还在用 `ProxyCommand`，效果类似，写法更底层。

```
Host lab
    HostName 10.0.0.5
    ProxyCommand ssh -W %h:%p jump
```

`-W` 表示把标准输入输出接到目标地址的指定端口上，`%h` 和 `%p` 会被替换成主机名和端口。`ProxyJump` 本质上就是这个命令的封装，从 OpenSSH 7.3 起可用，现在应该优先用它。

7.6 转发与堡垒机的关系

第七章讲的这些技巧，在堡垒机环境里会遇到额外的限制，这不是偶然。

堡垒机的核心价值是审计。它需要知道你连了哪台机器、敲了什么命令、传了什么文件。如果允许端口转发，你就可以把一条数据库连接塞进 SSH 通道，堡垒机只能看到一串加密的字节流，看不到你执行了什么 SQL。同理，动态转发可以让浏览器访问内网任意网站，堡垒机完全无法审计。所以多数堡垒机的默认策略是禁止端口转发，JumpServer 在资产授权里就有“允许开启 SSH 端口转发”这样的开关，默认关闭。

这也解释了为什么用堡垒机时，一些平时很顺手的做法会失效。VS Code 的 Remote-SSH 需要建立端口转发才能工作，如果堡垒机禁用转发，连接会在某个阶段卡住或者报错。遇到这种情况，正确做法是让管理员在授权里打开对应的权限，或者在堡垒机上使用专门为图形化与 IDE 场景设计的接入方式。绕过限制自己搭隧道，既违反策略，也会让审计记录出现空洞。

7.7 让隧道稳定下来

隧道最容易出的问题是莫名其妙地断开。原因通常是中间的网络设备认为连接闲置太久，把 NAT 表项清掉了。表现是终端卡住，过一会儿报 `Connection reset by peer` 或者 `Broken pipe`。

解决办法是定期发心跳。客户端侧用 `ServerAliveInterval`，服务端侧用 `ClientAliveInterval`。

```
Host *
    ServerAliveInterval 30
    ServerAliveCountMax 3
    TCPKeepAlive yes
```

`ServerAliveInterval 30` 表示每 30 秒发一个加密的空包，`ServerAliveCountMax 3` 表示连续 3 次没有回应才断开。加了这两个选项，闲置的隧道能撑很久。

另一个常见问题是转发端口被占用。本地 8080 已经被别的程序用了，`-L` 会报 `bind: Address already in use`。换一个端口，或者用 `-L 127.0.0.1:0:...` 让系统随机分配。后者在脚本里很有用，配合 `-v` 能看到实际分配的端口号。

还有 `ExitOnForwardFailure` 这个选项值得打开。默认情况下，如果转发建立失败，SSH 仍

然会登录成功，只是隧道没起来，你会以为连上了。打开之后，转发失败就整体失败，脚本里更容易发现。

```
ssh -o ExitOnForwardFailure=yes -L 8080:10.0.0.9:80 jump
```

7.8 一条通道能装多少东西

回到连接层。一条 SSH 连接里可以开多个通道，每个通道独立流控。本地转发的每个 `-L` 是一个通道，远程转发是另一种通道，交互式 shell 也是一个通道。它们的数量受限于服务端的 `MaxSessions`，默认是 10。

```
ssh -L 8080:10.0.0.9:80 -L 3306:10.0.0.8:3306 -L 6379:10.0.0.7:6379 jump
```

这一条命令同时开了三个转发，加上登录 shell 本身用掉的通道，还没到上限。VS Code 的远程开发会开更多通道，文件服务、终端、扩展通信各占一路，所以用 IDE 连接时偶尔会碰到 `open failed: administratively prohibited: open failed` 这类报错，含义是通道数超了，调大 `MaxSessions` 即可。

理解了通道复用，也就能理解为什么一条 SSH 连接可以很“重”。它不只是一个终端，而是一条可以承载多种用途的管道。下一章换一个角度，看看当几十个人、几百台机器都需要这样一条管道时，事情会变成什么样。

第 三 篇

从一个人到一群人

一个人管三台机器，密钥放在自己电脑上就够了。三十个人管三百台机器，事情就变了样。

谁有哪台机器的权限，说不清。谁在什么时候登录过，查不到。同事离职三个月，他的公钥还躺在某台机器的 `authorized_keys` 里。生产库被误删，日志里只有一条 `root` 的操作记录，没人知道是谁。

这一篇讲的是组织层面的技术。堡垒机把分散的信任收拢到一个节点，用统一的认证、授权和审计来管理。VS Code 的远程开发则是另一个方向，它把开发环境搬到远端，让代码和数据待在一起，只把界面留在手边。

这两件事看起来不相关，底下的机制却是同一套。堡垒机的 SSH 网关在代理你的连接，VS Code 的远程扩展在复用你的 SSH 连接。理解了前两篇的连接原理，这一篇更多是在看它们怎么被组合、被限制、被记录。

第 8 章

为什么需要堡垒机

8.1 一台没人记得动过的机器

某个周三的下午，运维群里有人问，测试环境那台数据库怎么连不上了。查了一圈，发现机器还在，端口也在听，就是数据少了一半。翻日志，前一天晚上 23 点 40 分有一条 `DELETE` 语句，执行者用的是 `root`。问题是，团队里五个人都有 `root` 的密码，谁也说不清是谁执行的。有人猜是定时任务，有人猜是实习生误操作，最后不了了之。

这类事情在只有几台机器的小团队里发生概率不高，机器一多就变成常态。人是会流动的，项目是会交接的，三年前配的免密登录还留在某台机器的 `authorized_keys` 里，离职同事的笔记本上可能还存着生产库的私钥。等到出事，你甚至无法回答最基本的问题，谁在什么时候从哪台机器做了什么。

堡垒机就是为了回答这几个问题而出现的设备。它把“所有人直接连服务器”改成“所有人先连堡垒机，由堡垒机连服务器”，中间这个节点负责认证、授权、记录。

8.2 网状连接的代价

没有堡垒机的时候，连接关系是网状的。图 8-1 左边画的是一个小团队的样子，六台办公终端、八台服务器，理论上每条“人”和“机器”的组合都需要一条通路。

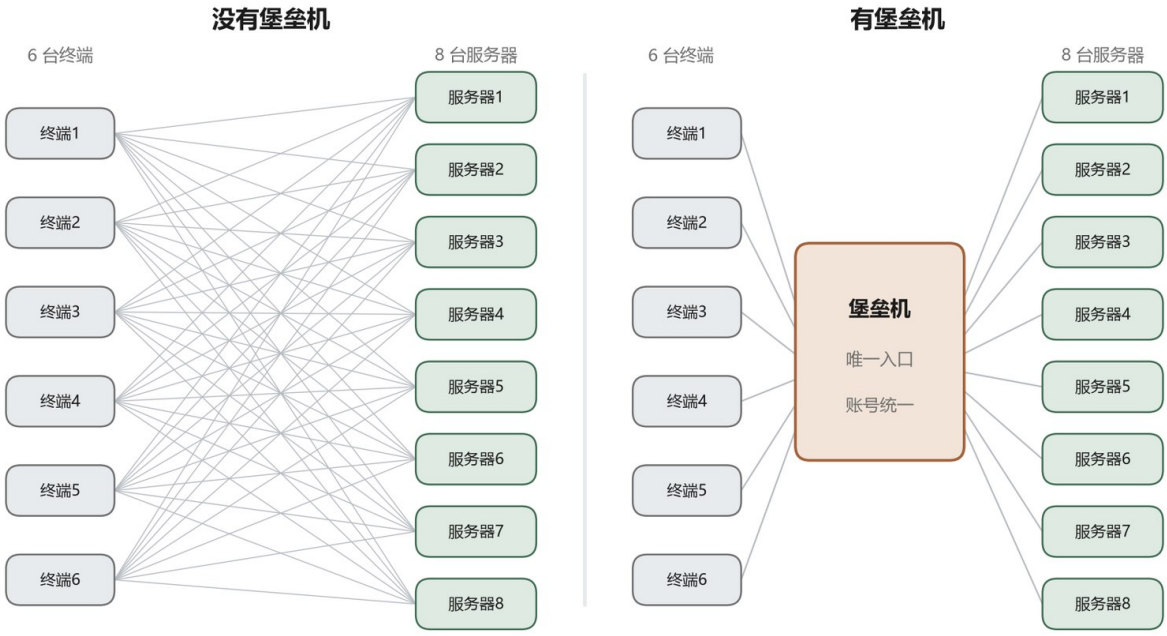


图 8-1 网状连接与星形连接的对比

网状结构带来三个具体问题。

第一个是凭据扩散。每台服务器上都要放每个人的公钥，每个人手里都要有每台服务器的地址和账号。新同事入职，要在一周内把几十台机器的访问配好。同事离职，要在一夜之间把几十台机器上的公钥删干净。漏掉一台，就是一条长期存在的后门。

第二个是权限失控。为了方便，很多人直接用 `root` 登录，或者在 `sudoers` 里给自己开全权限。时间一长，谁有什么权限没人说得清。等到要做权限审计，只能一台台机器去看 `authorized_keys` 和 `sudoers`，工作量巨大。

第三个是没有记录。Linux 的命令历史（`~/.bash_history`）是用户自己维护的，可以删、可以改、可以不写入。多个人共用同一个账号时，历史记录混在一起，无法归属到具体的人。真出了事故，只能靠猜。

改成星形之后，这三个问题都有了着力点，如图 8-1 右边。凭据只在堡垒机上有一份，人员进出只改堡垒机的授权。权限集中在授权表里，可以查询、可以回收。所有会话经过堡垒机，天然可以记录。

8.3 4A

堡垒机的功能通常用四个词概括，英文都以 A 开头，所以叫 4A。图 8-2 是这四个能力的示意。

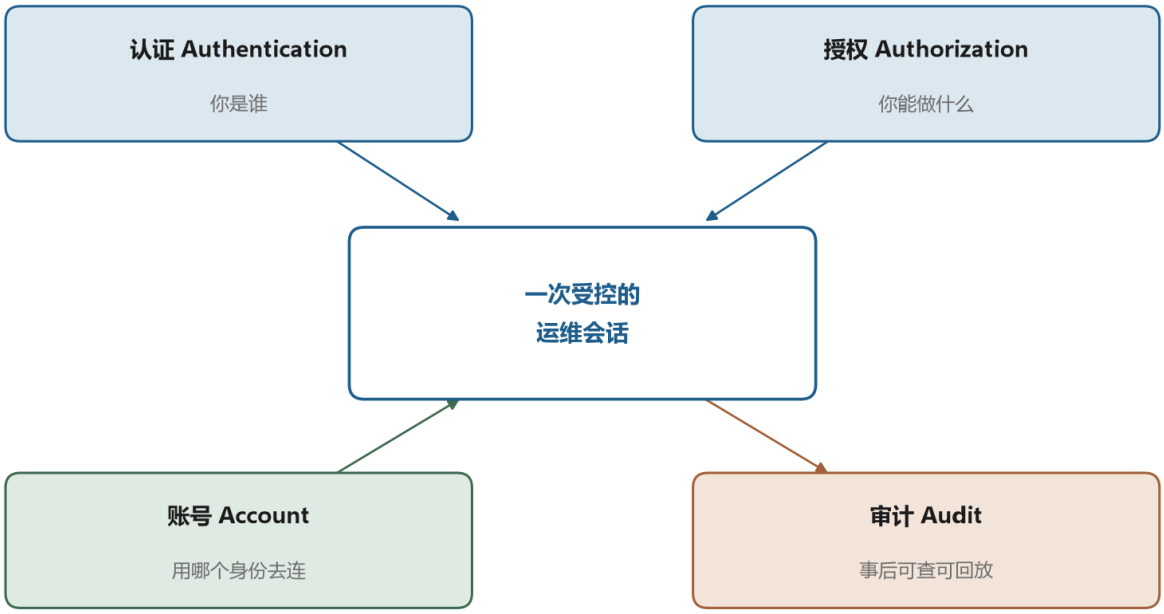


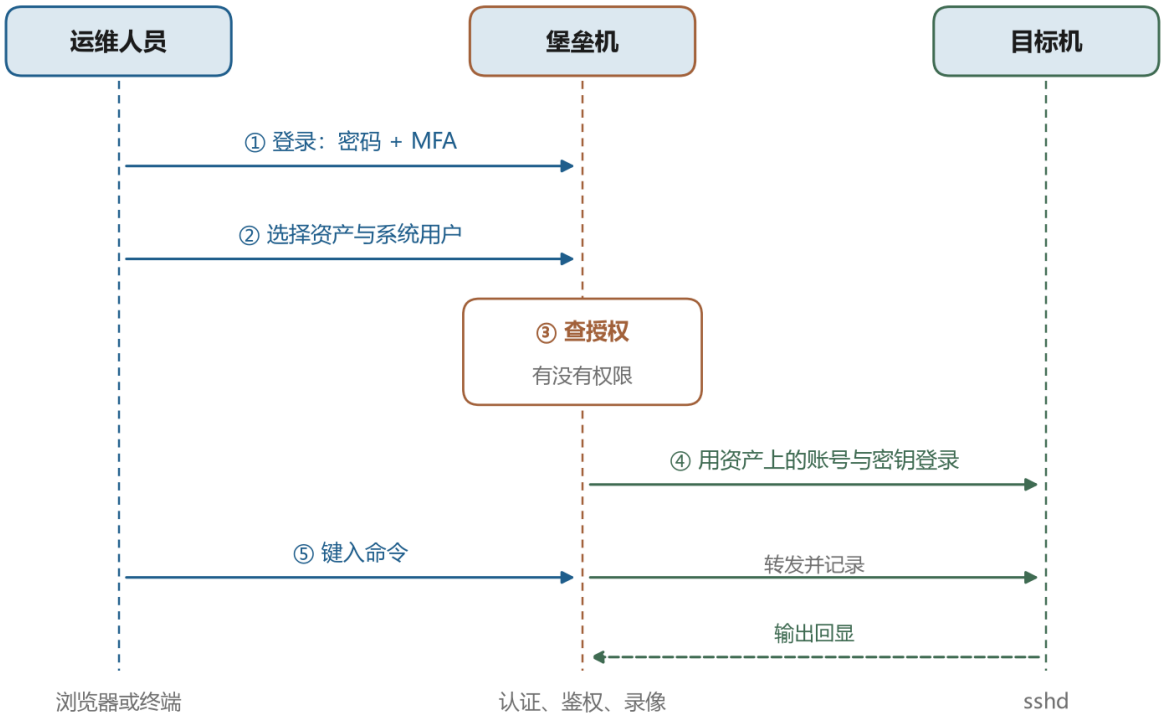
图 8-2 认证、授权、账号、审计构成一个闭环

认证解决“你是谁”。除了密码，还要支持公钥、短信验证码、动态口令、企业微信扫码，重要系统要求双因素。认证的意义在于把操作绑定到一个具体的人，而不是一个共用的账号。

授权解决“你能做什么”。给某个人分配某几台机器，允许他用某个系统账号登录，允许上传下载但不允许复制粘贴，这些都属于授权的范围。授权应该是可查询、可回收、有有效期的。

账号解决“用什么身份连机器”。这里有两层，一层是堡垒机自己的账号，一层是目标机器上的账号。用户登录堡垒机用自己的账号，堡垒机连目标机时用目标机上的账号。后者的密码或私钥由堡垒机托管，用户看不到。这就是所谓的“密码托管”，也是堡垒机最核心的价值之一。

审计解决“你做了什么”。会话录像、命令记录、文件传输记录、登录日志，都属于审计的范畴。审计数据要防篡改，要有保留期限，要能检索。



会话结束后，录像文件与命令记录一起入库，可按时间检索、回放。

图 8-3 一次受审计的登录过程

图 8-3 把一次登录的过程画了出来。用户先认证到堡垒机，选择要访问的资产，堡垒机检查他有没有对应授权，有的话用托管的凭据连到目标机，同时把整个会话录下来。用户看到的界面和直接连目标机几乎一样，区别是所有操作都留了痕。

8.4 流量怎么走

堡垒机接入流量的方式有两种，理解它们的区别对排查问题很重要。

第一种叫网关模式，也叫代理模式。用户直接用 SSH 客户端连堡垒机的某个端口，堡垒机在中间做协议代理。用户的连接终止在堡垒机上，堡垒机再向目标机发起一条新连接。这种方式下，堡垒机能完整解析 SSH 协议，所以能做到命令级的过滤和记录。代价是兼容性，某些客户端特性可能不支持，比如复杂的端口转发。

第二种叫直连模式，用户先通过 Web 界面拿到一张“通行证”，再直连目标机，堡垒机只做认证和记录。这种方式对网络要求低，性能好，但记录能力弱，因为流量不经过堡垒机。

多数企业用网关模式，因为它能提供完整的审计。JumpServer 的 Koko 组件就是干这个的，

第九章会详细讲。

这里有一个网络设计上的细节。网关模式下，堡垒机需要能够访问所有目标资产。它通常部署在运维区，同时能通向外网和内网。如果堡垒机到某台资产不通，用户会看到“连接超时”，而这条链路上其实有两段，用户到堡垒机、堡垒机到资产。排查时要分清是哪一段断了，方法是在堡垒机上直接 `nc -vz 资产IP 端口` 试一下。

8.5 合规要求

堡垒机在国内的普及和等级保护制度有直接关系。等保 2.0 三级对运维审计提出了明确要求，包括对远程运维进行身份鉴别、访问控制、操作审计，审计记录要包含日期时间、用户、事件类型、结果，并且要保护记录不被删除、修改。

另一条常被引用的是网络安全法的要求，网络日志留存不少于六个月。这条规定直接决定了堡垒机的存储设计，录像文件要能存半年，容量规划必须按这个来算。

合规要求解释了一些看起来不合理的限制。比如为什么不能直接把生产库的公网端口开出去，为什么运维必须走堡垒机而不能用 VPN 直连，为什么堡垒机要限制文件上传下载。这些限制不是为了给工程师添麻烦，而是审计上的硬性要求。

8.6 堡垒机保护不了什么

说了这么多好处，也要说清楚边界。堡垒机不是万能的，把它当成安全问题的终极答案会带来新的风险。

它防不住有权限的人做坏事。一个有生产库删除权限的工程师，通过堡垒机删了数据，录像会记录这一切，但数据已经没了。堡垒机的作用是事后追责和事前威慑，不是事前阻止。真正减少这类风险要靠权限最小化和审批流程。

它本身是单点。所有运维流量都经过它，它挂了，所有人都进不去。所以生产环境里的堡垒机通常要做高可用，至少两台，前面挂负载均衡。这一套的成本不低，小团队往往承担不起。

它集中了所有凭据。堡垒机上存着几百台机器的密码和私钥，一旦被攻破，后果比没有堡垒机更严重。这也是为什么堡垒机的部署要求特别高，要单独划分安全域，要限制访问来源，要定期打补丁，数据库要加密。

它可能和生产环境抢带宽。网关模式下所有流量都过堡垒机，传大文件时带宽是瓶颈。有些团队因此给堡垒机单独开了一条链路，或者对文件传输做限速。

理解了这些边界，再看堡垒机的配置项就不会觉得是繁文缛节。限制复制粘贴是因为剪贴板里的内容不会被录像记录；限制文件下载是因为下载走的是另一条通道；要求双因素是因为单靠密码无法确认操作者是不是账号的主人。每一条限制背后都有一个具体的攻击场景。

第 9 章

JumpServer 的内部构造

9.1 组件地图

JumpServer 是国内使用最广的开源堡垒机，用 Python 和 Go 写成，前后端分离，按协议类型拆成多个组件。图 9-1 是它的组件结构。

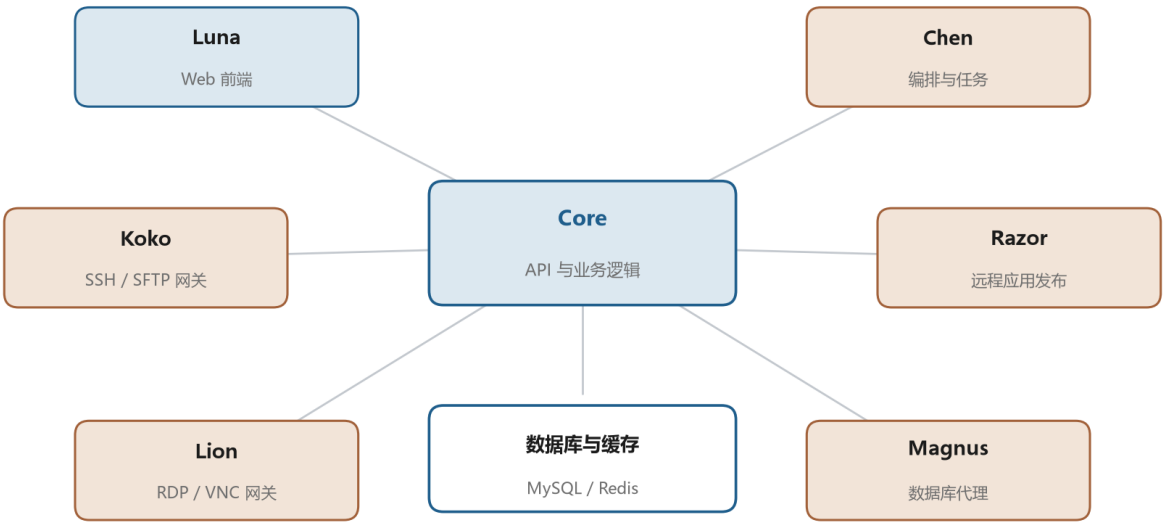


图 9-1 JumpServer 的主要组件与它们的分工

Core 是后端服务，用 Django 写成，提供 REST API，管理资产、用户、授权、会话记录。所有组件的配置都从这里取，所有操作记录都往这里写。它连接一个关系型数据库存元数据，连接 Redis 做缓存和消息队列。

Luna 是 Web 前端，用 Vue 写成。你在浏览器里看到的资产列表、会话窗口、录像回放都是它。它自己不碰目标机器，所有操作都通过 API 转给后端或者对应的网关组件。

Koko 是字符协议网关，用 Go 写成，处理 SSH、SFTP、Telnet。用户用 SSH 客户端连堡垒机时，连的就是它。它负责解析 SSH 协议、校验授权、用托管的凭据连目标机、把整个会话录下来。

Lion 是图形协议网关，处理 RDP、VNC。原理类似，只是它代理的是图形协议，录像是一段视频。

Magnus 是数据库代理，把 MySQL、PostgreSQL、Oracle、Redis 这些协议的连接接进来，做账号托管和 SQL 审计。

Razor 是远程应用发布，把只能跑在 Windows 上的客户端程序（比如某些管理工具）包装

成网页里的一个窗口。

Chen 是任务编排，用来批量执行脚本、做定期巡检，本质上是把 Ansible 包装成了可视化界面。

这套拆分的好处是各协议互不影响，SSH 网关出问题不影响 RDP。缺点是部署复杂，组件多了之后，端口、证书、配置同步都容易出问题。官方提供了一键安装脚本和容器化部署方案，生产环境通常用后者。

9.2 资产模型的四层

JumpServer 的权限模型有四个核心概念，理解了它们，界面上的大部分操作就不用猜了。图 9-2 是它们的关系。

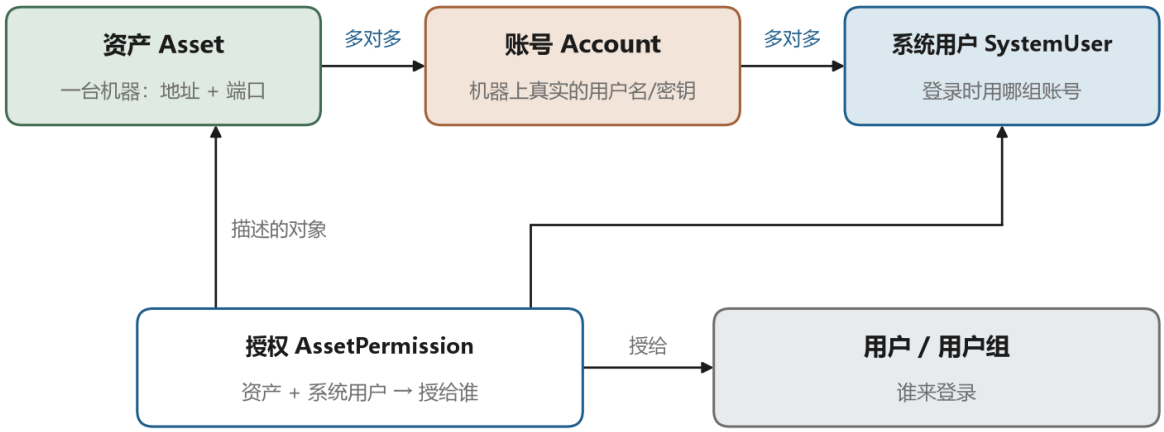


图 9-2 资产、系统用户、账号与授权的关系

资产（Asset）是一台被管理的机器，记录它的地址、端口、平台类型（Linux、Windows、网络设备）、节点归属。节点是一棵树的节点，用来给资产分组，比如“生产环境 / 华东 / 数据库”。

系统用户（SystemUser）是“登录资产时使用的身份模板”。它记录用户名、认证方式（密码或者密钥）、密码或私钥、提权方式（sudo 或者 su）。系统用户分为普通用户和特权用户，特权用户用来做自动化采集，普通用户用来给人登录。

账号（Account）是某个资产上真实存在的账号。系统用户和资产结合，就产生一条账号记录，里面存着这台机器上这个用户的实际密码或密钥。同一个系统用户在不同资产上的密码可以不同，JumpServer 会分别保存。

授权（AssetPermission）是把“谁”和“哪些资产、用哪个系统用户、能做哪些动作”绑

在一起的记录。授权的对象可以是用户，也可以是用户组。动作包括连接、上传、下载、复制、粘贴、删除等等，粒度很细。

这四层的关系可以用一句话概括，授权决定某个人能用某个系统用户登录某些资产，登录时用的具体凭据来自账号表。新人入职，管理员做的是建用户、加用户组、给用户组授权，不用碰任何一台真实机器。

9.3 三合一用户名

现在可以解释第一章那个奇怪的配置了。

```
User alice@ubuntu@119.45.34.99
```

JumpServer 的 SSH 网关需要在一开始就知道三件事，谁在登录、用哪个系统用户、连哪台资产。SSH 协议的用户名字段只有一个字符串，于是它把三件事拼在一起，用 at 符号分隔。图 9-3 是这个解析过程。

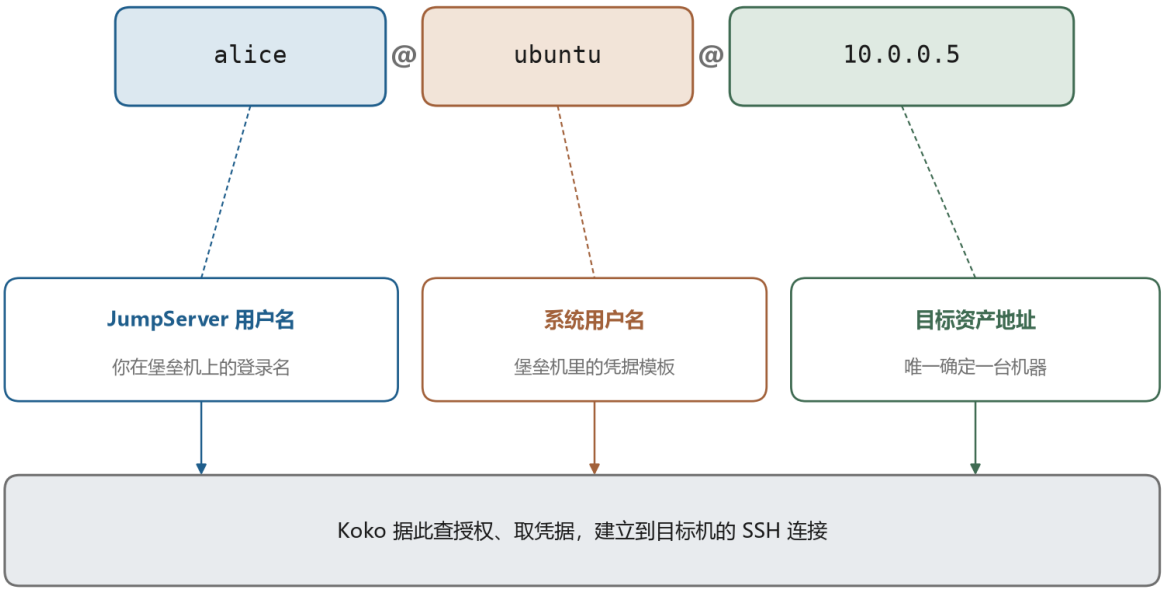


图 9-3 三合一用户名的解析

第一段 `alice` 是 JumpServer 上的用户名，用来认证你本人。第二段 `ubuntu` 是系统用户名，对应 JumpServer 里配置的一个系统用户，它决定了用哪套凭据去连目标机。第三段 `119.45.34.99` 是资产地址，Koko 用它来定位具体是哪台机器。

这个格式对 SSH 客户端来说是合法的，因为用户名只是一个字符串，协议没有规定不能包含 at 符号。但它在两个地方会带来麻烦。一是有些工具会按第一个 at 符号切分用户名和主机，导致解析错误。二是命令行里如果涉及 URL 编码，at 符号需要转义成 `%40`。VS Code 的 Remote-SSH 属于前者，所以它的配置文件里要特别小心，不能让它把用户名再拆一次。第十章会讲它怎么处理。

还要注意一点，第二段是“系统用户名”，不是目标机上真实的 Linux 用户名。系统用户里记录的用户名可能和它同名，也可能不同。你看到 `ubuntu`，不代表目标机上的账号一定叫 `ubuntu`，那只是 JumpServer 里给这套凭据起的名字。这一点在实际排查时很容易搞混。

9.4 一次 SSH 网关会话

把上面这些串起来，一次通过网关的登录是这样走的。

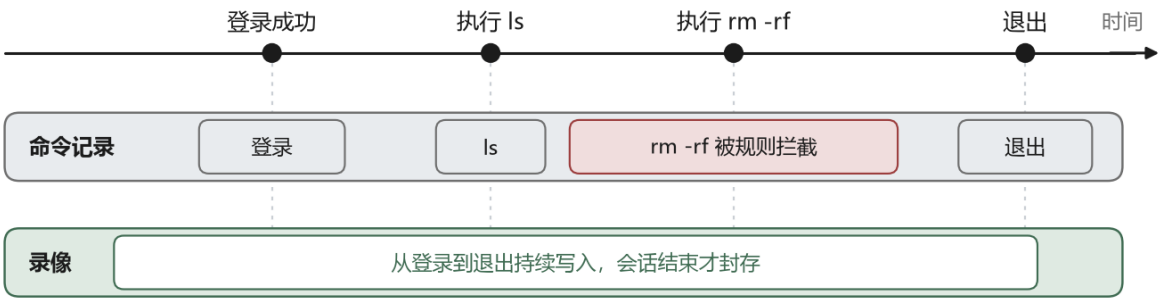
你的客户端连到堡垒机的 58222 端口，Koko 接受连接。它先和你完成 SSH 握手，用你上传的公钥或者密码认证你的身份，这一步验证的是“你是 alice”。认证通过后，Koko 解析用户名里的三段信息，去数据库查 alice 有没有权限用 `ubuntu` 这个系统用户连接 `119.45.34.99` 这台资产。有权限的话，它从账号表里取出这台机器上这个账号的凭据，向目标机的 22 端口发起一条新的 SSH 连接。这条连接用的是托管的密码或私钥，你完全看不到。

两条连接都建立之后，Koko 在中间做双向转发，同时把整个会话录下来。你敲的每一个字符、目标机返回的每一段输出，都写进录像文件。录像不是简单的文本记录，而是带时间戳的终端会话流，回放时能看到打字的速度和节奏。

如果授权里配置了命令过滤，Koko 还会在转发之前检查你输入的命令。匹配到禁用规则的命令会被拦下来，你在终端里会看到一段提示，命令不会到达目标机。这个检查和你在目标机上配置的 `sudo` 规则是两回事，前者在堡垒机侧，后者在目标机侧，两者都可能拦住你。

9.5 命令过滤与会话录像

图 9-4 是一次会话的时间轴，标出了几种记录发生的位置。



命令记录实时落库，方便检索；录像只在会话结束后封存，用于还原当时到底看到了什么。

图 9-4 会话过程中命令记录与录像的写入时机

命令记录是结构化的，每执行一条命令就写一条，包含时间、用户、资产、命令内容、执行结果。它适合检索，比如查“上周谁在这台机器上执行过 rm”。但命令记录有局限，它抓的是你敲进去的字符串，如果命令是通过脚本执行的，或者用了交互式程序（比如 `vi`、`mysql` 客户端），记录就不完整。

会话录像弥补了这个缺口。它记录的是终端上的原始字节流，回放时把这些字节按时间重新

播放一遍，看起来就像当时的屏幕。对于 `vi` 里改了什么、`mysql` 里执行了什么 SQL，录像都能还原。

两者结合，审计才算完整。查问题时先看命令记录定位到时间段，再调出录像看具体操作。这也解释了为什么录像文件占空间，一个小时的交互式会话，录像可能有几 MB 到几十 MB，按六个月留存算，容量规划要认真做。

9.6 凭据是怎么存的

堡垒机最敏感的数据是托管的密码和私钥。JumpServer 用对称加密保存这些内容，加密密钥放在配置文件或者环境变量里，和数据库分开存放。这样即使数据库被拖走，没有密钥也解不开。

```
SECRET_KEY=...
BOOTSTRAP_TOKEN=...
```

这两个值在安装时生成，写在 `/opt/jumpserver/config/config.txt` 里。很多人图省事，把它们留在默认值或者提交到了代码仓库，这是严重的错误。拿到这两个值加上数据库，等于拿到所有资产的登录权限。

更麻烦的是密钥轮换。改了加密密钥，已经存进去的密码全部解不开，需要重新录入。所以正规做法是在部署初期就规划好密钥管理，用密钥管理服务或者外部保险箱，而不是写死在配置文件里。

堡垒机托管私钥还带来一个架构上的变化。原来每个人手里有自己的私钥，现在私钥集中在堡垒机上。这意味着堡垒机本身成了最高权限的持有者，它的安全等级必须高于它管理的所有资产。这条要求在实际部署中经常被忽略，堡垒机和被管理的机器放在同一个网段、用同一套弱密码，等于把锁和钥匙放在同一个抽屉里。

9.7 部署时的几个关键项

JumpServer 的配置文件里有几项值得提前想清楚。

网关端口是其中之一。默认的 SSH 网关端口是 2222，示例里用的是 58222。改端口的意义不大，但能减少扫描噪音。真正重要的是把这个端口只对运维网段开放，不要暴露在公网。

另一个是 `SECURITY_INSECURE_MODE` 之类的开关，用来允许或禁止某些不安全的行为，比如跳过主机密钥校验。这些开关在生产环境必须关闭，否则等于放弃了中间人防护。

还有会话保留策略。录像文件默认存在本地磁盘，容量有限。生产环境要配置成存到对象存储，第四章讲的对象存储在这里派上了用场。这也顺带解释了为什么这本书把对象存储放在最后一篇，它和堡垒机是同一套运维体系的两端，一端管人怎么进来，一端管数据存在哪里。

第 10 章

VS Code 是怎么连过去的

10.1 一个编辑器，四个进程

VS Code 看起来是一个程序，实际上是一组进程。打开任务管理器，能看到好几个 Code.exe。这个结构继承自 Electron，也决定了远程开发可能的形态。

最外层是主进程，负责窗口、菜单、文件对话框、更新。中间是渲染进程，每个窗口一个，负责画界面。再往下是扩展宿主进程，所有扩展的代码都跑在这里，一个扩展崩溃不会把界面带崩。还有一个 pty 宿主进程，专门管理终端，让终端里的程序运行在独立进程里。

知道这个结构，就明白远程开发要解决什么问题。你希望代码在远端，编译和运行在远端，但界面和键盘在自己眼前。最直接的做法是把整个 VS Code 搬到远端，界面通过远程桌面显示，但那样延迟高、体验差。VS Code 选择的方案是把一部分进程搬到远端，本机只留界面。

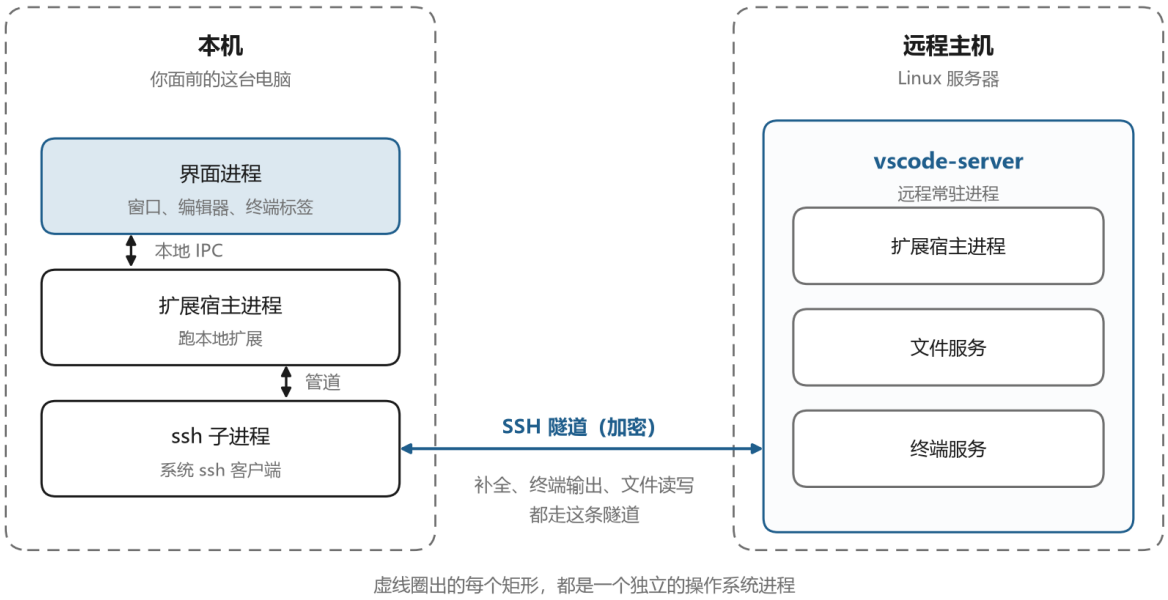


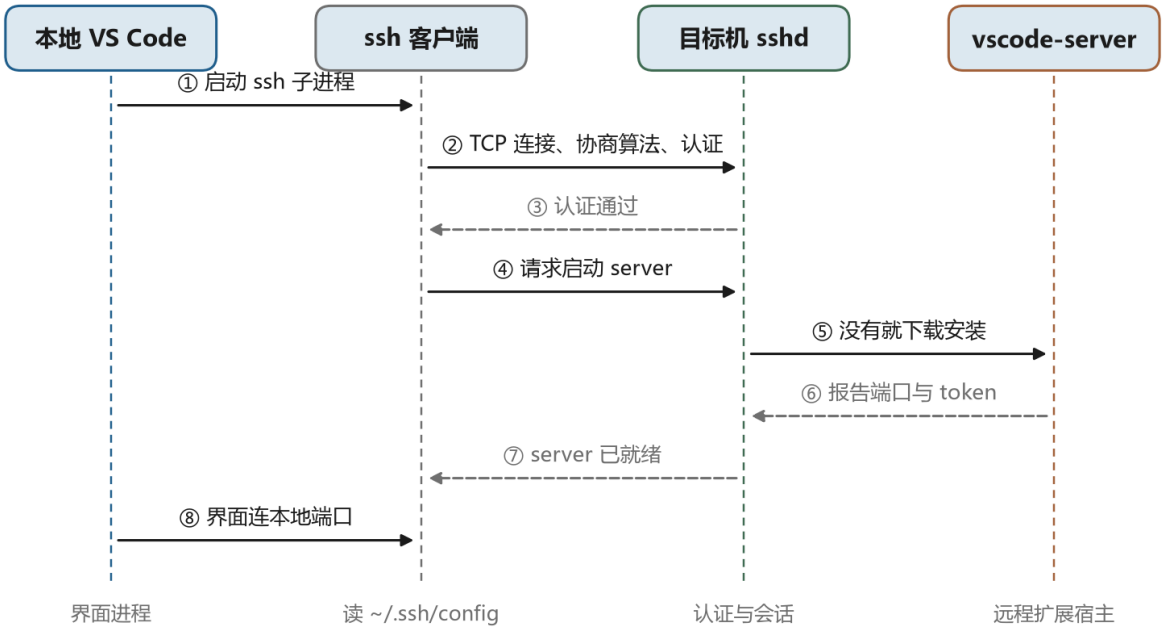
图 10-1 VS Code 远程开发时本机与远端的进程分工

图 10-1 是这个分工。本机保留界面进程和一部分扩展，远端跑一个叫 `vscode-server` 的服务，里面有远端扩展宿主、文件服务、终端服务。两者之间通过一条加密通道通信，通道上跑的是普通的 HTTP 和 WebSocket。

扩展被分成两类。只影响界面的扩展，比如主题、图标、快捷键提示，跑在本机。需要读写文件、调用编译器、访问远端的扩展，比如语言服务、调试器、Jupyter，跑在远端。VS Code 会根据扩展声明的 `extensionKind` 自动决定装在哪边，也可以手工指定。

10.2 连接时发生了什么

Remote-SSH 扩展做的事情，可以拆成图 10-2 里的几步。



箭头方向就是发起方；排错时先确定卡在第几步

图 10-2 Remote-SSH 建立连接的过程

你在界面上选一个主机，扩展读 `~/.ssh/config`，把别名展开成具体参数。然后它调用系统的 `ssh` 客户端，也就是你平时在终端里用的那个，建立一条连接。注意这一步用的是系统自带的 ssh，不是自己实现的协议栈，所以你在 `config` 里写的所有选项、你的 `known_hosts`、你的 `ssh-agent`，全都照常生效。

连上之后，扩展在远端执行一小段脚本来检查环境。它检查三件事，`~/.vscode-server` 目录是否存在、里面的版本是否匹配当前 VS Code 的版本、远端能不能正常执行命令。版本不匹配或者目录不存在，它就下载一个对应版本的 server 包，解压到 `~/.vscode-server/bin/<commit-id>/` 下面。

下载这一步是最容易失败的地方。远端机器需要能访问更新服务器，如果它在一个没有外网的机房里，下载就会超时。解决办法有两个，一是在本地下载好安装包，手工传上去解压；二是配置代理，让远端的下载走你的本机。后者的做法是在 `config` 里加 `Remote.SSH.RemoteServerListenOnSocket` 之类的选项，或者更简单地用第七章的本地转发把更新服务器的地址映射过去。

server 起来之后，它会监听一个本地端口或者一个 Unix socket。扩展在本机也启动一个监听，然后通过 SSH 的端口转发把两边接起来。界面通过 `127.0.0.1` 上的这个端口和远端通信，对你来说完全透明。

10.3 那条隧道的两端

图 10-3 画的是这条隧道两端的细节。

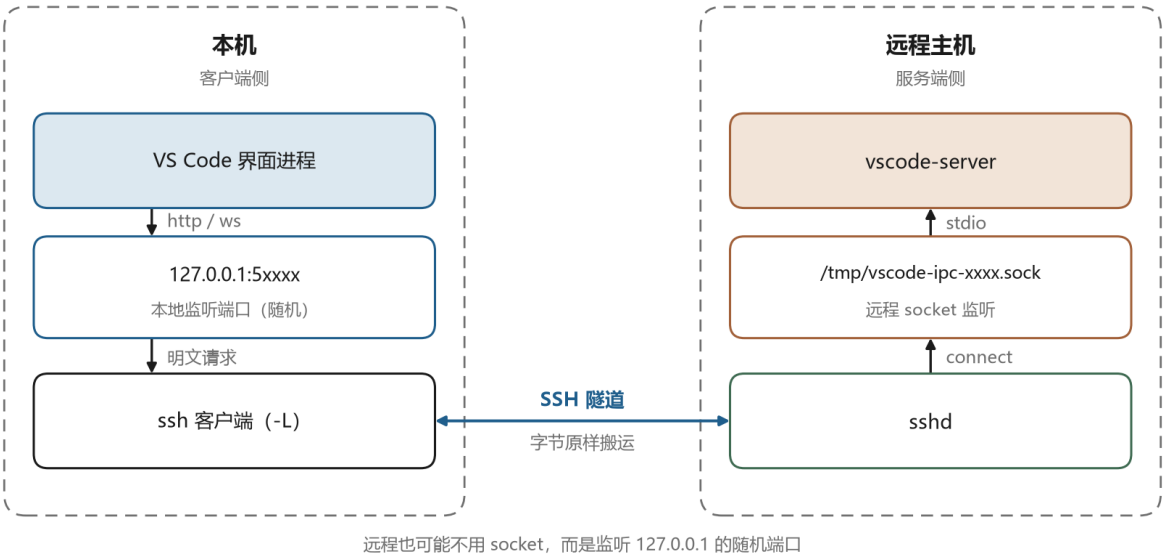


图 10-3 Remote-SSH 的端口转发，本地随机端口接到远端 socket

本机这一端，VS Code 随机选一个空闲端口，通常是 5 万多的一个数字，监听在 `127.0.0.1`。远端那一端，`vscode-server` 监听一个 Unix socket 文件，路径在 `~/.vscode-server/` 下面，或者监听一个 `127.0.0.1` 上的随机端口。中间的转发由 `ssh` 客户端完成，用的是 `-L` 或 `-R` 的等价机制。

这个设计解释了几个现象。第一，你在远端用 `ss -lntp` 会看到 `vscode-server` 监听在回环地址上，端口号是随机的，从外部连不进去。第二，如果你在堡垒机上禁用了端口转发，Remote-SSH 会在建立隧道这一步失败，报错通常出现在输出面板里，而不是终端里。第三，隧道断了之后界面会显示“正在重新连接”，因为 WebSocket 断了，VS Code 会尝试重连并重新建立端口转发。

10.4 ssh_config 的解析规则

Remote-SSH 完全依赖系统的 ssh 客户端，所以 `config` 文件的解析规则就是 OpenSSH 的规则，值得花点时间搞清楚。

文件从上往下读，对每一段 `Host`，检查当前连接的主机名（也可能是命令行里给的别名）是否匹配。匹配的段里的选项生效，后面的段可以覆盖前面的同名选项。`Host *` 匹配所有主机，所以它通常放在最前面作为默认值。

```
Host *
    ServerAliveInterval 30

Host lab-*
    User alice
    IdentityFile ~/.ssh/id_ed25519_lab

Host lab-gpu
    HostName 10.0.0.9
    ProxyJump jump
```

上面的例子里，连 `lab-gpu` 时，三段都匹配。`ServerAliveInterval` 来自第一段，`User` 和 `IdentityFile` 来自第二段，`HostName` 和 `ProxyJump` 来自第三段。这就是为什么默认值要写在最前面。

支持的写法还有 `Include`，可以把配置拆成多个文件。

```
Include ~/.ssh/config.d/*
```

OpenSSH 7.3 起支持，通配符和 `~` 都能用。企业里常用它把不同环境的配置分开管理。

几个替换符号在写 `ProxyCommand` 时会用到。`%h` 是目标主机名，`%p` 是端口，`%r` 是用户名，`%d` 是本机的家目录。写 `ProxyCommand ssh -W %h:%p jump` 时，这些符号会被替换成实际值。

还有两个容易忽略的点。一是 `HostName` 不写时默认等于 `Host` 后面的名字，所以 `Host lab` 加 `HostName 10.0.0.5` 两者都要写清楚，别指望它能猜。二是 `Host` 后面的名字区分大小写，`Lab` 和 `lab` 是两个不同的别名。

10.5 逐行拆解那份配置

现在回到第一章那份让人困惑的配置。图 10-4 把每一行的作用标了出来。

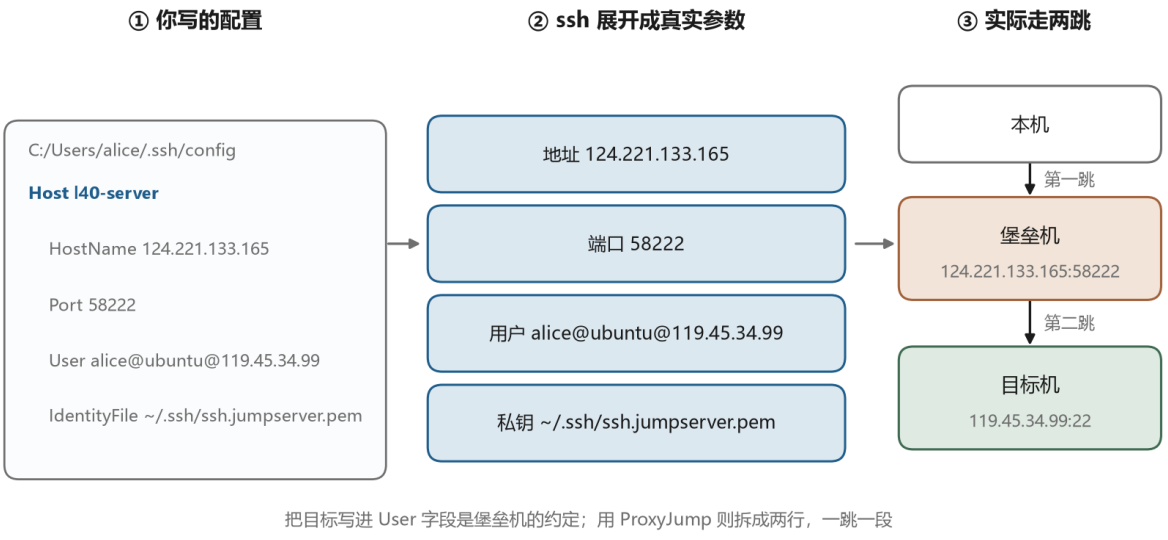


图 10-4 配置文件的每一行在做什么

```
Host l40-server
    HostName 124.221.133.165
    User alice@ubuntu@119.45.34.99
    Port 58222
    IdentityFile C:/Users/alice/.ssh/ssh.jumpserver.pem
```

`Host l40-server` 定义了一个别名，你在 VS Code 里选的就是这个名字，命令行里 `ssh l40-server` 用的也是它。名字本身可以随便起，只要自己记得住。

`HostName 124.221.133.165` 是真正要连接的地址。这里是堡垒机的公网地址，不是你目标机的地址。这一点是整份配置里最容易误解的地方。

`User alice@ubuntu@119.45.34.99` 是三合一用户名，第九章讲过它的三段含义。SSH 客户端不解析这个字符串，它原样发给服务器，由堡垒机去解析。

`Port 58222` 是堡垒机 SSH 网关的端口。不是 22，因为堡垒机的 22 端口通常留给系统自身的运维，网关用另一个端口。

`IdentityFile` 指向你在堡垒机上注册的公钥对应的私钥。这个私钥认证的是你本人，也就是 `alice` 这个堡垒机账号，不是目标机上的账号。

把五行合起来读，意思是“连接 124.221.133.165 的 58222 端口，用这把私钥证明我是 alice，然后告诉它我要用 ubuntu 这个系统用户去连 119.45.34.99”。一行配置里塞了两次认证、三段信息，难怪第一次看会懵。

这里有一个实际会遇到的问题。VS Code 的 Remote-SSH 在内部会解析用户名，某些版本会把 `User` 按第一个 at 符号切开，导致只把 `alice` 发过去。如果遇到这种版本，解决办法是在 `config` 里用 `User alice@ubuntu@119.45.34.99` 的同时，加上 `Remote.SSH.UseLocalServer` 或者改用 `ProxyCommand` 手工拼参数。具体做法随版本变化，判断依据是输出面板里的日志，看它实际发出的用户名是什么。

10.6 出问题时看哪里

Remote-SSH 的错误提示往往很笼统，只说“无法连接”。真正的细节在输出面板里。

打开方式是菜单里的“视图 → 输出”，然后在右侧下拉里选 “Remote - SSH”。这里会打印完整的 ssh 命令行、连接过程、每一步的输出。它其实就是在替你执行一条 `ssh` 命令，把参数都拼好了。

```
Running script with connection command: ssh -T -D 5xxx "l40-server" sh
```

看到这一行，就可以把它复制到终端里手工执行，加上 `-v` 看更详细的输出。这是排查最快的路径，因为你在终端里能拿到完整的调试信息，而输出面板有时候会截断。

几个常见故障和对应的原因。报“Could not establish connection”，先看是不是网络不通或者端口写错。报“Permission denied”，检查私钥路径、私钥权限、堡垒机上的公钥是否

配对。卡在“Downloading VS Code Server”，是远端下载失败，检查外网连通性或者手工安装。连上了但界面一直转圈，多半是端口转发被禁用，或者远端的 `vscode-server` 进程僵死。

清理远端残留是另一个常用操作。有时候 `vscode-server` 的进程卡住，删掉目录重来最快。

```
ls ~/.vscode-server/bin/
ps -ef | grep vscode-server
rm -rf ~/.vscode-server
```

VS Code 里还有一个命令叫 “Kill VS Code Server on Host”，效果一样，不用记路径。执行之后重新连接，它会重新下载并启动。

最后提醒一句资源问题。`vscode-server` 会占内存，语言服务和索引在大项目上能吃掉几个 GB。如果在共享的 GPU 机器上开发，注意别让它把内存吃满，导致训练任务被 OOM 杀掉。可以在远端限制扩展数量，只装必需的语言支持。

第 四 篇

数据的另一端

终端这一头是人，另一头是机器。机器跑起来之后会产生数据，数据要存下来，要能被下一台机器读到。

几百 GB 的训练集，几十万个日志文件，每天新增的检查点，这些东西放在本地磁盘上不现实。它们最终都去了对象存储。对象存储的接口简单到只有几个 HTTP 方法，可用起来却处处是坑，签名错一个字节就是 403，权限少一条策略就是拒绝访问。

这一篇把对象存储拆开看。先看它的数据模型为什么和文件系统不一样，再看分片上传、元数据、生命周期这些日常会用到的机制，最后把签名算法完整推导一遍。推导的过程有点长，但值得跟一遍，因为它是所有 403 的根源。

最后一章会把四篇的内容串成一条链路，从你按下连接按钮，到训练脚本把模型写回桶里。那条链路上的每一段，你都应该能说清楚它在做什么。

第 11 章

对象存储的模型

11.1 为什么不用文件夹

服务器上的数据通常放在文件系统里，一个目录套一个目录，文件有名字、有路径、有属性。这套模型用了几十年，很成熟。但当数据量到了几十 TB、文件数到了几千万，文件系统开始吃力。目录树的元数据要放在内存里，几千万个 inode 会把内存吃光；单机的容量有上限，扩容要停机；跨机器共享要靠 NFS，性能和稳定性都不理想。

对象存储是为这个场景设计的。它放弃了目录树，放弃了随机写，放弃了文件锁，换来的是近乎无限的容量、很高的持久性和简单的 HTTP 接口。代价是你不能像本地文件那样 `append` 一段内容，也不能 `rename` 一个目录，每次操作都是整体替换一个对象。

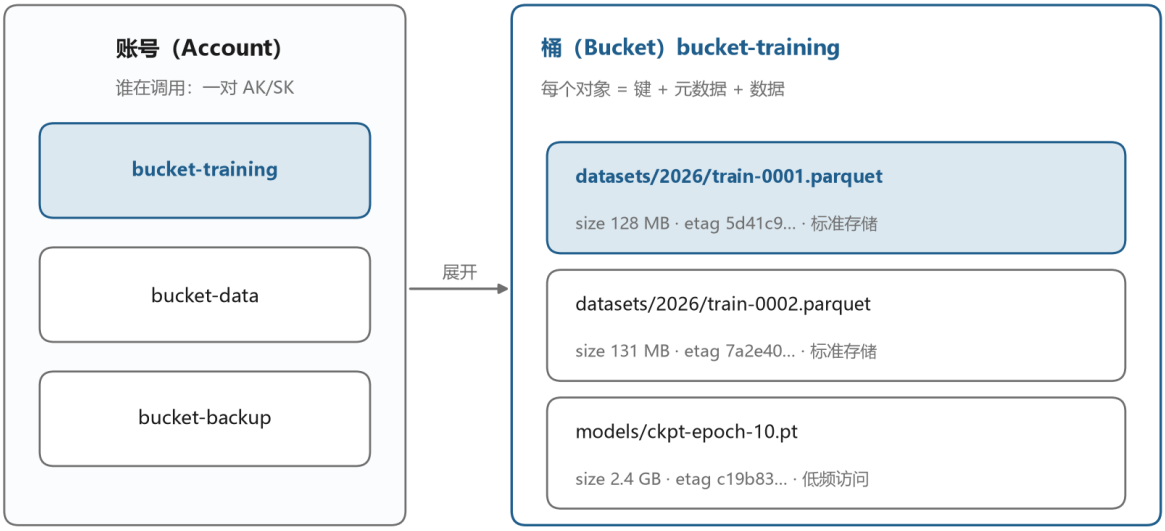


图 11-1 账号、存储桶与对象的层次

图 11-1 是它的数据模型。最上层是账号，账号下面可以建多个存储桶（Bucket），桶里放对象（Object）。对象由三部分组成，一个键（Key）、一段数据、一组元数据。键是一个字符串，可以包含斜杠，看起来像路径，但它不是路径。

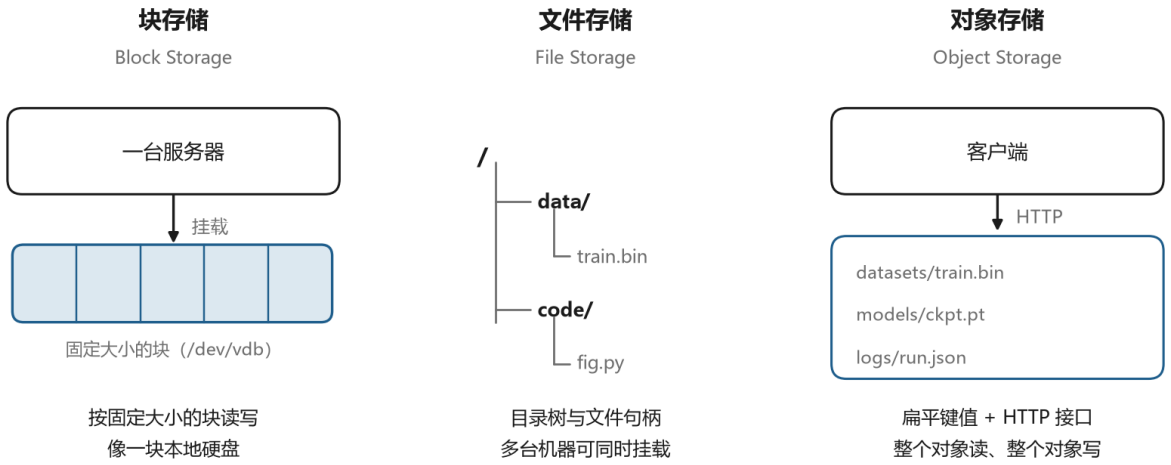
```
logs/2026/11/04/app.log
```

这个键在对象存储里就是一个完整的名字，没有 `logs` 这个目录。控制台上把它显示成目录树，是为了照顾人的习惯，实际存储时它就是一串字符。这个设计带来一个后果，按前缀列举很快，按“目录”操作不存在。想删除一个“目录”，得先列出所有以这个前缀开头的对象，逐个删。

桶的名字全局唯一，跨账号也不能重复，因为访问时要用它拼域名。腾讯云 COS 的域名形如 `example-1250000000.cos.ap-guangzhou.myqcloud.com`，其中前半段是桶名加 APPID，后半段是地域。AWS S3 的域名形如 `bucket-name.s3.region.amazonaws.com`。这个域名规则在排查访问问题时很重要，因为签名计算和域名绑定，写错地域就会报签名不匹配。

11.2 三种存储形态

块存储、文件存储、对象存储经常被放在一起比较，它们解决的是不同层次的问题。图 11-2 是这三者的示意。



三种形态对应三种用法：系统盘、共享协作目录、海量非结构化数据

图 11-2 块存储、文件存储与对象存储的区别

块存储提供的是一块裸设备，操作系统在上面建文件系统，用起来和本地磁盘一样。云厂商的云硬盘属于这一类。它的优点是延迟低、支持随机读写，适合数据库。缺点是同一时刻只能挂给一台机器，共享要靠集群文件系统。

文件存储提供的是共享目录，多台机器同时挂载，看到同一份数据。NFS、SMB 属于这一类，云上的 NAS 也是。它适合多台机器共享配置、共享代码，缺点是元数据操作多的时候性能下降明显。

对象存储提供的是 HTTP 接口，通过 PUT 和 GET 存取整个对象。它不适合放数据库文件，也不适合放需要频繁修改的文件。它适合放图片、视频、备份、日志、训练数据、模型权重，这些内容写进去之后很少改，但量很大。

判断用哪种存储，可以问三个问题。需要随机读写吗，需要多机共享吗，数据量大到单机放不下吗。数据库用块存储，共享配置用文件存储，海量冷数据用对象存储。

11.3 接口就是 HTTP

对象存储的接口是 REST 风格的 HTTP。上传一个对象用 PUT，下载用 GET，删除用 DELETE，查元数据用 HEAD。这几个方法都是幂等的，同一个 PUT 执行两次，结果一样，这对重试很友好。

```
curl -X PUT "https://bucket-1250000000.cos.ap-guangzhou.myqcloud.com/test.txt" \
     -H "Authorization: ..." \
     --data-binary "hello"
```

返回的状态码能说明大部分问题。200 表示成功，204 表示成功但没有内容，403 表示没有权限或者签名不对，404 表示对象不存在，409 表示冲突（比如桶名已被占用），412 表示前置条件失败，429 表示请求过于频繁，5xx 表示服务端问题。排错时先看状态码，再看返回的 XML 里的错误码，后者更具体。

```
<?xml version="1.0" encoding="UTF-8"?>
<Error>
  <Code>SignatureDoesNotMatch</Code>
  <Message>The request signature we calculated does not match...</Message>
  <Resource>/test.txt</Resource>
  <RequestId>...</RequestId>
</Error>
```

`Code` 字段是最有价值的信息，第十二章会把常见的几个错误码和它们的成因列出来。

还有一个容易忽略的方法 HEAD。它只返回头部，不返回内容，可以用来快速判断对象是否存在、有多大、是什么类型，不用把整个对象下载下来。

```
curl -I "https://bucket-1250000000.cos.ap-guangzhou.myqcloud.com/big-model.bin"
```

11.4 大文件怎么传

对象存储的单个 PUT 有大小限制，通常是 5 GB。超过这个大小必须用分片上传。即使没超，大文件用分片上传也更可靠，因为某一片失败只需要重传那一片。

分片上传分三步，如图 11-3。先发起一个分片上传请求，服务端返回一个 UploadId。然后按顺序或者并发上传各个分片，每片带上序号，服务端返回每片的 ETag。最后提交完成请求，把所有分片的序号和 ETag 列出来，服务端按顺序拼接成一个对象。



图 11-3 分片上传的三个阶段

这套机制有两个细节值得知道。第一，分片有大小限制，除了最后一片，每片不能小于 1 MB（不同厂商的数字略有差异，S3 是 5 MB），最大不能超过 5 GB。第二，未完成的分片上传会占用存储空间，因为已经上传的分片还在服务端。所以要配置生命周期规则，自动清理超过一定天数还没完成的分片，否则存储费用会莫名其妙地上涨。

分片上传还支持断点续传。客户端把 UploadId 和已完成的分片列表存下来，下次从断点继续。`coscli`、`aws s3 cp` 这些工具内部都实现了这套逻辑，命令行看起来是一条命令，背后可能是几十个并发请求。

11.5 元数据与存储类型

每个对象除了数据，还有一组元数据。系统元数据由服务端维护，包括大小、修改时间、ETag、内容类型。用户元数据是自定义的键值对，以 `x-cos-meta-` 或者 `x-amz-meta-` 开头，可以用来记录业务信息。

```
x-cos-meta-project: video-2dto3d
x-cos-meta-owner: alice
```

ETag 常被误认为是文件的 MD5。对普通上传的单文件，它确实是内容的 MD5；对分片上传的对象，它是各分片 MD5 拼接后再算 MD5，和整个文件的 MD5 不同。用它做完整性校验时要小心，比较靠谱的做法是自己在本地算 MD5 或者 SHA-256，存进用户元数据。

存储类型决定了费用和取回时间。标准存储适合频繁访问的数据，低频存储便宜一些但取回要等，归档存储更便宜但要解冻几个小时。这些类型可以按规则自动转换，比如“创建 30 天后转低频，90 天后转归档”，这就是生命周期规则。

11.6 一致性与持久性

对象存储宣传的持久性通常是“11 个 9”，意思是每年丢失数据的概率是千亿分之一。这个数字来自冗余设计，同一份数据存多个副本或者用纠删码编码后分散到多台机器、多个机架，坏掉几个不影响整体。

一致性是另一个维度。早期的 S3 只保证最终一致，上传之后立刻读可能读到旧版本。2020 年底 AWS 宣布 S3 对新建对象的读写提供了强一致性，覆盖列表操作。国内厂商的文档里也有类似说明，但具体行为要看版本。这一点对训练任务有实际影响，上传完数据立刻开始训练，如果读到的是旧版本或者读不到，会很困惑。稳妥的做法是上传后校验，或者等几秒再开始。

11.7 工具与挂载

日常使用对象存储有三类工具。

图形界面的客户端，用来浏览桶、上传下载、生成分享链接。这类工具适合偶尔用，不适合批量操作。

命令行工具，比如 `coscli`、`aws s3`、`rclone`。它们支持并发、断点续传、同步目录、按规则过滤，是脚本里的主力。

```
coscli cp ./data/ cos://bucket-1250000000/dataset/ --recursive
coscli sync ./checkpoints/ cos://bucket-1250000000/ckpt/ --delete
```

挂载工具，比如 `cosfs`、`s3fs`、`goofys`。它们把桶挂成一个本地目录，应用程序不用改代码就能读写。这看起来很美好，坑却不少。

第一个坑是延迟。每次读文件都是一次 HTTP 请求，随机读小文件的性能比本地磁盘差两个数量级。第二个坑是随机写。对象存储不支持在文件中间插入数据，很多程序（比如数据库、某些编译工具）会先写临时文件再改名，这种模式在挂载的文件系统上可能失败或者产生额外开销。第三个坑是文件锁。对象存储没有锁的概念，多台机器挂载同一个桶，同时写同一个文件会互相覆盖。

结论是，挂载适合“读多写少、顺序访问”的场景，比如让训练脚本直接读数据集。不适合放数据库、不合适放代码仓库、不合适放需要频繁 rename 的编译输出。

11.8 和堡垒机的关系

这两件事看起来无关，其实在企业里经常连在一起。

堡垒机的会话录像要存起来，本地磁盘存不下六个月的数据，于是存到对象存储。训练任务

的日志和模型权重也要存，于是也放对象存储。数据的入口和人的入口，构成了运维体系的两端。

这也带来一个安全上的共同点。两者都依赖凭据，堡垒机托管服务器的密码和私钥，对象存储依赖访问密钥。凭据泄露的后果一样严重。第十二章要讲的签名机制，本质上就是让凭据在使用时不出现在网络上，这一点和 SSH 的公钥认证思路相同。

第 12 章

签名、密钥与 403

12.1 每个请求都要证明自己

对象存储的接口是 HTTP，服务器不记得你上次来过。每个请求都得自己带上身份证明，否则任何人都能读写你的数据。这种证明就是签名。

签名的思路和 SSH 的公钥认证有相似之处。客户端持有一把密钥，服务端也存一份，客户端用密钥对请求内容算出一个值，附在请求里，服务端用同样的方法算一遍，比对是否一致。差别在于 SSH 用的是非对称签名，对象存储用的是对称的 HMAC，因为双方共享同一把密钥，不需要公钥基础设施。

这套机制有两个直接后果。第一，密钥绝不能泄露，它不像私钥那样有配对的公钥可以公开，拿到它就等于拿到全部权限。第二，请求的内容参与了签名，改一个字节签名就对不上，所以中间人无法篡改。

12.2 访问密钥

对象存储的凭据叫访问密钥，由两部分组成，一个公开的 ID 和一个保密的 Key。AWS 叫 Access Key ID 和 Secret Access Key，腾讯云叫 SecretId 和 SecretKey。ID 用来标识身份，Key 用来算签名。ID 可以出现在日志里，Key 一次都不能。

```
SecretId  = AKIDexampleExampleExampleExample
SecretKey = exampleSecretKeyDoNotUseThisInRealLife
```

凭据分几类。主账号的密钥权限最大，能操作账号下所有资源，绝不该写进代码或者放到客户端。子账号的密钥按策略授权，能限定到某个桶、某个前缀、某些操作。临时密钥由安全令牌服务签发，带有效期，用完自动失效，适合分发给临时任务或者前端。

这里有一个常见的错误。有人把密钥写进前端页面的 JavaScript 里，以为别人看不到。前端代码是公开的，任何人打开开发者工具就能读出来。正确做法是后端签发一个临时密钥或者预签名 URL，让前端拿着有限权限的凭据去操作。

12.3 签名的计算过程

对象存储的签名算法在业界基本统一，AWS 的叫 Signature Version 4，简称 SigV4，其他厂商大多兼容或借鉴。它的计算分四步，核心思想是把请求的各个部分按固定格式拼成一个字符串，然后逐层做 HMAC。

第一步，构造规范化请求。把 HTTP 方法、路径、查询参数、头部按规则整理成固定格式，

再拼上请求体的哈希。

```
CanonicalRequest =
  HTTPRequestMethod + '\n' +
  CanonicalURI + '\n' +
  CanonicalQueryString + '\n' +
  CanonicalHeaders + '\n' +
  SignedHeaders + '\n' +
  HexEncode(Hash(Payload))
```

图 12-1 是这个拼接过程的示意。

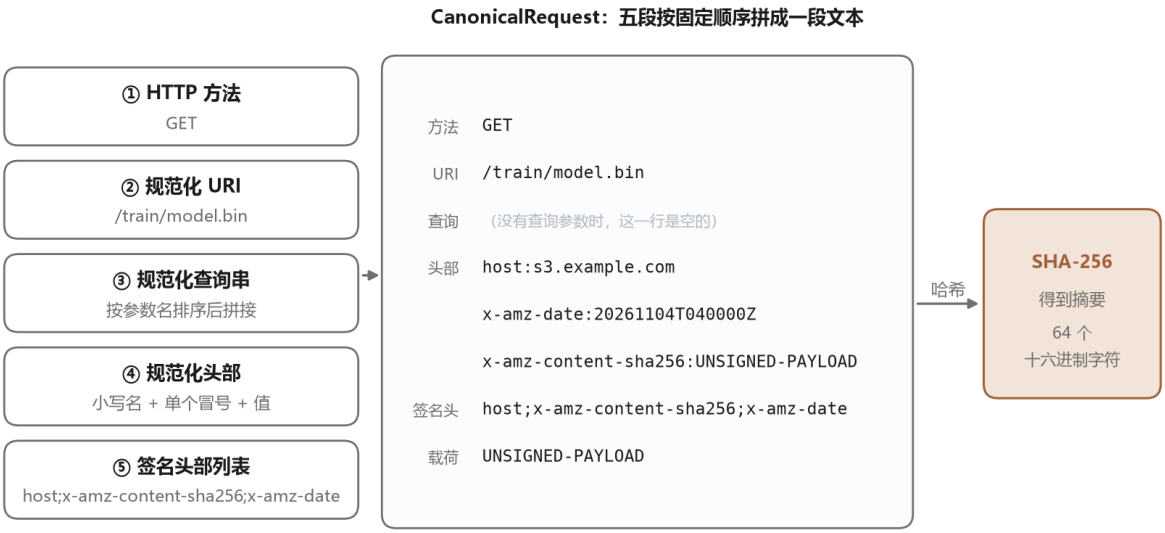


图 12-1 规范化请求的五段内容

每一步都有细节。URI 要做百分号编码，但斜杠不编码。查询参数要按参数名的字典序排列，同名的按值排序。头部的名字要转小写，值要去掉首尾空格、把连续空格压成一个，然后按名字排序。这些规则看起来繁琐，目的是让不同语言、不同库算出的结果一致。签名对不上，十次里有八次是某一步的规范化没做对。

第二步，构造待签字符串。把算法名、时间戳、凭据范围、上一步的哈希拼起来。

```
StringToSign =
  "AWS4-HMAC-SHA256" + '\n' +
  "20261104T040000Z" + '\n' +
  "20261104/ap-guangzhou/s3/aws4_request" + '\n' +
  HexEncode(Hash(CanonicalRequest))
```

第三步，派生签名密钥。这一步是四层 HMAC，每一层用上一层的输出作为密钥。

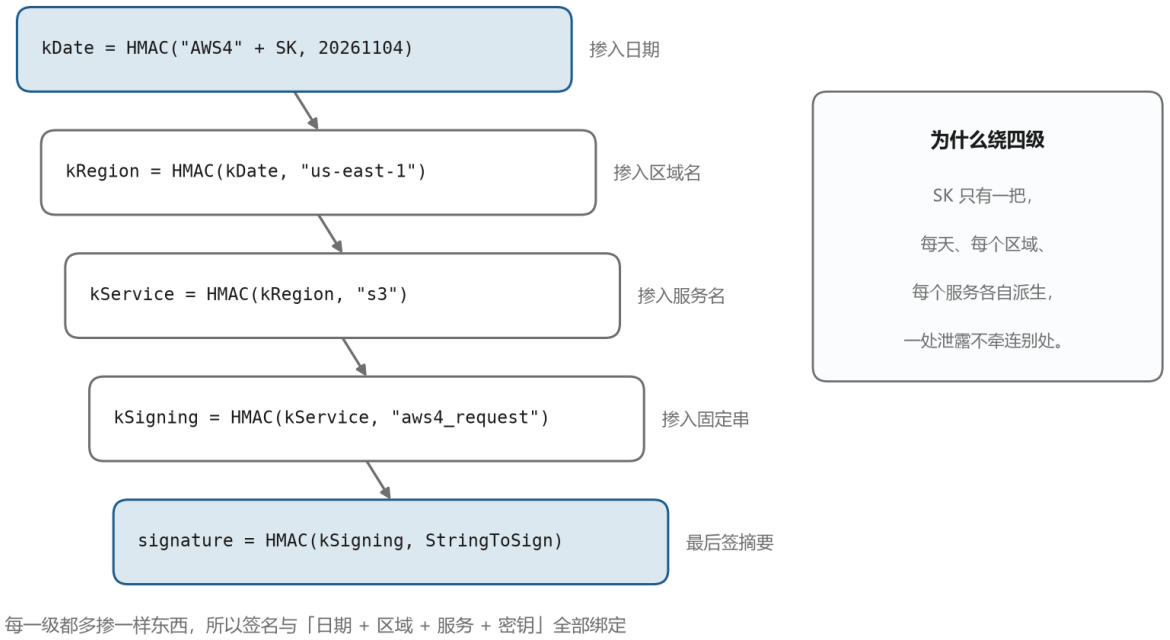


图 12-2 签名密钥的四层派生与最终签名

```
kDate    = HMAC("AWS4" + SecretKey, "20261104")
kRegion  = HMAC(kDate,    "ap-guangzhou")
kService = HMAC(kRegion,  "s3")
kSigning = HMAC(kService, "aws4_request")
```

为什么要派生而不是直接用 SecretKey，原因有两个。一是把密钥和日期、地域、服务绑定，同一个 SecretKey 在不同场景下算出的签名密钥不同，一处泄露不影响其他场景。二是让签名密钥每天变化，减少长期密钥被分析的风险。

第四步，算出签名并放进请求头。

```
Signature = HexEncode(HMAC(kSigning, StringToSign))

Authorization: AWS4-HMAC-SHA256
  Credential=AKID.../20261104/ap-guangzhou/s3/aws4_request,
  SignedHeaders=host;x-amz-content-sha256;x-amz-date,
  Signature=abcdef...
```

服务端收到请求后，用自己存的 SecretKey 重算一遍。算出来的值和你给的一致，就认为请求合法。整个过程中 SecretKey 从未出现在网络上，出现的只有它的运算结果。

这段推导看起来复杂，实际写代码时都用 SDK，不需要手工实现。但理解它有两个好处。一是看到 `SignatureDoesNotMatch` 时知道该检查什么，通常是路径编码、头部排序或者时间戳。二是能理解预签名 URL 的原理，它就是把签名结果放进查询参数。

12.4 预签名 URL

有时候你需要让别人临时访问一个对象，但不能把密钥给他，也不想让他走你的服务器中转。这种场景用预签名 URL。

```
https://bucket-1250000000.cos.ap-guangzhou.myqcloud.com/report.pdf
  ?X-Amz-Algorithm=AWS4-HMAC-SHA256
  &X-Amz-Credential=AKID...%2F20261104%2Fap-guangzhou%2Fs3%2Faws4_request
  &X-Amz-Date=20261104T040000Z
  &X-Amz-Expires=3600
  &X-Amz-SignedHeaders=host
  &X-Amz-Signature=abcdef...
```

图 12-3 是这段 URL 的组成。



图 12-3 预签名 URL 的字段构成

原理是把 Authorization 头里的内容挪到查询参数里，签名时把这些参数也算进去。任何拿到这个 URL 的人，在有效期内都能按指定的方法访问这个对象，过期就失效。签名的范围可以限定得很细，只允许 GET、只允许某个对象、只在一小时内有效。

预签名 URL 有几个坑。第一，它是一串明文，谁拿到谁就能用，所以要通过安全的渠道传递，不要贴在公开的群聊里。第二，有效期不能太长，几天甚至几个月的有效期等于长期后门。第三，URL 里的签名基于当时的头部和参数，如果中间有代理改写了头部（比如加了 `X-Forwarded-For`），签名可能对不上，这时要把那些头部从 `SignedHeaders` 里去掉。

12.5 临时密钥

比预签名 URL 更灵活的方式是临时密钥。用一个长期密钥去换一组短期的凭据，包含临时的 ID、Key 和一个会话令牌（Token）。请求时要把令牌放在 `x-amz-security-token` 头里，签名时也要把它算进去。

```
sts assume-role --role-arn ... --duration 3600   # pseudo command, syntax varies
```

临时密钥的好处是权限可控、自动过期、可以追溯。给一个训练任务发一组只在两小时内有效的密钥，任务结束后密钥自动失效，即使泄露了也没有长期风险。企业里的最佳实践是主密钥只在签发临时密钥的服务里使用，业务代码一律用临时密钥。

12.6 权限是怎么判的

对象存储的权限有三个层次。最外层是身份策略，绑定在子账号或者角色上，规定这个身份能做什么。中间是桶策略，挂在桶上，规定谁能对这个桶做什么，可以指定其他账号。最内层是 ACL，挂在桶或者对象上，粒度最粗，通常只在需要公共读的时候用。

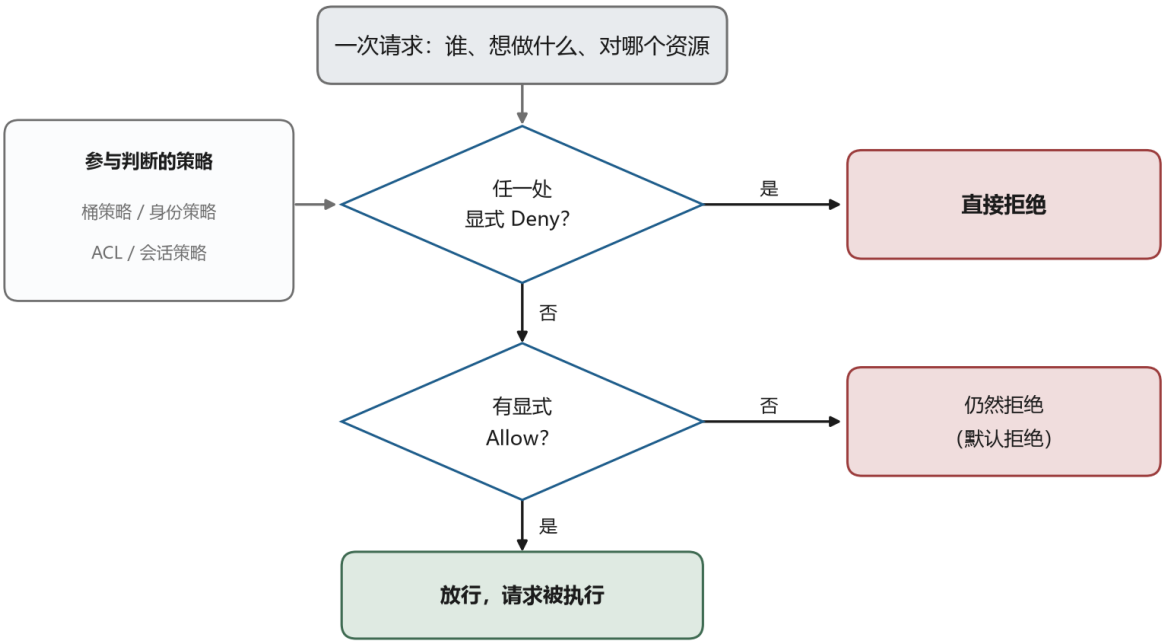


图 12-4 权限求值的顺序，显式拒绝优先

求值顺序如图 12-4。任何一个层次出现显式拒绝，请求就被拒绝，其他层次的允许都不作数。没有任何拒绝，但只要有一处显式允许，请求就通过。既没有拒绝也没有允许，默认拒绝。

这个顺序解释了一个常见困惑。你在身份策略里给了 `s3:*`，为什么还是 403。因为桶策略里有一条显式拒绝，或者桶的公共访问被组织级策略锁死了。排查时要把三个层次都看一遍，而不是只改自己手里那个。

另一个常见困惑是权限的粒度。对象存储的权限可以精确到前缀，比如只允许访问 `dataset/` 下面的对象，不允许访问 `ckpt/`。这在多人共享一个桶时很有用，每人一个前缀，互不干扰。配置时注意前缀要以斜杠结尾，否则 `dataset` 会匹配到 `dataset-backup`。

12.7 403 家族

表 12-1 是几个最常遇到的错误码和它们的成因。看到错误码先查这张表，能省很多时间。

表 12-1 对象存储常见错误码

错误码	含义	常见原因
SignatureDoesNotMatch	签名不匹配	路径编码不对、头部没排序、地域写错、SecretKey 错
RequestTimeTooSkewed	请求时间偏差过大	本机时钟不准，与服务器相差超过 15 分钟
InvalidAccessKeyId	密钥 ID 不存在	密钥被删、复制时漏了字符、用了别的账号的密钥
AccessDenied	没有权限	策略没授权、被显式拒绝、桶策略限制来源 IP
NoSuchBucket	桶不存在	桶名拼错、地域写错、桶已被删除
NoSuchKey	对象不存在	键名拼错、前缀不对、对象已过期被生命周期删除
InvalidRequest	请求格式错误	分片上传的片序号越界、Content-Length 与实际不符

地域写错是最隐蔽的一个。COS 的桶在某个地域，请求必须发到那个地域的域名，签名里的凭据范围也要写那个地域。如果你的 SDK 配置成了 `ap-shanghai`，桶却在 `ap-guangzhou`，请求会先被重定向，签名随之失效，报出来的错却是 `SignatureDoesNotMatch`。遇到这个错误先检查地域，再检查密钥。

时钟偏差是第二隐蔽的。虚拟机的时钟偶尔会漂移，尤其是长期挂起的实例恢复之后。签名里带了时间戳，服务端允许的正负偏差通常是 15 分钟，超出就拒绝。

```
date -u
sudo systemctl restart systemd-timesyncd
```

这两条命令能确认和修复时间问题。云端机器建议开启 NTP 同步，避免这类莫名其妙的失败。

12.8 几条实践建议

把这一章的内容收拢成几条可执行的建议。

密钥分级。主密钥只放在签发临时凭据的服务里，业务用子账号密钥，临时任务用临时密钥。任何密钥都不要写进代码仓库，用环境变量或者密钥管理服务注入。

权限最小化。给一个任务只授它需要的桶和前缀，只给 GET 就不给 PUT。上线前用一组只读密钥跑一遍，确认没有多余的写操作。

打开访问日志。桶的访问日志能记录每个请求的来源、操作、结果，出问题时是唯一的证据。日志本身也存到一个单独的桶里，设置较长的保留期。

关闭公共读。除非确实要做公开的资源分发（比如网站静态文件），否则桶一律私有。公共读的桶会被爬虫和扫描器盯上，数据泄露往往就是这么发生的。

定期轮换。长期密钥每季度换一次，换的时候先在代码里加新密钥，确认生效后再停用旧的。临时密钥的有效期尽量短，够用就行。

第 13 章

把四件事串成一条链路

13.1 一个真实的工作流

前面十二章各自讲了一个部件，这一章把它们装到一起。

设想这样的场景。你在办公室的 Windows 电脑上写代码，代码要跑在实验室的一台 GPU 服务器上，那台服务器在机房内网，外面进不去，只有一台堡垒机暴露在公网。训练数据有几百 GB，放在对象存储的一个桶里，模型权重也要定期存回桶里。你希望能在本地的 VS Code 里编辑远端代码、开终端、跑训练，同时让训练脚本自己从桶里读数据、往桶里写检查点。

这个场景里用到了四样东西，SSH 客户端、堡垒机、VS Code 的远程开发、对象存储的 API。它们分别对应这本书的四篇。图 13-1 是整体的拓扑。

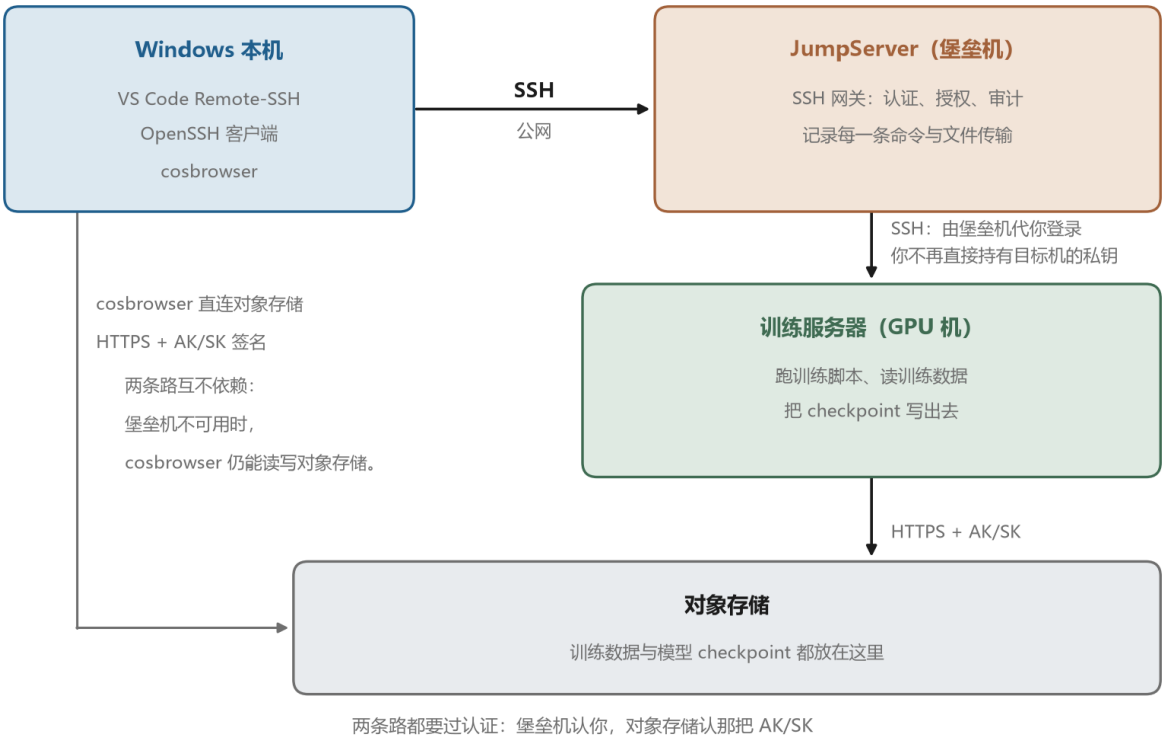
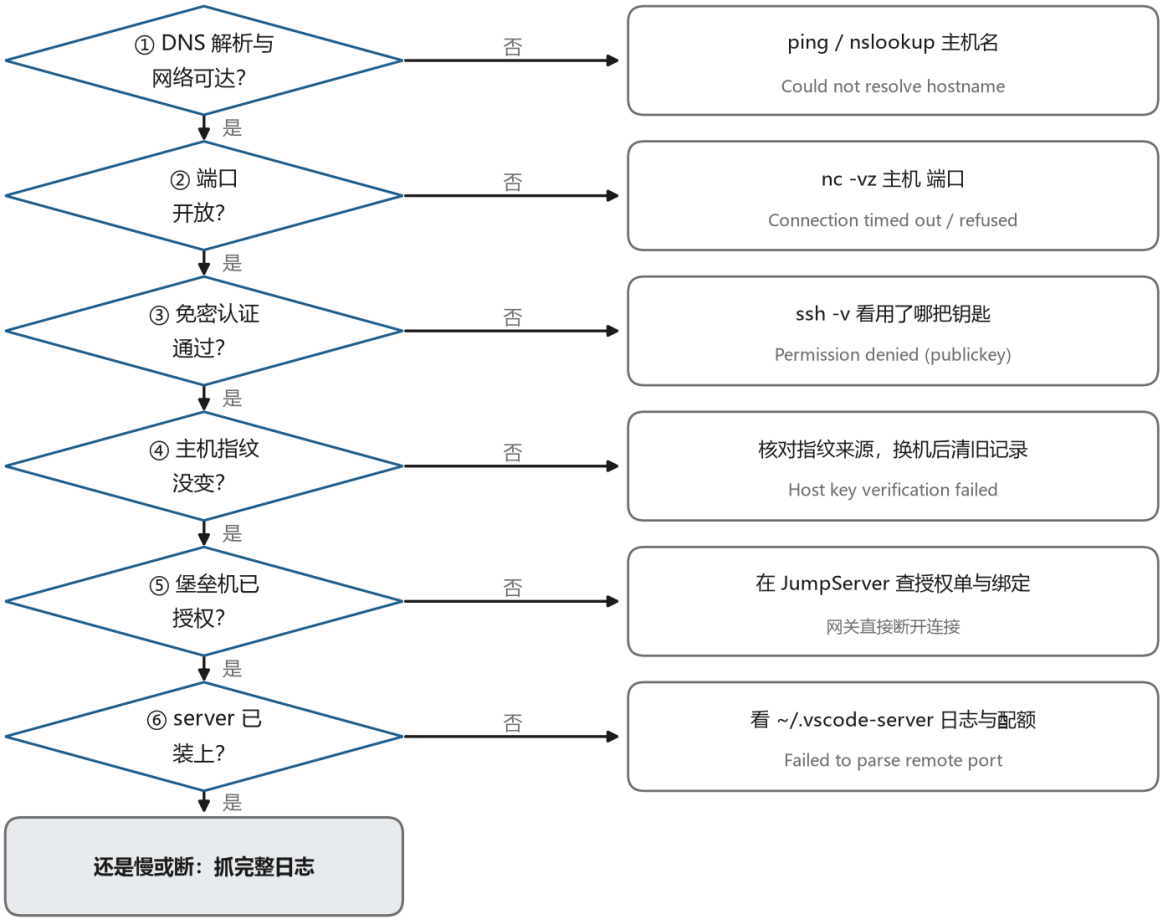


图 13-1 从本机到训练机再到对象存储的完整链路

图里有四条链路，每条都有自己的认证方式。本机到堡垒机是一次 SSH 认证，用的是你的私钥。堡垒机到训练机是另一次认证，用的是堡垒机托管的凭据，你看不到。本机到训练机之间还有一条逻辑上的 VS Code 通道，它跑在前两条连接之上。训练机到对象存储是一次 HTTPS 请求，用的是访问密钥的签名。

把这四条链路的认证对象分清楚，排错时就不会乱。图 13-2 是本机侧常见问题的判断路径，后面几节会用到。



按顺序问下去，第一个答「否」的地方就是问题所在

图 13-2 从连不上到具体原因的判断路径

13.2 第一步，把登录打通

一切从一条能用的 SSH 连接开始。在本机建好配置文件，内容大致如下。

```
Host lab-gpu
    HostName 124.221.133.165
    Port 58222
    User alice@ubuntu@10.0.0.9
    IdentityFile C:/Users/alice/.ssh/ssh.jumpserver.pem
    IdentitiesOnly yes
    ServerAliveInterval 30
    ServerAliveCountMax 3
```

五行核心配置，第一章已经逐行拆过。这里补充三行可选项。`IdentitiesOnly yes` 让客户端只用指定的这把钥匙，不去试 agent 里的其他钥匙，避免因为尝试次数过多被服务器拒绝。两个 `ServerAlive` 选项负责保持连接，防止中间设备把闲置的会话清掉。

配好之后先别急着开 VS Code，在终端里试一下。

```
ssh -v lab-gpu
```

能进到远端的提示符，说明前三段链路都通了。进不去的话，按图 13-2 从上往下查。先确认域名或者地址能通，再确认端口开放，再看认证。

```
Test-NetConnection 124.221.133.165 -Port 58222
```

这条命令在 Windows 上测端口连通性，比 `ping` 靠谱，因为很多服务器禁 ping 但端口是开的。如果端口不通，问题在网络或者安全组，和密钥无关。

认证失败时，报错通常是 `Permission denied (publickey)`。这时依次检查四件事，私钥文件是否存在、权限是否过宽、公钥是否已经登记在堡垒机、用户名里的三段是否写对。第三件事最容易被忽略，很多人把公钥加到了目标机的 `authorized_keys` 里，却忘了在堡垒机上登记，结果怎么都不通。

13.3 第二步，让编辑器连上

终端能登录之后，VS Code 的远程连接通常一次就成。装好 Remote-SSH 扩展，按 `F1` 调出命令面板，选“连接到主机”，输入 `lab-gpu`。

第一次连接会慢一些，因为要在远端安装 `vscode-server`。如果远端的家目录所在分区空间紧张，这一步会失败。先看一下余量。

```
df -h ~
```

装好之后，界面左下角会显示远端的主机名，表示已经连上。这时打开远端的项目目录，编辑、保存、开终端都在远端进行。你的本机只负责显示。

如果卡在“正在下载 VS Code Server”，说明远端下载失败。可以在本机下载对应的安装包，用 `scp` 传上去手工解压。安装包的地址在输出面板的日志里能找到，形如 `https://update.code.visualstudio.com/commit/<commit-id>/server-linux-x64/stable`。

```
wget https://update.code.visualstudio.com/commit/<commit-id>/server-linux-x64/stable
mkdir -p ~/.vscode-server/bin/<commit-id>
tar -xzf stable -C ~/.vscode-server/bin/<commit-id> --strip-components=1
```

这条路在离线环境里很常用，值得记下来。`<commit-id>` 在 VS Code 的“关于”对话框里能找到。

13.4 第三步，让训练机读写桶

训练脚本要读写对象存储，需要在训练机上配置凭据。这一步最容易出事，因为凭据一旦落到磁盘上，就成了一条长期风险。

最稳妥的做法是用临时密钥。用一个有签发权限的身份去换一组只在几小时内有效的凭据，注入到环境变量里，不写文件。

```
export COS_SECRET_ID="<TEMP_SECRET_ID>"
export COS_SECRET_KEY="<TEMP_SECRET_KEY>"
export COS_SESSION_TOKEN="<TEMP_SESSION_TOKEN>"
```

如果没有临时密钥的签发能力，退一步用子账号密钥，把它写进只有自己能读的配置文件，权限 600。

```
mkdir -p ~/.coscli
cat > ~/.coscli/config.yaml <<'EOF'
cos:
  base:
    secretid: <YOUR_SECRET_ID>
    secretkey: <YOUR_SECRET_KEY>
    sessiontoken: ""
  buckets:
    - name: dataset-1250000000
      alias: dataset
      region: ap-guangzhou
EOF
chmod 600 ~/.coscli/config.yaml
```

配置好之后先做一次最小验证，列一下桶里的内容，确认权限没问题。

```
coscli ls cos://dataset/
coscli ls cos://dataset/train/ --limit 20
```

能列出来，说明读权限有了。再试一次写，往一个临时前缀里传一个小文件。

```
echo test > /tmp/probe.txt
coscli cp /tmp/probe.txt cos://dataset/tmp/probe.txt
coscli rm cos://dataset/tmp/probe.txt
```

这三条命令验证了列、写、删三个动作。很多 403 就是在这个阶段暴露出来的，早发现比训练跑到一半才发现好。

13.5 第四步，把结果取回来

训练结束，检查点和日志要取回来。小文件直接下载，大文件用分片加快，中断了能续。

```
coscli cp cos://dataset/ckpt/epoch-10.pt ./ckpt/epoch-10.pt
coscli sync cos://dataset/logs/ ./logs/ --include "*.txt"
```

`sync` 只传输有差异的文件，适合日志这种持续增长的内容。`--include` 和 `--exclude` 支持通配符，可以精确控制传哪些。传大文件时注意本机磁盘空间，一个模型检查点几个 GB 很常见。

如果要把结果分享给别人，不要直接给桶的公共链接，用预签名 URL，设一个合理的有效期。

```
coscli sign cos://dataset/ckpt/epoch-10.pt --expire 3600
```

这条命令返回一个带签名的链接，一小时后失效。分享之前确认链接里没有敏感信息，因为 URL 里的路径本身就是对象名。

13.6 排错的时候按顺序来

图 13-2 的决策树可以展开成一张检查清单。遇到问题，从上往下逐项确认，不要跳步。

先看能不能通。`Test-NetConnection` 测端口，`ping` 测地址，`tracert` 看路径。这一步不通，后面全是白费。

再看认证。`ssh -v lab-gpu` 的输出里找 `Authentications that can continue`，它列出了服务器接受的认证方式。如果只有 `publickey`，说明密码登录被禁用，得用密钥。如果连这一行都没有，问题在更早的阶段。

然后看授权。堡垒机的授权是独立于 SSH 认证的一层，SSH 认证通过不代表你有权限连某台资产。表现是连接建立后立刻被断开，或者提示没有权限。这时要去堡垒机界面确认授权是否包含这台资产和这个系统用户。

接着看隧道。VS Code 连上但界面转圈，多半是端口转发被禁用。在输出面板里找 `Remote-SSH` 的日志，如果看到 `open failed: administratively prohibited`，就是这个原因。

最后看对象存储。`SignatureDoesNotMatch` 查地域和密钥，`AccessDenied` 查策略，`RequestTimeTooSkewed` 查时钟。三条线索对应三个不同的层面，不要混着改。

13.7 一份可以照着做的检查清单

把全书的安全要点收成一份清单，部署或者接手一个新环境时可以逐条对照。

凭据方面。私钥权限 600，目录 700。密钥设口令，口令进 agent。不用共享账号，一人一把钥匙。离职当天回收授权，不只是删机器上的公钥，还要删堡垒机上的账号。对象存储的主密钥不进代码，业务用子账号，临时任务用临时密钥。

连接方面。关闭密码登录，只留公钥。关闭 root 直接登录。改默认端口能减少噪音但不是安全保障。`known_hosts` 的警告要认真看，不要习惯性删掉。

堡垒机方面。默认禁止端口转发和复制粘贴，按需开放。开启会话录像和命令记录，保留期不少于六个月。录像存到对象存储，别放本地磁盘。堡垒机本身单独划分安全域，只对运维网段开放。

对象存储方面。桶一律私有，公共读要单独评估。开启访问日志。配置生命周期，清理未完成的分片和过期对象。跨域规则只对需要的域名开放，不要写 `*`。

这些条目看起来琐碎，每一条背后都有具体的事故。把它们做成检查表，新环境上线前过一

遍，比事后复盘便宜得多。

13.8 最后

这本书从一台机器讲到了一群机器，从人的入口讲到了数据的出口。中间经过了网络分层、密码学、协议握手、堡垒机的权限模型、编辑器的进程结构、对象存储的签名算法。这些内容看起来分散，其实都在回答同一个问题，一个远方的资源，凭什么相信你，你又凭什么相信它。

SSH 用密钥和签名回答这个问题，堡垒机用集中管理和审计回答这个问题，对象存储用 HMAC 签名回答这个问题。手段不同，思路一致。理解了这一点，再遇到新的工具、新的协议，你也能顺着这条线索把它拆开。

剩下的就是动手。把配置文件改一改，把 `ssh -v` 的输出读一读，把桶的权限调一调，看看会发生什么。工具的价值在使用中才会显现，而这些原理的价值，在于让你在使用时知道自己在做什么。

附录 A 命令速查

A.1 客户端连接

命令	作用
`ssh host`	用配置文件里的别名连接
`ssh -p 2222 user@host`	指定端口连接
`ssh -i key.pem user@host`	指定私钥
`ssh -v host`	打印调试信息，`-vv -vvv` 更详细
`ssh -N -L 8080:10.0.0.9:80 jump`	只做转发，不登录 shell
`ssh -J jump1,jump2 target`	两级跳板
`ssh -D 1080 jump`	起一个 SOCKS5 代理
`ssh -T host`	不分配伪终端，适合脚本
`ssh -t host cmd`	强制分配伪终端
`ssh -o Option=Value host`	临时指定配置项
`ssh -G host`	打印最终生效的配置，不发起连接

A.2 密钥管理

命令	作用
`ssh-keygen -t ed25519 -C "note"`	生成 Ed25519 密钥
`ssh-keygen -t rsa -b 4096`	生成 4096 位 RSA 密钥
`ssh-keygen -l -f key.pub`	查看公钥指纹
`ssh-keygen -y -f key`	从私钥导出公钥
`ssh-keygen -p -f key`	修改私钥口令
`ssh-keygen -p -m PEM -f key`	转换成 PEM 格式
`ssh-keygen -R host`	从 known_hosts 删除某主机的记录
`ssh-keygen -F host`	在 known_hosts 里查找记录

命令	作用
`ssh-copy-id -i key.pub user@host`	把公钥装到远端
`ssh-keyscan -t ed25519 host`	抓取主机公钥

A.3 agent

命令	作用
`eval "$(ssh-agent -s)"`	启动 agent
`ssh-add key`	把私钥加入 agent
`ssh-add -l`	列出已加载的钥匙
`ssh-add -D`	清空所有钥匙
`ssh-add -t 3600 key`	加载并在一小时后自动移除

A.4 诊断

命令	作用
`ss -lntp`	列出监听的 TCP 端口和进程
`ss -tnp`	列出已建立的连接
`nc -vz host port`	测试端口连通性
`curl -v telnet://host:port`	测试端口并看握手过程
`traceroute host`	逐跳显示路径
`tcpdump -n -i any port 22`	抓 SSH 流量
`journalctl -u ssh -n 50`	看 sshd 最近日志
`sudo sshd -T`	打印服务端生效配置

A.5 服务端

命令	作用
`systemctl status ssh`	查看 sshd 状态

命令	作用
`systemctl restart ssh`	重启 sshd
`sshd -t`	检查配置文件语法
`chmod 700 ~/.ssh`	修正目录权限
`chmod 600 ~/.ssh/authorized_keys`	修正公钥文件权限
`chmod 600 ~/.ssh/id_ed25519`	修正私钥权限

A.6 对象存储

命令	作用
`coscli ls cos://bucket/prefix/`	列出对象
`coscli cp local cos://bucket/key`	上传单个文件
`coscli cp cos://bucket/key local`	下载单个文件
`coscli cp dir/ cos://bucket/prefix/ -r`	递归上传
`coscli sync local/ cos://bucket/prefix/`	增量同步
`coscli rm cos://bucket/key`	删除对象
`coscli sign cos://bucket/key --expire 3600`	生成预签名 URL
`curl -I "https://bucket.cos.region.myqcloud.com/key"`	查看对象头部信息

A.7 常用配置项

配置项	客户端或服务端	说明
`HostName`	客户端	真实地址
`Port`	客户端	端口
`User`	客户端	登录名
`IdentityFile`	客户端	私钥路径
`IdentitiesOnly`	客户端	只用指定私钥
`ProxyJump`	客户端	跳板机
`ServerAliveInterval`	客户端	心跳间隔

配置项	客户端或服务端	说明
`LocalForward`	客户端	等价于 `-L`
`RemoteForward`	客户端	等价于 `-R`
`RequestTTY`	客户端	是否分配伪终端
`PasswordAuthentication`	服务端	是否允许密码登录
`PermitRootLogin`	服务端	是否允许 root 登录
`PubkeyAuthentication`	服务端	是否允许公钥登录
`AllowTcpForwarding`	服务端	是否允许端口转发
`GatewayPorts`	服务端	远程转发能否绑定非回环地址
`MaxSessions`	服务端	单连接最大通道数
`ClientAliveInterval`	服务端	服务端心跳间隔
`TrustedUserCAKeys`	服务端	用户证书的 CA 公钥

附录 B 术语表

SSH Secure Shell，一套用于在不安全网络上安全访问远端主机的协议族，由传输层、用户认证层、连接层三个子协议组成。

sshd SSH 的服务端程序，通常在 22 端口监听，接受连接、完成认证、分配会话。

OpenSSH 最广泛使用的 SSH 实现，包含客户端 `ssh`、服务端 `sshd`、密钥工具 `ssh-keygen` 等。

公钥认证 客户端用私钥对一段包含会话标识的数据签名，服务端用对应公钥验证，从而确认客户端持有私钥。

私钥 只能由持有者保存的密钥，用于签名或者解密，权限必须是 600。

公钥 可以公开分发的密钥，用于验证签名或者加密，以一行文本形式存放在 `authorized_keys` 中。

指纹 对公钥做哈希得到的短字符串，用来快速比对两把公钥是否相同。

known_hosts 客户端记录见过的服务器主机公钥的文件，用于检测主机身份变化。

TOFU Trust On First Use，首次使用即信任，第一次连接时记录主机公钥，之后比对。

主机密钥 服务器用于证明自己身份的一对密钥，安装 sshd 时自动生成。

authorized_keys 服务端文件，列出允许登录的公钥，每行一把，可在行首加限制选项。

ssh-agent 后台进程，保管解密后的私钥，替客户端完成签名，避免反复输入口令。

Agent Forwarding 把 agent 的签名能力转发到远端主机，方便连续跳转，但有被滥用的风险。

ProxyJump 先连跳板机再连目标的连接方式，第二段认证由本机完成。

端口转发 把某个 TCP 端口上的流量通过 SSH 通道转送到另一端，分本地、远程、动态三种。

SOCKS 一种代理协议，客户端在运行时指定目标地址，常用于动态转发。

伪终端 用一对虚拟设备模拟终端，master 端由 sshd 持有，slave 端交给 shell。

对称加密 加密与解密使用同一把密钥，速度快，用于加密实际数据。

非对称加密 使用公钥与私钥一对密钥，速度慢，用于密钥交换与签名。

AES 最常见的对称加密算法，分组长度 128 位，密钥长度 128、192 或 256 位。

ChaCha20 流密码，在没有 AES 硬件加速的设备上性能更好。

GCM 一种认证加密模式，同时提供加密与完整性校验。

哈希 把任意长度输入压成固定长度输出的函数，不可逆，用于比对与指纹。

SHA-256 输出 256 位的哈希函数，SSH 与对象存储签名中广泛使用。

HMAC 带密钥的哈希，用于验证消息完整性，只有持有密钥的人能算出正确值。

Diffie-Hellman 一种密钥交换协议，双方各自算出相同的共享密钥，而共享密钥从不在网络上传输。

ECDH 基于椭圆曲线的 Diffie-Hellman，密钥更短，计算更快。

Curve25519 一条适合密钥交换的椭圆曲线，对应 SSH 算法 `curve25519-sha256`。

Ed25519 基于 Curve25519 的签名算法，密钥 32 字节，签名 64 字节。

RSA 基于大整数分解困难性的算法，可用于加密与签名，密钥通常 2048 位以上。

前向保密 长期密钥泄露后，过去的会话仍然无法解密，前提是使用了临时密钥交换。

重密钥 在连接使用一段时间或传输一定数据后重新协商密钥，不影响上层会话。

会话标识 第一次密钥交换产生的哈希，整条连接内不变，用于绑定签名与重密钥。

KEXINIT 算法协商消息，双方各发一次，列出各自支持的算法清单。

ETM Encrypt-then-MAC，先认证再解密的数据包处理顺序，OpenSSH 默认使用。

通道 SSH 连接内的逻辑数据流，一条连接可同时承载多个通道。

全局请求 独立于通道的请求，如请求服务端监听端口（远程转发）。

堡垒机 集中管控运维访问的设备，提供统一认证、授权、账号托管与操作审计。

4A 认证、授权、账号、审计四个能力的合称。

JumpServer 开源堡垒机，组件包括 Core、Luna、Koko、Lion、Magnus、Razor、Chen。

Koko JumpServer 的字符协议网关，处理 SSH、SFTP、Telnet。

系统用户 JumpServer 中用于连接资产的凭据模板，包含用户名、认证方式、密码或私钥。

资产授权 把用户、资产、系统用户、动作绑定在一起的记录。

三合一用户名 网关模式下 `用户@系统用户@资产地址` 形式的登录名。

会话录像 记录终端原始字节流与时间戳的文件，可回放当时的屏幕内容。

Remote-SSH VS Code 扩展，通过系统 ssh 客户端连接远端并在远端运行 `vscode-server`。

vscode-server VS Code 在远端运行的服务，提供扩展宿主、文件服务、终端服务。

对象存储 以 HTTP 接口存取整个对象的存储服务，无目录树，容量近乎无限。

桶 对象的容器，名字全局唯一，访问时用于拼域名。

键 对象的名字，可含斜杠，但不表示目录层次。

ETag 对象的标识，单文件上传时等于内容 MD5，分片上传时是分片 MD5 的组合。

分片上传 把大文件切成多片分别上传，最后合并，支持并发与断点续传。

预签名 URL 把签名放进查询参数的临时访问链接，在有效期内可用。

SigV4 AWS 的签名算法第四版，被多数对象存储兼容，核心是规范化请求加四层 HMAC。

规范化请求 把方法、路径、查询串、头部按固定规则整理成标准形式，是签名的输入。

访问密钥 对象存储的凭据，由公开的 ID 与保密的 Key 组成。

临时密钥 由安全令牌服务签发的短期凭据，带有效期与会话令牌。

生命周期规则 按时间自动转换存储类型或删除对象的规则。

纠删码 把数据编码成多份分片分散存放，坏掉若干份仍可恢复，用于提升持久性。

后 记

写这本书的起因，是一位朋友发来的那份配置指南。

指南本身没有问题，五行配置，照着填就能用。问题在于它没法回答“为什么”。为什么 `HostName` 不是目标机的地址，为什么用户名里有两个 at 符号，为什么端口是五位数。这些问题在指南里没有位置，因为它们不属于操作步骤，属于背景知识。

背景知识的特点是，平时用不上，卡住的时候又缺不得。它不成体系地散落在博客、问答、同事的口头传授里，每个人捡到一点，拼不出全貌。这本书想做的就是把这些碎片按一条线索串起来，让一个没有系统学过网络的人，也能顺着读下去。

写作过程中最大的困难不是技术细节，而是分寸。讲得太浅，读者还是不知道为什么；讲得太深，一半人会在第三章放弃。最后的处理办法是，每个概念都从它要解决的问题讲起，公式能不写就不写，但关键的那几个必须写清楚，比如密钥派生的六个字母、签名密钥的四层派生。这些地方含糊过去，后面就全是黑箱。

书里的插图都是用代码画出来的，没有一张是拍下来或者生成的图片。这么做的原因很实际，示意图最怕画错，而代码可以改，改完重新跑一遍就行。所有的地址、密钥、桶名都是示例，不指向任何真实资产。

感谢在写作过程中被我反复追问的几位同事，他们提供了 JumpServer 和 COS 在实际环境里的细节，也指出了初稿里好几处想当然的地方。感谢开源社区，OpenSSH、JumpServer、docsify 这些项目的文档和源码是这本书重要的参考。

技术书总有过时的一天。协议会演进，界面会改版，命令的参数会变。如果哪天你发现书里的某条命令跑不通了，先去看那个工具的手册，再用书里的原理去判断差异。原理的寿命比参数长，这也是这本书把大量篇幅花在原理上的原因。

Dr. Dezeming

2026 年秋

这本书讲什么

一条 ssh 命令背后有多少层协议，为什么要先加密再认证，
堡垒机的“用户@系统用户@资产”是怎么被解析的，
VS Code 在你和目标机之间架了什么，
对象存储的签名为什么总是对不上。

知道为什么这么做，配置错了能自己看出来，
报错信息能自己读懂，别人给的指南也能自己判断。

Dr. Dezeming

远程之门 · 2026